

OpenFrame ジョブ制御言語文法書

OpenFrame/Batch for MVS 7.1



Copyright © 2023 TmaxSoft Co., Ltd. All Rights Reserved.

文書情報

文書名: OpenFrame ジョブ制御言語文法書

発行日: 2023年12月29日

ソフトウェアバージョン: OpenFrame/Batch for MVS 7.1

ガイドバージョン: v3.1.3

Website

<http://www.tmaxsoft.com>

<https://technet.tmaxsoft.com>

Copyright Notice

Copyright © 2023 TmaxSoft Co., Ltd. All Rights Reserved.

Restricted Rights Legend

このソフトウェア（Tmax OpenFrame®）マニュアルの内容とプログラムは、日本国の著作権法および国際条約によって保護されています。マニュアルの内容とプログラムは、TmaxSoft Co., Ltd.との使用許諾契約書の下でのみ使用することができ、マニュアルは使用許諾契約で許可されている範囲を除いては、配布または複製することができません。TmaxSoftの書面による事前の承諾を得ることなく、このマニュアルの全部または一部を電子的または機械的な方法を問わず、転送、複製、配布したり、または二次的著作物を作成する等の行為を一切禁じます。

このソフトウェアのマニュアルとプログラムの使用許諾契約は、いかなる場合においても、マニュアル及びプログラムと関連する知的財産権（登録の有無を問わず）を譲渡するものと解釈されず、TmaxSoftのブランド、ロゴ、商標等の使用権限を与えるものではありません。

このマニュアルは、情報を提供する目的でのみ提供しており、これに伴う契約上の直接的ないしは間接的な責任を負わず、マニュアルの内容は法律上もしくは商業的な特定の条件が満たされることを保証しません。マニュアルの内容は、製品のアップグレード及び修正により、その内容が予告なく変更されることがあり、内容上の誤りがないことを保証しません。

Trademarks

Tmax®およびTmax OpenFrame®は、TmaxSoft Co., Ltd.の登録商標です。その他、記載されている会社名、製品名などは、各社の商標または登録商標です。

Open Source Software Notice

この製品には、OpenSSL、RSA Data Security, Inc.、Apache FoundationおよびJean-loup Gaillyと Mark Adlerによって開発またはライセンス取得されたオープンソフトウェアが含まれています。詳細は、製品ディレクトリの\${INSTALL_PATH}/license/oss_licensesに記載されている事項を参照してください。

目次

このガイドについて	xiii
第1章 JCLの紹介	1
1.1. 記述形式	1
1.1.1. JCL文	1
1.1.2. JES2 JCL文	4
1.2. オペランド	5
1.2.1. 位置オペランド	5
1.2.2. キーワード・オペランド	5
1.2.3. 記述規約	6
1.2.4. 例	8
1.3. JCL	9
1.3.1. ジョブの開始と終了	9
1.3.2. 記述手順	10
第2章 JCL文	13
2.1. 概要	13
2.2. CNTL文	14
2.3. DD文	14
2.3.1. アスタリスク(*)とDATA	19
2.3.2. ACCODE	20
2.3.3. AMP	20
2.3.4. AVGREC	20
2.3.5. BLKSIZE	21
2.3.6. BLKSZLIM	22
2.3.7. BURST	22
2.3.8. CCSID	22
2.3.9. CHARS	23
2.3.10. CHKPT	23
2.3.11. CNTL	23
2.3.12. COPIES	24
2.3.13. DATACLAS	24
2.3.14. DCB	25
2.3.15. DDNAME	28
2.3.16. DEST	29
2.3.17. DISP	30
2.3.18. DLM	32
2.3.19. DSID	33
2.3.20. DSNNAME/DSN	33
2.3.21. DSNTYPE	37
2.3.22. DUMMY	38
2.3.23. DYNAM	38

2.3.24.	EXPDT	38
2.3.25.	FCB	39
2.3.26.	FILEDATA	39
2.3.27.	FLASH	39
2.3.28.	FREE	40
2.3.29.	HOLD	40
2.3.30.	KEYLEN	41
2.3.31.	KEYOFF	41
2.3.32.	LABEL	42
2.3.33.	LGSTREAM	43
2.3.34.	LIKE	43
2.3.35.	LRECL	44
2.3.36.	MGMTCLAS	45
2.3.37.	MODIFY	46
2.3.38.	OUTLIM	46
2.3.39.	OUTPUT	47
2.3.40.	PATH	48
2.3.41.	PATHDISP	48
2.3.42.	PATHMODE	49
2.3.43.	PATHOPTS	50
2.3.44.	PROTECT	51
2.3.45.	RECFM	51
2.3.46.	RECORD	52
2.3.47.	REFDD	52
2.3.48.	RETPD	53
2.3.49.	RLS	54
2.3.50.	SECMODEL	54
2.3.51.	SEGMENT	54
2.3.52.	SPACE	54
2.3.53.	SPIN	56
2.3.54.	STORCLAS	56
2.3.55.	SUBSYS	57
2.3.56.	SYMBOLS	57
2.3.57.	SYSOUT	58
2.3.58.	TERM	59
2.3.59.	UCS	59
2.3.60.	UNIT	59
2.3.61.	VOLUME/VOL	60
2.4.	スペシャルDD文	62
2.4.1.	SYSIN DD	62
2.4.2.	SYSOUT DDとSYSPPRINT DD	62
2.4.3.	JOBLIB DDとSTEPLIB DD	63
2.4.4.	JOBCAT DDとSTEPCLAT DD	64

2.5.	ENDCNTL文	65
2.6.	EXEC文	65
2.6.1.	ACCT	67
2.6.2.	ADDRSPC	67
2.6.3.	CCSID	67
2.6.4.	COND	68
2.6.5.	DYNAMNBR	71
2.6.6.	MEMLIMIT	72
2.6.7.	PARM	72
2.6.8.	PGM	73
2.6.9.	PROC	75
2.6.10.	PERFORM	77
2.6.11.	RD	78
2.6.12.	REGION	78
2.6.13.	RLSTMOUT	78
2.6.14.	SPARM	79
2.6.15.	TIME	79
2.6.16.	記号パラメータ	80
2.7.	IF/THEN/ELSE/ENDIF文	81
2.8.	INCLUDE文	85
2.9.	JCLコマンド文	86
2.9.1.	S	87
2.10.	JCLLIB文	87
2.11.	JOB文	88
2.11.1.	ADDRSPC	90
2.11.2.	BYTES	90
2.11.3.	CARDS	91
2.11.4.	CCSID	91
2.11.5.	CLASS	91
2.11.6.	COND	92
2.11.7.	GROUP	94
2.11.8.	JESLOG	94
2.11.9.	LINES	94
2.11.10.	MEMLIMIT	95
2.11.11.	MSGCLASS	95
2.11.12.	MSGLEVEL	96
2.11.13.	NOTIFY	96
2.11.14.	PAGES	97
2.11.15.	PASSWORD	97
2.11.16.	PERFORM	98
2.11.17.	PRTY	98
2.11.18.	RD	99
2.11.19.	REGION	99

2.11.20.	RESTART	100
2.11.21.	SECLABEL	100
2.11.22.	SCHENV	101
2.11.23.	SPARM	101
2.11.24.	TIME	102
2.11.25.	TIMECONTROLLER	103
2.11.26.	TYPRUN	105
2.11.27.	USER	108
2.12.	OUTPUT文	109
2.12.1.	ADDRESS	112
2.12.2.	AFPSTATS	113
2.12.3.	BUILDING	113
2.12.4.	BURST	114
2.12.5.	CHARS	114
2.12.6.	CKPTLINE	114
2.12.7.	CKPTPAGE	115
2.12.8.	CKPTSEC	115
2.12.9.	CLASS	115
2.12.10.	COLORMAP	116
2.12.11.	COMPACT	116
2.12.12.	COMSETUP	116
2.12.13.	CONTROL	116
2.12.14.	COPIES	117
2.12.15.	DATAACK	117
2.12.16.	DEFAULT	117
2.12.17.	DEPT	118
2.12.18.	DEST	118
2.12.19.	DPAGELBL	119
2.12.20.	DUPLEX	119
2.12.21.	FCB	119
2.12.22.	FLASH	119
2.12.23.	FORMDEF	120
2.12.24.	FORMLEN	121
2.12.25.	FORMS	121
2.12.26.	FSSDATA	121
2.12.27.	GROUPIX	121
2.12.28.	INDEX	122
2.12.29.	INTRAY	122
2.12.30.	JESDS	122
2.12.31.	LINDEX	122
2.12.32.	LINECT	123
2.12.33.	MAILBCC	123
2.12.34.	MAILCC	124

2.12.35.	MAILFILE	124
2.12.36.	MAILFROM	124
2.12.37.	MAILTO	125
2.12.38.	MODIFY	125
2.12.39.	NAME	126
2.12.40.	NOTIFY	127
2.12.41.	OFFSETXB	127
2.12.42.	OFFSETXF	127
2.12.43.	OFFSETYB	127
2.12.44.	OFFSETYF	127
2.12.45.	OUTBIN	128
2.12.46.	OUTDISP	128
2.12.47.	OVERLAYB	129
2.12.48.	OVERLAYF	130
2.12.49.	OVFL	130
2.12.50.	PAGEDEF	130
2.12.51.	PIMSG	131
2.12.52.	PORTNO	131
2.12.53.	PRMODE	131
2.12.54.	PRTATTRS	132
2.12.55.	PRTERORR	132
2.12.56.	PRTOPTNS	132
2.12.57.	PRTQUEUE	132
2.12.58.	PRTY	133
2.12.59.	REPLYTO	133
2.12.60.	RESFMT	133
2.12.61.	RETAINS	134
2.12.62.	RETAINF	134
2.12.63.	RETRYL	134
2.12.64.	RETRYT	135
2.12.65.	ROOM	135
2.12.66.	SYSAREA	136
2.12.67.	THRESHLD	136
2.12.68.	TITLE	136
2.12.69.	TRC	137
2.12.70.	UCS	137
2.12.71.	USERDATA	138
2.12.72.	USERLIB	138
2.12.73.	USERPATH	139
2.12.74.	WRITER	139
2.13.	PEND文	139
2.14.	PROC文	140
2.15.	SET文	142

2.16.	XMIT文	144
2.17.	空文	144
2.18.	段落見出し	144
2.19.	コメント文	145
第3章	JES2 JCL文	147
3.1.	概要	147
3.2.	JES2コマンド文	147
3.2.1.	S	149
3.2.2.	/DBR, /STA(/START)	149
3.3.	JOBPARM文	149
3.3.1.	BURST/B	150
3.3.2.	BYTES/M	151
3.3.3.	CARDS/C	151
3.3.4.	COPIES/N	151
3.3.5.	FORMS/F	151
3.3.6.	LINECT/K	151
3.3.7.	LINES/L	152
3.3.8.	PAGES/G	152
3.3.9.	PROCLIB/P	152
3.3.10.	RESTART/E	153
3.3.11.	ROOM/R	153
3.3.12.	SYSAFF/S	153
3.3.13.	TIME/T	154
3.4.	MESSAGE文	154
3.5.	NETACCT文	154
3.6.	NOTIFY文	154
3.7.	OUTPUT文	155
3.8.	PRIORITY文	155
3.9.	ROUTE文	156
3.10.	SETUP文	157
3.11.	SIGNOFF文	157
3.12.	SIGNON文	157
3.13.	XEQ文	157
3.14.	XMIT文	158
第4章	動の変数制御文	159
4.1.	概要	159
4.2.	%%制御文	159
4.2.1.	SET	161
4.2.2.	GLOBAL	163
4.2.3.	IF/ELSE/ENDIF	163
4.3.	OPC制御文	164
4.3.1.	SCAN	165

4.3.2.	SETFORM	165
4.3.3.	SETVAR	166
4.3.4.	BEGIN	167
4.3.5.	END	167
索引		169

図目次

[図 1.1]	JCL文の記述形式	1
[図 1.2]	JES2 JCL文の記述形式	4
[図 2.1]	実行プログラムの指定	74

このガイドについて

対象読者

本書は、リホスト・ソリューションのTmax OpenFrame[®](以下、OpenFrame)/Batchシステムでジョブ制御言語(JCL)を作成する開発者を対象としており、IBMメインフレームのJCLに対応するOpenFrameのMVS JCL文について説明します。

前提知識

本書を理解するためには、IBMメインフレームとJCLに関する基本知識が必要です。

制限事項

本書では、ジョブ制御言語の構文についてのみ説明しています。OpenFrame/Batchを使用するための環境設定やユーティリティについては、各ガイドを参照してください。

参考

1. 環境設定の詳細については、『OpenFrame 環境設定ガイド』を参照してください。
 2. ユーティリティの詳細については、『OpenFrame ユーティリティリファレンスガイド』を参照してください。
-

本書の構成

本書は、計4章で構成されています。

各章の主な内容は以下のとおりです。

- 第1章: JCLの紹介

OpenFrameで使えるJCLを紹介し、JCLの記述形式、オペランド、開始と終了について説明します。

- 第2章: JCL文

JCL文とオペランドについて説明します。

- 第3章: JES2 JCL文

JES2 JCL文とオペランドについて説明します。

- 第4章: 動的変数制御文

動的変数制御文とオペランドについて説明します。

表記上の規則

表記	意味
<AaBbCc123>	プログラム・ソースコードのファイル名
Ctrl+C	CtrlキーとCキーを同時に押す
[Button]	GUIのボタン、メニュー名
太字	強調
「」、『』（鍵カッコ）	関連文書、あるいはガイド内の他の章および節の表示
「入力項目」	画面UI上の入力項目
ハイパーリンク	メール・アカウント、Webサイト
>	メニューの実行順
+----	サブディレクトリ/ファイル有り
----	サブディレクトリ/ファイル無し
参考	参照/注意事項
[図 1.1]	図の名前
AaBbCc123	コマンド、コマンド実行結果の画面出力、サンプル・コード
{ }	必須パラメータ値
[]	オプション・パラメータ値
	選択パラメータ値

システム要件

	要求事項
プラットフォーム	Solaris 11 (SunOS 5.11)以上 (32ビット, 64ビット)
	Linux x86 2.6以上 (32ビット, 64ビット)
	Linux ia64 2.6以上 (32ビット, 64ビット)
ハードウェア	5GB以上のハードディスク
	8GB以上のメモリ
データベース	Tibero 6 (Fixset07)
コンパイラー	Micro Focus COBOL、NetCOBOL、OpenFrame COBOL
	OpenFrame PL/I
	OpenFrame ASM
OpenFrame製品群	OpenFrame/Base 7.1

参考

IBMまたはHP-UXプラットフォームをご使用の場合は、ティーマックスソフトの技術サポート・チームにお問い合わせください。

関連文書

ガイド	説明
OpenFrame Batchガイド	OpenFrame/Batchの紹介および機能について説明しています。
OpenFrame TJESガイド	TJESを使用してOpenFrameシステムのバッチ・ジョブを管理する方法について説明しています。
OpenFrame Batchインストールガイド	OpenFrame/Batch製品のインストール方法について説明しています。
OpenFrame ユーティリティ リファレンスガイド	OpenFrameエンジンと一緒に提供される多様なユーティリティ・プログラムについて説明しています。
OpenFrame データセットガイド	OpenFrameのデータセットを紹介し、データセットの種類やカタログ方法などについて説明しています。
OpenFrame 環境設定ガイド	OpenFrameシステムの運用に必要な環境設定について説明しています。

参考文献

製品	ガイド
メインフレーム	z/OS MVS JCL Reference

第1章 JCLの紹介

OpenFrameは、IBMメインフレームのJCLに対応するMVS JCLを提供しています。本章では、JCLの基本概念について説明します。

1.1. 記述形式

本節では、JCLの記述形式について記述します。

1.1.1. JCL文

以下は、JCL文の記述形式です。

[図 1.1] JCL文の記述形式

1	2	3	...	...	...	72	73~80
/	/	Name	△ ¹ Operation	△ ¹ Operand	△ ¹ Comment	Continued operand	Ignored space

△¹: More than 1 empty space

項目	説明
名前(Name)	JCL文の名前を指定します。他のJCL文と区別したり、システムで参照したりする際に使用します。 名前を指定する際は、必ず3桁目から記述します。名前は8文字以内の英数字にし、最初の文字は必ず英文字にします。 コメント文、段落見出し、空文には名前がありません。
オペレーション (Operation)	JCLのオペレーションを指定します。 名前を指定した場合、名前との間に1つ以上の空白を入れます。名前を省略した場合は、4桁目以降から記述します。 コメント文、段落見出し、空文にはオペレーションがありません。
オペランド(Operand)	各JCLの構文に適するオペランドを記述します。各オペランドはコンマ(,)で区切ります。 オペランドはオペレーションの後に記述し、その間には1つ以上の空白を入れます。

項目	説明
	コメント文、段落見出し、空文、PEND文にはオペランドがありません。
コメント(Comment)	オペランドの後にコメントを記述する場合、間に1つ以上の空白を入れます。 オペランドが存在する構文(JOB文、EXEC文、DD文、PROC文)でオペランドを省略した場合、コメントは記入できません。 また、空文にもコメントを記入できません。
継続行(Continued operand)	1つのJCL文を1行に記述できない場合、以下に記述する方法を用い、複数行に亘って記述できます。 コメント文、段落見出し、空文には継続行を指定できません。継続行の詳細については、「継続的表現」で説明します。
任意指定領域(Ignored space)	73桁目から80桁目までは任意の文字を入力できます。 81桁目以降のスペースはシステムで無視します。

継続的表現

以下は、オペランド、コメント、引用符付き文字列を継続して表現するための方法です。

● オペランドの継続1

- 71桁目の前のオペランドがコンマ(,)で終わった場合、オペランドの継続が指定されたと判断します。
- 継続行の1桁目と2桁目には「//」を記述します。
- 継続行では名前を記述してはなりません。
- 継続しているオペランドは4桁目から16桁目までの間に記述を始めます。

● オペランドの継続2

- オペランドがコンマ(,)で終わらず、コメントを記述していない状態で72桁目に空白以外の文字を記述した場合、オペランドが継続するものと判断します。
- 継続行の1桁目と2桁目には「//」を記述します。
- 継続行では名前を記述してはなりません。
- 継続しているオペランドは、4桁目から16桁目までの間に記述を始めます。まずオペランドを区切るためのコンマ(,)を記述します。

● コメントの継続

コメントを記述できる制御文において、オペランドの記述が終わってコメントを記述する際、72桁目に空白以外の文字を記述すると、次の行までコメントが記述されるものと判断します。

- 引用符付き文字列の継続

オペランドの値が単一引用符(')で囲まれており、その値の内容が長くて複数行に亘って記述された場合は継続するものと判断します。以降、継続する行の1桁目と2桁目には「//」を記述し、内容は4桁目から16桁目の間に記述を始めます。

以下は、継続的表現の例です。

- 例1

「オペランドの継続1」のルールに基づいて、継続が指定されたものと判断します。

```
123-----72
//JOB1 JOB CLASS=A,MSGCLASS=A,COND=(0,EQ),
//      MSGLEVEL=(1,1)
```

- 例2

コンマ(,)以降にコメントを記述しても、それとは関係なく「オペランドの継続1」のルールに従います。

```
123-----72
//JOB1 JOB CLASS=A,MSGCLASS=A,COND=(0,EQ), this is comment
//      MSGLEVEL=(1,1)
```

- 例3

オペランドがコンマ(,)で終わってコメントを記述した状態で、72桁目に継続文字が記述されているため、「オペランドの継続1」と「コメントの継続」のルールが満たされています。この場合、「オペランドの継続1」のルールとして判断するため、次の行がコメント処理されず、MSGLEVEL オペランドを認識することになります。

```
123-----72
//JOB1 JOB CLASS=A,MSGCLASS=A,COND=(0,EQ), this is comment      A
//      MSGLEVEL=(1,1)
```

- 例4

オペランドがコンマ(,)で終わっていませんが、コメントがない状態で継続文字が72桁目に記述されているため、「オペランドの継続2」のルールに従います。

```
123-----72
//JOB1 JOB CLASS=A,MSGCLASS=A,COND=(0,EQ)                        B
//      ,MSGLEVEL=(1,1)
```

- 例5

オペランドがコンマ(,)で終わっておらずコメントを記述した状態で、継続文字が72桁目に記述されているため、「コメントの継続」ルールに従います。したがって、次の行はコメントと見なされ、MSGLEVEL=(1,1)はコメントとして処理されます。

```

123-----72
//JOB1 JOB CLASS=A,MSGCLASS=A,COND=(0,EQ) this is comment      C
//          ,MSGLEVEL=(1,1)

```

● 例6

EXEC文のPARMオペランドの値が「引用符付き文字列の継続」ルールに基づいて継続として処理されました。PARMの値は「this is tmaxsoft program argument」です。

```

123-----72
//JOB1 JOB CLASS=A,MSGCLASS=A,COND=(0,EQ),MSGLEVEL=(1,1)
//STEP1 EXEC PGM=TMAXSOFT,COND=(0,NE),PARM='this is tmaxsoft program a
//          rgument'

```

● 例7

「tmaxsoft_」はtmaxsoftの後に空白があることを意味します。このように設定した場合、空白までを値として判断します。したがって、PARMの値は「this is tmaxsoft program argument」となります。

```

123-----72
//JOB1 JOB CLASS=A,MSGCLASS=A,COND=(0,EQ),MSGLEVEL=(1,1)
//STEP1 EXEC PGM=TMAXSOFT,COND=(0,NE),PARM='this is tmaxsoft_
//          program argument'

```

1.1.2. JES2 JCL文

以下は、JES2 JCL文の記述形式です。

[図 1.2] JES2 JCL文の記述形式

1	2	...	...	73~80
/	*	Operation	△ ¹ Operand	Ignored space

△¹: More than 1 empty space

項目	説明
オペレーション (Operation)	JES2 JCLのオペレーションを指定します。「/」に続く3桁目から記述します。
オペランド(Operand)	JES2 JCLの各構文に適するオペランドを記述します。各オペランドはコンマ(,)で区切ります。
任意指定領域(Ignored space)	73桁目から80桁目までは任意の文字を記述することができます。81桁目以降の空白はシステムで無視します。

1.2. オペランド

JCLのオペランドを記述する方法には、位置オペランド方式とキーワード・オペランド方式があります。

1.2.1. 位置オペランド

位置オペランドは、オペランドの位置が決定されており、最初から何番目に位置するかによってオペランドの値が指定される方式です。

以下は、位置オペランドの設定についてのルールです。

- 各位置オペランドはコンマ(,)で区切ります。

```
値1, 値2, 値3
```

- 位置オペランドを省略する場合も、その位置を示すためにコンマ(,)を省略してはなりません。

```
値1,, 値3
```

- 省略する位置オペランドの以降に位置オペランドが存在しない場合は、以降の位置オペランドを区切る必要がないため、コンマ(,)を省略できます。

```
値1, 値2,, キーワード = 値4
```

```
値1, 値2, キーワード = 値4
```

- 位置オペランドの以降にキーワード・オペランドが存在しない場合、最後のコンマ(,)は省略します。

```
値1, 値2
```

1.2.2. キーワード・オペランド

キーワード・オペランドは、キーワードと値を等号(=)で指定する方式です。

```
キーワード = 値
```

以下は、キーワード・オペランドの設定についてのルールです。

- 各キーワード・オペランドはコンマ(,)で区切ります。キーワードオペランドは位置に関係ないため、キーワード・オペランドを省略する場合はコンマ(,)も一緒に省略します。
- キーワードオペランドは、EXEC文のPROCオペランドとPGMオペランドを除いては、順序に意味がありません。
- 位置オペランドとキーワード・オペランドが両方ある場合、すべての位置オペランドを先に記述してから、キーワード・オペランドを記述します。

- 位置オペランドをすべて省略した場合、キーワード・オペランドの前にコンマ(,)を指定する必要はありません。

1.2.3. 記述規約

以下は、本書のオペランドの記述規約です。

- `[]` (brackets)

この記号が使用されたオペランドは省略できます。

– 例

以下の演算式からは、次の4つのケースが可能です。

```
A[,B][,C]
```

- A
- A,B
- A,C
- A,B,C

- `{}` (braces)

この記号が使用された場合、いずれかのオペランドを選択します。

– 例

以下の演算式からは、次の2つのケースが可能です。

```
{B}  
{C}
```

- B
- C

- `|` (vertical bar)

この記号が使用された場合、いずれかのオペランドを選択します。

– 例

以下の演算式からは、次の2つのケースが可能です。

```
{ PS | PSU }
```

- PS
- PSU

- **Bold**

`{}`内に何も記述されていない場合、デフォルト値として使用される値です。

– 例

以下の演算式でFREEパラメータが省略されると、FREE=ENDが指定されます。

```
FREE = {END | CLOSE}
```

- ... (ellipsis)

この記号の直前に存在する項目を繰り返して指定できることを意味します。

- 指定値のタイプ

以下は、オペランドに指定できる指定値のタイプについての説明です。

– 数字

意味	0 1 2 3 4 5 6 7 8 9
例	0, 1, 9

– 英文字と記号

意味	A B C D E F G H I J K L M N O P Q R S T U V W X Y Z \$ # @
例	C,J,@,\$

– 記号と特殊文字

意味	英数字を除いた任意の文字として、コンマ(,), 単一引用符('), 等号(=), 括弧、空白などを単一引用符(')で囲んで記述します。
例	, , , , / , ' , (,) , * , & , + , - , = , 空白

– 英数字

意味	英文字、数字、または英文字と数字の組み合わせです。
例	A, 7, A56, 8X, ABC, 997

– 符号なし整数

意味	符号なし数字の組み合わせです。
例	01, 375

– 整数

意味	符号付き数字と数字の組み合わせです。
例	99, +76, -143

– 特殊文字列

意味	英文字、数字、記号、特殊文字、または英文字と数字と記号と特殊文字の組み合わせです。
例	32 A5, *315'?&A

– 引用符付き文字列

意味	<英数字> '<特殊文字列>'のように特殊文字列が単一引用符(')で囲まれている場合、システムで1つの単一引用符(')として認識するためには、2つを連続して指定する必要があります。 アンパサンド(&)の場合も、2つが連続で指定されていると、1つのアンパサンド(&)と見なします。ただし、アンパサンド(&)は1つのみ指定されていても、1文字のアンパサンド(&)と見なします。引用符付き文字列が単一引用符(')で囲まれている場合、文字数に引用符(')は含まれません。
例	'5&23' '8=/ 'A"B', 8X3B

– 記号名

意味	英文字で始まる8文字以内の英数字と特殊文字の組み合わせです。
例	Z7@, LMN, A123BC

1.2.4. 例

以下は、JOB文に課金識別情報とプログラマー名を位置オペランドで指定し、CLASSとMSGCLASSをキーワード・オペランドで指定する例です。

```
//JOB1 JOB A001, TMAX.PROGRAM, CLASS=A, MSGLEVEL=(1,1)
```

課金識別情報とプログラマー名を指定する際、課金識別情報(A001)を省略する場合は最初のコンマ(,)を省略できません。

```
//JOB1 JOB , TMAX.PROGRAM, CLASS=A, MSGLEVEL=(1,1)
```

プログラマー名(TMAX.PROGRAM)を省略する場合は、位置オペランドがそれ以上存在しないため、省略のためのコンマ(,)は指定しなくても構いません。

```
//JOB1 JOB A001, CLASS=A, MSGLEVEL=(1,1)
```

課金識別情報とプログラマー名の両方を省略する場合、以下のように指定できます。

```
//JOB1 JOB CLASS=A, MSGLEVEL=(1,1)
```

パラメータの指定

オペランドの値がパラメータのリストとして記述されている場合にも、各パラメータは、「[1.2.1. 位置オペランド](#)」と「[1.2.2. キーワード・オペランド](#)」の記述方法に従います。

以下は、パラメータを指定する例です。

```
LABEL=(3,SL,PASSWORD,OUT,RETPD=350)
```

以下は、PASSWORD位置パラメータを省略した場合の例です。

```
LABEL=(3,SL,,OUT,RETPD=350)
```

以下は、SLパラメータとRETPDパラメータのみ指定する場合の例です。

```
LABEL=(,SL,RETPD=350)
```

1.3. JCL

本節では、ジョブ制御言語(JCL)で使用されるジョブの開始と終了および記述手順について説明します。

1.3.1. ジョブの開始と終了

以下は、ジョブの開始と終了についての説明です。

- ジョブの開始

JOB文が出現したら、ジョブの開始です。

- ジョブの終了

空文の次のジョブの開始を示すJOB文がジョブの終了です。入力ストリームの最後のジョブの場合、入力ストリームの最後がジョブの終了です。

ジョブには1つ以上のステップが必要です。つまり、JOB文と次のJOB文の間には1つ以上のEXEC文が存在しなければなりません。以下は、ジョブ・ステップの開始と終了についての説明です。

- ジョブ・ステップの開始

EXEC文の開始がジョブ・ステップの開始です。

- ジョブ・ステップの終了

次のジョブ・ステップの開始を示すEXEC文がジョブ・ステップの終了です。ジョブの最後のステップの場合、ジョブの終了とジョブ・ステップの終了が同じです。

1.3.2. 記述手順

ジョブ・ステップは、各ジョブの範囲内に記述します。したがって、EXEC文はJOB文の後に記述します。

JCL文の記述手順

以下は、JCL文の記述手順です。

1. JOB文

ジョブの開始を意味します。ただし、PRIORITY文がある場合はPRIORITY文の後に記述します。

2. EXEC文

EXEC文は以下の3つに分けられます。

– プロシージャ内に記述されていない場合

JOB文、JOBLIB DD文、JOB CAT DD文の次に、各ジョブ・ステップの先頭に記述します。

– 入力ストリーム・プロシージャ内に記述されている場合

PROC文の次に、各プロシージャ・ステップの先頭に記述します。

– カタログ・プロシージャ内に記述されている場合

各プロシージャ・ステップの先頭にPROC文がある場合は、PROC文の次に記述します。

3. DD文

DD文は以下の2つに分けられます。

– JOBLIB DD文、JOB CAT DD文

JOB文の直後に、最初のジョブ・ステップを定義するEXEC文の前に記述します。ただし、JOB文とDD文の間にJES2 JCL文を記述することは可能です。

– その他のDD文

EXEC文の次に記述します。

4. 段落見出し

SYSINデータセットの最後に記述します。

5. 空文

ジョブの最後に記述します。入力ストリーム・プロシージャまたはカタログ・プロシージャには記述できません。

6. コメント文

ジョブの範囲内であればどこでも構いません。ただし、インストリーム・データセット内には指定できません。

7. PROC文

PROC文は以下の2つに分けられます。

- 入カストリーム・プロシージャ

各入カストリーム・プロシージャの先頭に記述します。JOB文の次に、入カストリーム・プロシージャを呼び出すジョブ・ステップの前に記述します。

- カタログ・プロシージャ

カタログ・プロシージャの先頭に記述します。

8. PEND文

各入カストリーム・プロシージャの最後に記述します。

JES2 JCL文の記述手順

以下は、JES2 JCL文の記述手順です。

1. PRIORITY文

JOB文の直前に記述します。

2. PRIORITY文を除いたその他の構文

どこに位置しても構いません。

第2章 JCL文

本章では、JCL文と各オペランドについて説明します。

2.1. 概要

以下は、JCL文の一覧です。

JCL文	説明
CNTL文	プログラムの制御文の開始を示します。
DD文	ジョブまたはジョブ・ステップで使用されるデータセットとその属性を定義します。
スペシャルDD文	特別な名前のDD名を使用するDD文です。
ENDCNTL文	CNTL文に続くプログラム制御文の終わりを示します。
EXEC文	ステップの開始を示し、ステップの属性を定義します。
IF/THEN/ELSE/ENDIF文	JCL文を分岐して実行できるように条件を指定し、条件に応じて実行する文を記述します。
INCLUDE文	指定したメンバーをJCL文に含めることを示します。
JCLコマンド文	JCLコマンドを記述します。
JCLLIB文	JCLで使用するプロシージャまたはINCLUDE文で指定したJCL文を取得するためのライブラリを指定します。
JOB文	ジョブの開始を示し、ジョブの属性を定義します。
OUTPUT文	出力処理のためのSYSOUTデータセットの属性を定義します。
PEND文	プロシージャの終わりを示します。
PROC文	プロシージャの開始を示します。
SET文	JCLで使用する記号パラメータの値を指定します。
XMIT文	別のノードにデータを送信します。
空文	ジョブの終わりを示します。
段落見出し	インストリーム・データの最後の行の直後に位置し、インストリーム・データの終わりを示します。
コメント文	コメントを記述します。

2.2. CNTL文

プログラム制御文の開始を示します。CNTL文が記述されると、ENDCNTL文が現れるまで、構文はプログラム制御文として処理されます。OpenFrameでは、CNTL文とプログラム制御文、そしてENDCNTL文に対して構文エラーのみをチェックし、使用されません。

- 構文

```
//[ラベル]△1CNTL△[* コメント]
```

- 例

以下は、CNTL文とENDCNTL文を使用した例です。

```
//STEP1      EXEC      PGM=PRINT
//ALPHA      CNTL      * PROGRAM CONTROL STATEMENT FOLLOWS
//PRGCNTL    PRINTDEV  BUFNO=20,PIMSG=YES,DATAACK=BLOCK
//OMEGA      ENDCNTL
//AGAR       DD        UNIT=AFP1
```

2.3. DD文

ジョブまたはジョブ・ステップで 사용되는データセットとその属性を定義します。

- 構文

```
//[DD名]△1DD△1[位置オペランド][,キーワードオペランド]...△1[コメント]
//[プロシージャステップ名.DD名]△1DD△1[位置オペランド][,キーワードオペランド]...△1[コメント]
```

項目	説明
DD名	DD名を指定します。「//」に続いて3桁目から始める必要があります。
プロシージャステップ名.DD名	DD名を省略したDDは、CONCATED DDと判断されます。 DD名は8文字以内の英数字で、最初の文字は必ず英文字にする必要があります。 同じステップ内で同じ名前のDD名が重複している場合、システムはDDの割り当て時に重複した最初のDD以降の重複DD名を任意のDD名に設定します。任意のDD名は\$DD0000\$から1ずつ増やしながら生成されます。 プロシージャ・ステップ名と一緒にDD名を記述した場合、そのDDはプロシージャ・ステップの同じ名前のDD属性をオーバーライドします。同じ名前のDDがなければ、その名前でDDが追加されます。プロシージャ・ステップ名は8文字以内の英数字で、最初の文字は英文字にします。

項目	説明
	CONCATENATED DDおよびプロシージャ・ステップへの追加とオーバーライドについては、以下で詳しく説明します。
DD	オペレーションを記述する位置であり、DD名の後に1つ以上の空白を入れて「DD」と記述します。
[位置オペランド],[キーワード オペランド]	「DD」の後に1つ以上の空白を入れてオペランドを記述します。先にすべての位置オペランドを記述してから、その後にキーワード・オペランドを記述します。詳細については、 オペランド と各オペランドの節を参照してください。
[コメント]	オペランドの次に1つ以上の空白を入れて記述します。コメントは71桁目まで記述できます。すべてのオペランドを省略した場合は、コメントを記述することができません。

CONCATENATED DDおよびプロシージャ・ステップ

以下は、CONCATENATED DD、プロシージャ・ステップへの追加またはオーバーライドについての説明です。

- CONCATENATED DD

複数の入力用データセットを論理的に1つのデータセットであるかのように処理して使用することをデータセットの連結といいます。データセットを連結するためにCONCATENATED DDを使用します。

CONCATENATED DDはDD名を省略する方式で使用しますが、このDDは先に宣言されたDDと論理的に連結されます。JOBCATまたはSTPECATのようなスペシャルDDも連結して使用できます。

以下は、3つのデータセットをCONCATENATED DDを使用して論理的に連結した例です。

```
//SYSIN DD DSN=TMAX.SYSIN1,DISP=SHR
//      DD DSN=TMAX.SYSIN2,DISP=SHR
//      DD DSN=TMAX.SYSIN3,DISP=SHR
```

- プロシージャ・ステップへの追加またはオーバーライド

プロシージャを呼び出すステップで指定したDDはそのステップでは使用されず、呼び出されたプロシージャ内のステップで使用されます。プロシージャを呼び出すステップでDDを指定するときは、そのDDがプロシージャ内のどのステップで使用されるかを明確に指定する必要があります。プロシージャ内のステップは、「//」の後にプロシージャ・ステップ名.DD名と指定します。

プロシージャ・ステップ名を指定しなかった場合は、直前のDDで指定したプロシージャ・ステップ名が使われます。プロシージャ・ステップ名が省略されたDDがステップで最初のDDである場合、そのDDはプロシージャ内の最初のステップで使用されるものと判断します。指定したプロシージャ・ステップ名がプロシージャ内に存在しない場合は、そのDDは使用されません。

以下は、PSTEP1でDD1が、PSTEP2でDD2が使用される例です。

```
//JOB1      JOB
//TMAXPROC   PROC
//PSTEP1     EXEC  PGM=TMAXSOFT
//PSTEP2     EXEC  PGM=TMAXSOFT
//          PEND
//STEP1      EXEC  TMAXPROC
//PSTEP1.DD1 DD  DSN=TMAX.DSN,DISP=SHR
//PSTEP2.DD2 DD  DSN=TMAX.DSN2,DISP=SHR
//
```

以下は、DD2のプロシージャ・ステップ名が省略されたが、DD1のプロシージャ・ステップ名がPSTEP2であるため、PSTEP2でDD2が使用される例です。

```
//JOB1      JOB
//TMAXPROC   PROC
//PSTEP1     EXEC  PGM=TMAXSOFT
//PSTEP2     EXEC  PGM=TMAXSOFT
//          PEND
//STEP1      EXEC  TMAXPROC
//PSTEP2.DD1 DD  DSN=TMAX.DSN,DISP=SHR
//DD2        DD  DSN=TMAX.DSN2,DISP=SHR
//
```

以下は、DD1とDD2のプロシージャ・ステップ名が省略され、TMAXPROCの最初のステップであるPSTEP1で使用された例です。

```
//JOB1      JOB
//TMAXPROC   PROC
//PSTEP1     EXEC  PGM=TMAXSOFT
//PSTEP2     EXEC  PGM=TMAXSOFT
//          PEND
//STEP1      EXEC  TMAXPROC
//DD1        DD  DSN=TMAX.DSN,DISP=SHR
//DD2        DD  DSN=TMAX.DSN2,DISP=SHR
//
```

オペランド

以下は、オペランドについての説明です。各オペランドの詳細については、各節を参照してください。

- 位置オペランド

DD文の位置オペランドは、アスタリスク(*)、DATA、DUMMY、DYNAMのいずれかを記述します。

項目	説明
アスタリスク(*)と DATA	インストリーム・データの入力を指定します。DLMオペランドを使用すると、JCL文もインストリーム・データで取得することができます。
DUMMY	ダミー・データセットを指定します。
DYNAM	OpenFrameでは構文解析のみサポートします。

● キーワード・オペランド

項目	説明
ACCODE	OpenFrameでは構文解析のみサポートします。
AMP	OpenFrameでは構文解析のみサポートします。
AVGREC	レコードの保存スペースの初期値と追加値の単位を指定します。
BLKSIZE	ブロックの最大長を指定します。
BLKSZLIM	OpenFrameでは構文解析のみサポートします。
BURST	OpenFrameでは構文解析のみサポートします。
CCSID	OpenFrameでは構文解析のみサポートします。
CHARS	データセットを出力する際の出力文字とサイズに関する設定テーブルを指定します。
CHKPT	OpenFrameでは構文解析のみサポートします。
CNTL	OpenFrameでは構文解析のみサポートします。
COPIES	SYSOUTデータセットの全体およびページ単位のコピー回数を指定します。
DATACLAS	SMS データセット・クラスを指定します。
DCB	非VSAMデータセットのDCB(Data Control Block)情報を指定します。
DDNAME	DD名を指定し、指定されたDDが出てくるまでデータセットの定義を先送りします。
DEST	SYSOUTデータセットの出力先を指定します。
DISP	データセットの状態および後処理について設定します。
DLM	アスタリスク(*)またはDATAオペランドと一緒に使用され、インストリーム・データの終わりを示す区切り文字を指定します。
DSID	OpenFrameでは構文解析のみサポートします。
DSNAME/DSN	データセットの名前を指定します。
DSNTYPE	データセットの形式を指定します。

項目	説明
EXPDT	データセットの満了日を指定します。
FCB	SYSOUTデータセットを出力するプリンターを指定します。出力時にフォーマットが可能です。
FILEDATA	OpenFrameでは構文解析のみサポートします。
FLASH	用紙に一定の書式、範囲をあらかじめ印刷する場合に使用するフィルム・オーバーレイの識別名と適用枚数を指定します。
FREE	OpenFrameでは構文解析のみサポートします。
HOLD	SYSOUTデータセットの出力を保留するかどうかを指定します。
KEYLEN	新規データセットのキー長を指定します。
KEYOFF	新規データセットのキーが始まるオフセットを指定します。
LABEL	データセットのラベルを指定します。
LGSTREAM	OpenFrameでは構文解析のみサポートします。
LIKE	既存のデータセットの属性を新しく作成したデータセットの属性にコピーします。既存のデータセットはカタログ化されている必要があります。
LRECL	新規作成されるデータセットのレコード長を指定します。
MGMTCLAS	SMSデータクラスを指定します。
MODIFY	SYSOUTデータセットのコピー修飾モジュール名とCHARSオペランドで指定するテーブルのシーケンス番号を指定します。
OUTLIM	SYSOUTデータセットに出力できる最大レコード数を指定します。
OUTPUT	参照するためのOUTPUT文を指定します。
PATH	UNIXファイルのパスを指定します。
PATHDISP	PATHパラメータで指定したUNIXファイルの状態および後処理の設定を行います。
PATHMODE	PATHパラメータで指定したUNIXファイルを作成する際に、ユーザーごとにファイルへのアクセス・モードを指定します。ファイルの作成は、「PATHOPTS=OCREAT」の設定によって実行されます。
PATHOPTS	PATHパラメータで指定したUNIXファイルのアクセス・オプションと状態オプションを指定します。
PROTECT	OpenFrameでは構文解析のみサポートします。
RECFM	データセットのレコード形式と特性を指定します。
RECORDG	OpenFrameでは構文解析のみサポートします。
REFDD	前に設定したDDの属性を参照します。

項目	説明
RETPD	データセットの保存期間を日数で指定します。
RLS	OpenFrameでは構文解析のみサポートします。
SECMODEL	OpenFrameでは構文解析のみサポートします。
SEGMENT	OpenFrameでは構文解析のみサポートします。
SPACE	直接アクセス・ボリュームに新規データセットのスペース割り振り量を指定します。
SPIN	OpenFrameでは構文解析のみサポートします。
STORCLAS	SMSストレージ・クラスを指定します。
SUBSYS	OpenFrameでは構文解析のみサポートします。
SYMBOLS	インストリーム・データで記号パラメータを使用します。
SYSOUT	SYSOUTデータセットの出力プロパティを指定します。
TERM	OpenFrameでは構文解析のみサポートします。
UCS	OpenFrameでは構文解析のみサポートします。
UNIT	データセットを割り当てるI/O デバイスを指定します。
VOLUME/VOL	データセットのボリュームを指定します。

2.3.1. アスタリスク(*)とDATA

インストリーム・データの入力を指定します。DLMオペランドを使用すると、JCL文もインストリーム・データで取得することができます。該当するDD文以降にインストリーム・データが現れることを意味します。また、そのDDはインストリーム・データの内容でジョブ・スプールに一時的に作成され、一般的なデータセットと同じように使用されます。

インストリーム・データは、以下のような場合に終わりとして判断します。

- 段落見出しが現れた場合
- 入カストリームの終わり、UNIXファイルでEOFが現れた場合
- 段落見出し以外の制御文が現れた場合
- DLMオペランドが指定され、区切り文字が現れた場合

以下は、アスタリスク(*)とDATAの構文です。

- 構文

* DATA

- 注意事項

アスタリスク(*)の場合は、DLMオペランドを使用できません。

- 例

以下は、アスタリスク(*)を使用して、インストリーム・データをSYSINとして使用する例です。

```
//SYSIN DD *  
this is sysin data  
/*
```

以下は、制御文が現れてインストリーム・データが終了したことを意味する例です。

```
//SYSIN DD *  
this is sysin data  
//SYSOUT DD SYSOUT=*
```

2.3.2. ACCODE

OpenFrameでは構文解析のみサポートします。

- 構文

```
ACCODE = 値
```

項目	説明
値	8文字以内の記号名です。

2.3.3. AMP

OpenFrameでは構文解析のみサポートします。

- 構文

```
AMP = 値
```

2.3.4. AVGREC

レコード保存スペースの初期値と追加値の単位を指定します。

- 構文

```
AVGREC = {U|K|M}
```

項目	説明
U	値の単位が1バイトであることを示します。
K	値の単位が1024バイトであることを示します。
M	値の単位が1048576バイトであることを示します。

SPACEオペランドの最初のパラメータが「レコード長」と指定されている場合に適用され、SPACEオペランドの初期値と追加値の単位として使用されます。

AVGRECオペランドが使用される場合、スペースの割り当て量は以下のように計算されます。

初期スペースの割り当て量 = 初期値 * レコード長 * AVGRECで指定した単位
追加スペースの割り当て量 = 追加値 * レコード長 * AVGRECで指定した単位

- 注意事項

SPACEオペランドの最初のパラメータがTRKあるいはCYLの場合、このオペランドは無視されます。

- 例

AVGRECをMと指定します。この場合、初期スペースの割り当て量は $5 * 100 * 1048576$ となり、追加スペースの割り当て量は $2 * 100 * 1048576$ となります。

```
//EX1 DD DSN=NEW.DATASET,DISP=(NEW,CATLG),SPACE=(100,(5,2)),AVGREC=M
```

2.3.5. BLKSIZE

ブロックの最大長を指定します。BLKSIZEオペランドで指定できる最大値はレコード形式に関係なく2097152バイトです。

- 構文

BLKSIZE = {値 | 値K | 値M}

項目	説明
値	0~999999999の符号なし整数を指定します。
値K	0~98303の符号なし整数を指定します。 値に1024を掛けた値をブロックの最大長とします。
値M	0~95の符号なし整数を指定します。 値に1024*1024を掛けた値をブロックの最大長とします。

- 注意事項

DCBオペランドのパラメータにBLKSIZEが指定されている場合、このオペランドはオーバーライドされて無視されます。

- 例

BLKSIZEを4000と指定します。

```
//EX1 DD DSN=NEW.DATASET, DISP=(NEW,CATLG),
//      RECFM=FB, LRECL=80, BLKSIZE=4000
```

2.3.6. BLKSZLIM

OpenFrameでは構文解析のみサポートします。

- 構文

```
BLKSZLIM = {値 | 値K | 値M | 値G}
```

項目	説明
値	0~2147483648の符号なし整数を指定します。
値K	0~2097151の符号なし整数を指定します。
値M	0~2047の符号なし整数を指定します。
値G	0~1の符号なし整数を指定します。

2.3.7. BURST

OpenFrameでは構文解析のみサポートします。

- 構文

```
BURST = {YES | Y | NO | N}
```

2.3.8. CCSID

OpenFrameでは構文解析のみサポートします。

- 構文

```
CCSID = 値
```

項目	説明
値	1~65535の符号なし整数を指定します。

2.3.9. CHARS

データセットを出力する際の出力文字とサイズの設定テーブルを指定します。

- 構文

```
CHARS = {テーブル名}
        {(テーブル名[, テーブル名])}
        {DUMP}
        {(DUMP[, テーブル名])}
```

項目	説明
テーブル名	データセットを出力する際の出力文字とサイズの設定テーブルを1~4文字の英数字で指定します。テーブル名は、最大4つまで指定できます。
DUMP	OpenFrameではサポートされません。

- 注意事項

- SYSOUTデータセットの出力でなければ、CHARSオペランドは無視されます。
- OpenFrameでは、印刷のためのCHAR情報を外部プリンター・モジュールに渡すために、ジョブの実行後も保存します。この情報を使用するかどうかは、外部プリンター・モジュールの必要によって異なります。

- 例

以下は、外部プリンター・モジュールにCHARS=(TBJ1,TBJ2)の情報を渡す例です。

```
//EX1 DD SYSOUT=A,CHARS=(TBJ1,TBJ2)
```

2.3.10. CHKPT

OpenFrameでは構文解析のみサポートします。

- 構文

```
CHKPT = EOV
```

2.3.11. CNTL

OpenFrameでは構文解析のみサポートします。

- 構文

```
CNTL = {*.label }
        {*.stepname.label }
        {*.stepname.procstepname.label}
```

項目	説明
label	1～8文字の英数字で指定します。

2.3.12. COPIES

SYSOUTデータセットの全体およびページ単位のコピー回数を指定します。

- 構文

```
COPIES = (コピー回数[, (グループコピー回数[, グループコピー回数]...)])
```

項目	説明
コピー回数	SYSOUTデータセットのコピー回数を0~255の符号なし整数を指定します。 (デフォルト値: 1)
グループコピー回数	SYSOUTデータセットのページ単位のコピー回数を1~255の符号なし整数を指定します。(デフォルト値: 1)

- 注意事項

- SYSOUTデータセットの出力でなければ、COPIESオペランドは無視されます。
- OpenFrameでは、印刷出力のためのCOPIES情報を外部プリンター・モジュールに渡すために、ジョブの実行後も保存します。この情報を使用するかどうかは、外部プリンター・モジュールの必要によって異なります。

- 例

以下は、外部プリンター・モジュールにCOPIES=(2,(2,1))の情報を渡す例です。

```
//OUTPUT DD SYSOUT=A, COPIES=(2, (2, 1))
```

2.3.13. DATACLAS

SMSデータクラスを指定します。データクラスを指定すると、Openframe環境設定のsmsサブジェクトに指定されているデータ・クラスからそのクラスを検索し、記述されているデータセットの設定を使用します。

参考

smsサブジェクトの詳細については、『OpenFrame 環境設定ガイド』を参照してください。

- 構文

```
DATACLAS = クラス名
```

項目	説明
クラス	データクラス名を1～8文字の記号名で指定します。指定しない場合は、デフォルト値が使用されます。

- 注意事項

OpenFrame環境設定の**sms**サブジェクトに該当するデータクラスが設定されていない場合は、エラーが発生します。

- 例

以下は、データクラスをDATA01に指定する例です。

```
//DD1 DD DSN=NEW.DATASET,DISP=(NEW,CATLG),DATACLAS=DATA01
```

2.3.14. DCB

非VSAMデータセットのDCB(Data Control Block)情報を指定します。

- 構文

```
DCB = (dcbパラメータ[,dcbパラメータ]...)
DCB = ({データセット名}                                [,dcbパラメータ]...)
      ({*.DD名}                                           )
      ({*.ステップ名.DD名}                               )
      ({*.ステップ名.プロシージャステップ名.DD名}      )
```

– dcbパラメータ

dcbパラメータ	用途
BFALN={F D}	OpenFrameでは構文解析のみサポートします。
BFTEK={A S R D}	OpenFrameでは構文解析のみサポートします。
BLKSIZE={値 値K 値M}	DDオペランド BLKSIZE の説明と同じです。
BUFIN=値	OpenFrameでは構文解析のみサポートします。
BUFL=値	0～32760の符号なし整数を指定します。OpenFrameでは構文解析のみサポートします。
BUFMAX=値	OpenFrameでは構文解析のみサポートします。
BUFNO=値	OpenFrameでは構文解析のみサポートします。
BUFOFF={nn L}	0～99の符号なし整数またはLを指定します。OpenFrameでは構文解析のみサポートします。
BUFOUT=値	OpenFrameでは構文解析のみサポートします。

dcbパラメータ	用途
BUFSIZE=値	31~65535の符号なし整数を指定します。OpenFrameでは構文解析のみサポートします。
CPRI={R E S}	OpenFrameでは構文解析のみサポートします。
CYLOFL=値	OpenFrameでは構文解析のみサポートします。
DEN=値	1~4の符号なし整数を指定します。OpenFrameでは構文解析のみサポートします。
DIAGNS=TRACE	OpenFrameでは構文解析のみサポートします。
DSORG={PS PSU PO POU DA DAU IS ISU}	データセットの構造を指定します。 – PS: 順次データセットです。 – PSU: 移動できない順次データセットです。 – PO: 区分データセットです。 – POU: 移動できない区分データセットです。 – DA: 直接データセットです。 – DAU: 移動できない直接データセットです。 – IS: 索引データセットです。 – ISU: 移動できない索引データセットです。
EROPT={ABE ACC SKP}	OpenFrameでは構文解析のみサポートします。
FUNC={ R P W RP PW RW}	OpenFrameでは構文解析のみサポートします。
GNCP=値	OpenFrameでは構文解析のみサポートします。
INTVL=値	OpenFrameでは構文解析のみサポートします。
IPLTXID=値	OpenFrameでは構文解析のみサポートします。
KEYLEN=値	DDオペランド KEYLEN の説明と同じです。
LIMCT=値	OpenFrameでは構文解析のみサポートします。
LRECL=レコード長	DDオペランド LRECL の説明と同じです。
MODE={C E CO ER}	OpenFrameでは構文解析のみサポートします。
NCP=値	1~99の符号なし整数を指定します。OpenFrameでは構文解析のみサポートします。
NTM=値	OpenFrameでは構文解析のみサポートします。
OPTCD=値	OpenFrameでは構文解析のみサポートします。

dcbパラメータ	用途
PAGESIZE=値	OpenFrame固有の仕様として、READを使用してVSAMデータセットを読み取る際、一度に取得する行数を指定します。
PCI=値	最初にパラメータにN、R、A、Xで始まる文字列を指定します。OpenFrameでは構文解析のみサポートします。
PRTSP=値	0~3の符号なし整数を指定します。OpenFrameでは構文解析のみサポートします。
RECFM=F[B][A M] V[B S BS][A M] L[A M] U[A M]	DDオペランドRECFMの説明と同じです。
RESERVE=値	OpenFrameでは構文解析のみサポートします。
RKP=値	DDオペランドのKEYOFFの説明と同じです。KEYOFFと同時に指定された場合は、RKPに指定された値が優先されます。
STACK=値	1~2の符号なし整数を指定します。OpenFrameでは構文解析のみサポートします。
THRESH=値	OpenFrameでは構文解析のみサポートします。
TRTCH=値	OpenFrameでは構文解析のみサポートします。

– その他の項目

項目	説明
データセット名	指定したデータセットのDCB属性を使用することを指定します。 データセット名は、世代別データ・グループ名(GDG)と異なる必要があり、GDGの相対番号のメンバー名は使用できません。 指定されたデータセットはカタログされている必要があります。OpenFrameでは、カタログされずにパスされたDD文のデータセット名を使用して参照することはサポートしていません。
*.DD名 *.ステップ名.DD名 *.ステップ名.プロシージャステップ名.DD名	「*[.ステップ名[.プロシージャステップ名].DD名」の部分で逆方向参照といいます。同ジョブの先行DD文で指定されたDCB属性を使用することを意味します。 先行DD文の位置が先行ジョブ・ステップなのか先行プロシージャ・ステップなのかに応じて、各ステップ名またはステッ

項目	説明
	ブ名とプロシージャ・ステップ名を指定します。逆方向参照が指定するDDが存在しない場合、ジョブはフラッシュとして処理されます。

2.3.15. DDNAME

DD名を指定し、指定されたDDが出てくるまでデータセットの定義を先送りします。以降、指定されたDDのデータセット設定でデータセットを定義します。

DDNAMEにDD名が指定された場合、そのDDは指定されたDDが出てくるまでデータセットの定義を先送りします。指定されたDDが出てきたら、その時点でDDNAMEに指定したDDのデータセットを設定します。指定されたDDは、実際には使用されません。

主にプロシージャを呼び出すステップで使用されます。プロシージャ内でインストリーム・データを使用できないため、プロシージャ内のステップではDDNAMEを利用してデータセットの定義を先送りし、プロシージャを呼び出すステップでDDNAMEに指定したDDを設定して使用します。

以下のJCLでは、プロシージャ内のステップであるPSTEP1でSYSIN DDをOUTSTEP1と定義しておき、これを呼び出すためのステップであるSTEP1でOUTSTEP1を参照してPSTEP1.OUTSTEP1 DDとすることで、インストリーム・データを使用できるようになります。したがって、これらは同じDD属性を持ちます。TMAXSOFTはSYSINでインストリーム・データを取得します。

```
//JOB1    JOB
//PROC1   PROC
//PSTEP1  EXEC  PGM=TMAXSOFT
//SYSIN   DD   DDNAME=OUTSTEP1
//        PEND
//STEP1   EXEC  PROC1
//PSTEP1.OUTSTEP1 DD *
instream data
/*
//
```

以下は、DDNAMEオペランドの構文です。

- 構文

```
DDNAME = DD名
```

項目	説明
DD名	データセット設定を取得するDD名を記号名で指定します。

- 注意事項

- DDNAMEオペランドを使用したDDとDDNAMEオペランドで指定したDDは、同じステップ内に存在する必要があります。プロシージャ内のステップでDDNAMEオペランドを使用した場合は、そのプロシージャを呼び出すステップでDDNAMEオペランドで指定したDDが使用されると認識するため、同じステップ内にあるものと判断します。
- DDNAMEオペランドを使用したDDは、DDNAMEオペランドで指定したDDより先に現れる必要があります。他のDDで逆方向参照をした場合、DDNAMEオペランドで指定したDDは参照できません。
- DDNAMEオペランドで指定したDDはスペシャルDD名を使用してはなりません。このDDは、DDNAMEオペランドを使用したDDでのみ使用されるため、スペシャルDDとしての機能は実行できません。
- DDNAMEオペランドで指定したDDが存在しない場合は、DDNAMEオペランドを使用したDDはダミー・データセットと見なします。

● 例

以下は、SYSOUT DDがDD2の内容を参照し、SYSOUTデータセットとなる例です。DD2は参照されるだけで使用されません。

```
//STEP1 EXEC PGM=TMAXSOFT
//SYSOUT DD DDNAME=DD2
//DD2 DD SYSOUT=*
```

以下は、STEP2のINがSTEP1のOUTを逆方法参照する例です。参照される部分は、STEP1のDD2ではなく、STEP1のOUTです。

```
//STEP1 EXEC PGM=TMAXSOFT
//OUT DD DDNAME=DD2
//DD2 DD DSN=TEST.DATASET,DISP=(NEW,PASS)
//STEP2 EXEC PGM=TMAXSOFT
//IN DD DSN=*.STEP1.OUT,DISP=(OLD,DELETE)
//
```

2.3.16. DEST

SYSOUTデータセットの出力先を指定します。

● 構文

```
DEST = 出力先
```

項目	説明
出力先	SYSOUTデータセットの出力先を指定します。

● 注意事項

- SYSOUTデータセットの出力でなければ、DESTオペランドは無視されます。
- OpenFrameでは、印刷出力のためのDEST情報を外部プリンター・モジュールに渡すために、ジョブの実行後も保存します。この情報を使用するかどうかは、外部プリンター・モジュールの必要によって異なります。

- 例

以下は、外部プリンター・モジュールにDEST=LOCALの情報を渡す例です。

```
//EX1 DD SYSOUT=A,DEST=LOCAL
```

2.3.17. DISP

データセットの状態および後処理について設定します。

- 構文

```
DISP = ([NEW] [,DELETE] [,DELETE])
        [OLD] [,KEEP]    [,KEEP])
        [SHR] [,PASS]    [,CATLG])
        [MOD] [,CATLG]   [,UNCATLG])
        [,]    [,UNCATLG]
        [,]
```

- パラメータ1

データセットの状態を指定します。

項目	説明
NEW	データセットがジョブ・ステップで新規作成されることを示します。
OLD	データセットがジョブ・ステップの前にすでに存在していることを示します。
SHR	データセットがジョブ・ステップ前にすでに存在していることを示します。また、別のジョブで同時に使用されることを許可します。
MOD	データセットを拡張（追加書き込み）することを示します。 順次データセットを出力でオープンすると、そのまま追加書き込みができます。 MODパラメータを指定した場合、VOLUMEオペランドでSERパラメータまたはREFパラメータが指定されると、すでにデータセットが作成されているものとして処理します。 VOLUMEオペランドで上記の指定がない場合、指定したデータセットがパスされているか、カタログされていれば、データセットの作成済みと見なします。それ以外の場合は、NEWパラメータの指定と見なします。

– パラメータ2

ジョブ・ステップが正常終了した場合（ABENDでない場合）、データセットの後処理方法を指定します。

項目	説明
DELETE	ジョブ・ステップの終了後、データセットを削除します。 ボリューム情報をカタログから取得した場合、DELETE処理が正常に終了したときに限ってカタログから削除されます。
KEEP	データセットを保存します。VSAMデータセットが新しく作成される場合は、カタログ化まで自動的に行われます。ただし、データセットを作成したジョブ・ステップが異常終了(ABEND)した場合、パラメータ3のDELETEパラメータ指定があれば削除されます。
PASS	ジョブ・ステップ間のデータセットを使用した後、そのデータセットを後続ジョブ・ステップに渡します。後続ジョブ・ステップでデータセットを受け取った場合、そのデータセットを受け取るDD文にVOLUMEオペランドのSERパラメータまたはREFパラメータを指定してはなりません。 後続ジョブ・ステップで、PASSパラメータが指定されたデータセットを受け取るのは1回のみ可能です。そのため、データセットを受け取った後、もう一度、後続ジョブ・ステップに渡そうとする場合は、毎回DISPオペランドでPASSパラメータを指定する必要があります。 1つのジョブで同じデータセット名を持つデータセットが複数存在する場合、特定の時点で複数のデータセットを渡してはなりません。VSAMデータセットは渡すことができません。
CATLG	データセットをジョブ・ステップの終了時にカタログします。システムがカタログを参照してデータセットを割り当てた場合、CATLGパラメータの指定があれば、再カタログします。
UNCATLG	ジョブ・ステップの終了時にデータセットを保存しますが、カタログからカタログ情報は削除します。

– パラメータ3

ジョブ・ステップが異常終了（ABEND）した場合のデータセット処理を指定します。指定できるパラメータは、パラメータ2のPASSパラメータを除いては、パラメータ2と同じです。

● 注意事項

- パラメータ1のみ指定する場合、括弧で囲む必要はありません。
- DISPオペランドを省略した場合、DISP=(NEW,DELETE,DELETE)と処理されます。
- パラメータ1の指定を省略し、パラメータ2を指定した場合、パラメータ1としてNEWが指定されます。

- パラメータ1のみを指定して残りを省略すると、パラメータ1がNEWの場合は、パラメータ2と3はDELETEが指定され、パラメータ1がNEWでない場合は、パラメータ2と3は、KEEPが指定されます。
- パラメータ3を省略した場合は、パラメータ2に指定された値がパラメータ3に適用されます。
- 一時データセットを指定する際、DISPオペランドのパラメータ3にいかなるパラメータを指定しても、内部的にPASSパラメータと見なして処理します。
- VSAMデータセットに対してパラメータ2と3をDELETEに指定しなかった場合は、KEEPパラメータの指定と見なして処理します。

● 例

以下は、アクセス・ボリュームに一時データセットを直接作成し、ジョブ・ステップを終了する際に削除する例です。

```
//TEMP DD UNIT=DISK, SPACE=(TRK, (5, 1))
```

以下は、シリアル番号が333333のボリュームに作成されているデータセットORG.SOURCE.TESTの排他的な使用を要求する例です。ジョブ・ステップが終了する際にデータセットをカタログします。

```
//DD1 DD DSN=ORG.SOURCE.TEST, UNIT=8598,  
//      DISP=(OLD, CATLG),  
//      VOL=SER=333333
```

以下は、既にカタログされているデータセットORG.LIBの排他的な使用を要求する例です。ジョブ・ステップが終了する際にデータセットを削除します。データセットが削除されるだけでなく、カタログからも削除されます。

```
//DD2 DD DSN=ORG.LIB, DISP=(OLD, DELETE)
```

2.3.18. DLM

DATAオペランドと一緒に使用され、インストリーム・データの終わりを示す区切り文字を指定します。

一般的にインストリーム・データの終わりを意味する段落記号は「/」で指定します。ただしこの場合、インストリーム・データとして「/」や「/」で始まるデータは使用できなくなります。DLMオペランドを使用して段落記号を「/」以外のもので指定すると、インストリーム・データとして「/」や「/」で始まるデータ、つまりJES2 JCL文やJCL文も使用できるようになります。

● 構文

DLM = 段落記号

項目	説明
段落記号	インストリーム・データの終わりを2文字の英文字で指定します。

- 例

以下は、段落記号をAAと指定し、DD1のインストリーム・データとしてOS JCLが使用可能な例です。

```
//DD1      DD DATA,DLM=AA
//JOB1     JOB                                --|
//STEP1    EXEC PGM=TMAXSOFT                  | ---> DD1のインストリームデータ
//                                                --|
AA
//DD2     ...
```

2.3.19. DSID

OpenFrameでは構文解析のみサポートします。

- 構文

```
DSID = {id}
      {(id[,V])}
```

項目	説明
id	8文字以内の記号名を指定します。

2.3.20. DSNAME/DSN

データセットの名前を指定します。DSNAMEとDSNが同時に指定された場合、DSNAMEに指定された内容が使用されます。「&&」で始まる場合は一時データセットを意味し、アスタリスク(*)で始まる場合は、以前宣言されたDDのデータセット名をそのまま使用することを意味します。

- 構文

```
{DSNAME | DSN} = {データセット名          }
                  {データセット名(メンバー名)  }
                  {データセット名(世代番号)    }
                  {&&データセット名            }
                  {*.DD名                       }
                  {*.ステップ名.DD名            }
                  {*.ステップ名.プロシージャステップ名.DD名}
```

項目	説明
データセット名	DDで使用するデータセット名を記号名で指定します。

項目	説明
	データセット名は、"記号名[.<記号名>]..."形式で最大44桁まで指定できます。各記号名は最大8桁までで、GDGは最大35桁までです。データセット名に使用できる特殊文字は、@、#、\$です。
メンバー名	データセットのメンバー名をデータセットと一緒に指定します。1~8文字の記号名で指定します。データセット名に使用できる特殊文字は、@、#、\$です。
世代番号	世代別グループ・データセットの世代番号を符号と一緒に指定します。符号が省略された場合はプラス(+)と見なされます。世代番号は、世代別グループ・データセットの最大世代番号を超えてはなりません。世代別グループ・データセットについては、『OpenFrame データセットガイド』を参照してください。
&&データセット名	指定されたジョブでのみ一時的に使用するデータセットを指定します。一時データセットは基本的に桁数を占めているため、長いデータセット名を作成すると、エラーが発生する場合があります。そのため、一時データセットとして作成するデータセットは、「ピリオド(.)なしで<記号名>」のように作成することをお勧めします。
*.DD名 *.ステップ名.DD名 *.ステップ名.プロシージャステップ名.DD名	「*[.ステップ名[.プロシージャステップ名].DD名]」を逆方向参照といいます。同じジョブの先行DD文で指定されたデータセット名を使用することを意味します。先行DD文の位置が先行ジョブ・ステップなのか先行プロシージャ・ステップなのかに応じて、それぞれステップ名またはステップ名とプロシージャ・ステップ名を指定します。 逆方向参照が指定するDDが存在しない場合は、ジョブはフラッシュで処理されます。

以下は、前述の「&&データセット名」と関連する一時データセットの説明です。一時データセットは、以下のような形式で作成されます。

- 「&&データセット名」の場合

```
SYSyddddd.Thhmmss.RA000.jobname.データセット名.Hgg
```

項目	説明
yyddd	ジョブの実行日です。
hhmmss	ジョブの実行時間です。
jobname	ジョブ名です。
gg	01です。

- DSNオペランドが省略された場合

SYSydd . Thhmmss . RA000 . jobname . Rggnnnn

項目	説明
ydd	ジョブの実行日です。
hhmmss	ジョブの実行時間です。
jobname	ジョブ名です。
nnnn	DDテーブルの順番であり、ジョブ内ではユニークな番号です。

- 「&&データセット名」であり、かつSYSOUTデータセットの場合

jobid(userid.jobname.jobid.Dnnnnnn.データセット名)

項目	説明
userid	ジョブのユーザー名です。
jobname	ジョブ名です。
jobid	ジョブのJOBIDです。(JOB00001 ~ JOB99999)
nnnnnn	DDテーブルの順番で、ジョブ内ではユニークな番号です。

- DSNオペランドが省略されており、かつSYSOUTデータセットの場合

jobid(userid.jobname.jobid.Dnnnnnn)

項目	説明
userid	ジョブのユーザー名です。
jobname	ジョブ名です。
jobid	ジョブのJOBIDです。(JOB00001 ~ JOB99999)
nnnnnn	DDテーブルの順番で、ジョブ内ではユニークな番号です。

- 「&&データセット名」であり、かつインストリーム・データセットの場合

jobid(userid.jobname.jobid.Dnnnnnn.データセット名)

項目	説明
userid	ジョブのユーザー名です。
jobname	ジョブ名です。
jobid	ジョブのJOBIDです。(JOB00001 ~ JOB99999)
nnnnnn	DDテーブルの順番で、ジョブ内ではユニークな番号です。

- DSNオペランドが省略されているか、「&&データセット名」形式ではないが、インストリーム・データセットの場合

```
jobid(userid.jobname.jobid.Dnnnnnn)
```

項目	説明
userid	ジョブのユーザー名です。
jobname	ジョブ名です。
jobid	ジョブのJOBIDです。(JOB00001 ~ JOB99999)
nnnnnn	DDテーブルの順番で、ジョブ内ではユニークな番号です。

● 注意事項

- DSNオペランドを省略すると、該当するDDは一時データセットを使用すると判断しますが、データセットの状態をNEWまたはMODと指定しない場合は、ボリュームを割り当てるものと判断します。
- DSNオペランドが存在しており、「&&データセット名」形式ではないが、SYSOUTデータセットである場合、ジョブはフラッシュが発生します。
- SYSOUTデータセットは、DD文のSYSOUTオペランドでクラスまたはアスタリスク(*)が指定されている場合を意味します。

● 例

以下は、データセットをTMAX.DATASETに指定する例です。

```
//DD1 DD DSN=TMAX.DATASET, DISP=SHR
```

以下は、データセットの長さが44桁を超えた場合に構文エラーが発生する例です。

```
//DD1 DD DSN=TMAX.DATASET.OVERFLOW.LENGTH.COLUMN44.ISOCCUR.PARSE.ERROR
```

以下は、データセットとメンバーを指定する例です。

```
//DD1 DD DSN=TMAX.PDSLIB(DATASET), DISP=(NEW, DELETE, DELETE)
```

以下は、世代データセットを指定する例です。

```
//DD1 DD DSN=TMAX.GDG(+1), DISP=(NEW, DELETE, DELETE)
```

以下は、一時データセットを指定する例です。

```
//DD1 DD DSN=&&TEMP, DISP=(NEW, PASS)
```

以下は、一時データセットをSYSOUTデータセットに指定する例です。

```
//DD1 DD DSN=&&TEMP, SYSOUT=*
```

以下は、DSNオペランドを省略する例です。

```
//DD1      DD  DISP=(NEW,PASS)
```

以下は、ステップのデータセット名を参照する例です。例でのDD2のデータセット名は、TMAX.DATASETとなります。

```
//DD1      DD  DSN=TMAX.DATASET,DISP=SHR
//DD2      DD  DSN=*.DD1,DISP=SHR
```

以下は、以前のステップのデータセット名を参照する例です。例でのSTEP2 DD1のデータセット名は、TMAX.DATASETとなります。

```
//STEP1    EXEC PGM=TMAXSOFT
//DD1      DD  DSN=TMAX.DATASET,DISP=SHR
//STEP2    EXEC PGM=TMAXSOFT
//DD1      DD  DSN=*.STEP1.DD1,DISP=SHR
```

以下は、以前のプロシージャ・ステップのデータセット名を参照する例です。例でのSTEP2 DD1のデータセット名は、TMAX.DATASETとなります。

```
//JOB1     JOB  CLASS=A,MSGCLASS=A
//TMAXPROC PROC
//PSTEP1    EXEC PGM=TMAXSOFT
//PDD1     DD  DSN=TMAX.DATASET
//         PEND
//STEP1     EXEC TMAXPROC
//STEP2     EXEC PGM=TMAXSOFT
//DD1      DD  DSN=*.STEP1.PSTEP1.PDD1
```

2.3.21. DSNTYPE

データセットの形式を指定します。OpenFrameではLIBRARYとPDSのみをサポートします。

- 構文

```
DSNTYPE = {LIBRARY|HFS|PDS|PIPE|EXTREQ|EXTPREF|LARGE|BASIC}
```

項目	説明
LIBRARY	データセットのDSORGをPOと設定します。
PDS	データセットのDSORGをPSと設定します。

- 注意事項

DDオペランドDSORGが指定されると、このオペランドは無視されます。

- 例

以下は、DSNTYPEをPDSと指定する例です。

```
//DD1 DD DSN=NEW.DATASET, DISP=(NEW,CATLG), DSNTYPE=PDS
```

2.3.22. DUMMY

データセットがダミー・データセットであることを意味します。ダミー・データセットで指定されたDDにデータを記録しても、実際には記録されずに捨てられます。

DSNAME=NULLFILEを設定すると、ダミーを設定したのと同じ機能を行います。

- 構文

```
DUMMY
```

- 例

以下は、SYSOUTをダミーに指定する例です。

```
//SYSOUT DD DUMMY
```

2.3.23. DYNAM

OpenFrameでは構文解析のみサポートします。

- 構文

```
DYNAM
```

2.3.24. EXPDT

データセットの満了日を指定します。データセットを最初に作成する際のみ指定できます。

- 構文

```
EXPDT = {yyddd|yyyy/ddd}
```

項目	説明
{yyddd}	– yy: 2桁の年で記録します。（範囲: 0 ~ 99、1900年代として認識）
	– ddd: 日付を記録します。（範囲: 1~ 366）
{yyyy/ddd}	– yyyy: 年度を指定します。（範囲: 1900 ~ 2099）
	– ddd: 日付を記録します。（範囲: 1~ 366）

参考

EXPDT=99365またはEXPDT=1999/365を指定した場合、永久保存として処理されます。

- 注意事項

- LABELオペランドでEXPDTを指定した場合、このオペランドはオーバーライドされて無視されます。
- OpenFrameでは、保存期間が満了する前にデータセットを追加、修正、削除できないようにする機能はサポートしていません。保存期間が満了したデータセットをシステムが自動的に削除する機能のみ提供されます。

- 例

以下は、データセットの満了日を2010年12月31日に指定する例です。

```
//DD1 DD DSN=NEW.DATASET, DISP=(NEW,CATLG), EXPDT=2010/365
```

2.3.25. FCB

SYSOUTデータセットを出力するプリンターを指定します。出力時にフォーマットが可能です。

- 構文

```
FCB = fcb-name
```

項目	説明
fcb-name	1~4文字の記号名で指定します。

2.3.26. FILEDATA

OpenFrameでは構文解析のみサポートします。

- 構文

```
FILEDATA = {BINARY | TEXT }
```

2.3.27. FLASH

用紙に一定の書式、範囲を予め印刷する場合に使用するフィルム・オーバーレイの識別名と適用枚数を指定します。

- 構文

```
FLASH = (フィルム識別名 [, 適用枚数] )
```

項目	説明
フィルム識別名	プリンターが出力を開始する前にオペレータが設定するフィルム・オーバーレイの識別名を1~4文字で指定します。
適用枚数	指定したフィルム・オーバーレイを適用し、コピーする枚数を1~255の符号なし整数を指定します。（デフォルト値: 255）

- 注意事項

- SYSOUTデータセットの出力でなければ、FLASHオペランドは無視されます。
- OpenFrameでは、印刷出力のためのFLASH情報を外部プリンター・モジュールに渡すために、ジョブの実行後も保存します。この情報を使用するかどうかは、外部プリンター・モジュールの必要に応じて異なります。

- 例

以下は、外部プリンター・モジュールにFLASH=(FLM1,2)の情報を渡す例です。

```
//EXF1 DD SYSOUT=A, FLASH=(FLM1, 2)
```

2.3.28. FREE

OpenFrameでは構文解析のみサポートします。

- 構文

```
FREE = {END | CLOSE}
```

2.3.29. HOLD

SYSOUTデータセットの出力を保留するかどうかを指定します。

- 構文

```
HOLD = {YES | Y | NO | N}
```

項目	説明
YES Y NO N	SYSOUTデータセットの出力を保留するかどうかを指定します。 – YES: SYSOUTデータセットの出力をHOLDに指定します。 – NO: SYSOUTデータセットの出力をWRITEに指定します。

- 注意事項

SYSOUTデータセットの出力でない場合、HOLDオペランドは無視されます。

- 例

以下は、SYSOUTデータセットの出力をHOLDに指定する例です。

```
//SYSOUT DD SYSOUT=A,HOLD=YES
```

2.3.30. KEYLEN

新規データセットのキー長を指定します。KEYLENはISAMデータセットの作成時にのみ有効です。

- 構文

```
KEYLEN = 値
```

項目	説明
値	データセットのキーの長さです。1~255まで指定可能であり、レコード長以下である必要があります。

- 注意事項

- ISAMデータセットでない場合、KEYLENオペランドは無視されます。
- DCBオペランドのパラメータにKEYLENが指定された場合、このオペランドはオーバーライドされて無視されます。

- 例

以下は、ISAMデータセットのKEYLENを10と指定する例です。

```
//DD1 DD DSN=ISAM.NEW.DATASET, DISP=(NEW,CATLG),  
//      KEYLEN=10, DCB=(RECFM=FB, DSORG=IS, LRECL=72)
```

2.3.31. KEYOFF

新規データセットのキーが始まるオフセットを指定します。KEYOFFは、ISAMデータセットの作成時にのみ有効です。

- 構文

```
KEYOFF = 値
```

項目	説明
値	0~32760の符号なし整数を指定します。

- 注意事項

- ISAMデータセットでない場合、KEYOFFオペランドは無視されます。

- DCBオペランドのパラメータとしてKEYOFFが指定された場合、このオペランドはオーバーライドされます。

- 例

以下は、ISAMデータセットのKEYOFFを20に指定する例です。

```
//DD1 DD DSN=ISAM.NEW.DATASET, DISP=(NEW, CATLG),
//      KEYLEN=10, KEYOFF=20, DCB=(RECFM=FB, DSORG=IS, LRECL=72)
```

2.3.32. LABEL

データセットのラベルを指定します。

- 構文

```
LABEL = ([データセットシーケンス番号][,AL] [,PASSWORD][,IN] [,EXPDT={yyyy/ddd}])
          [,AUL][,NOPWREAD][,OUT]          {yyddd}
          [,BLP][,]          [,]          [,RETPD=nnnn]
          [,LTM]
          [,NL]
          [,NSL]
          [,SL]
          [,SUL]
          [,JL]
          [,JUL]
          [,]
```

項目	説明
データセットシーケンス番号	0~9999の符号なし整数を指定します。指定したデータセットのボリュームがテープ・ボリュームであり、OpenFrame環境設定のdsサブジェクト、 DATASET_DEFAULT セクションの USE_TAPE_FILESEQ キーの値が「YES」の場合に有効です。
EXPDT ={yyyy/ddd} {yyddd}	データセットの満了日を指定します。 データセットを最初に作成する際のみ指定できます。指定日以内にデータセットを追加・修正・削除することはできません。 - yyyy: 年度を指定します（範囲: 1900から2099） - yy: 2桁の年で記録します（範囲: 0から99、1900年代として認識） - ddd: 日付を記録します（範囲: 1から366） EXPDT=99365またはEXPDT=1999/365を指定した場合、永久保存として処理されます。
RETPD=nnnn	データセットの保存期間を日数で指定します。データセットを最初に作成する際のみ指定できます。

項目	説明
	– nnnn: 0~9999の符号なし整数を指定します。0を指定するとNONEに指定され、9999を指定した場合は、1999年12月31日に指定されます。 満了日が1999年365日となる値を指定した場合も、永久保存として処理されます。

参考

OpenFrameでは、データセットシーケンス番号、EXPDT、RETPDパラメータをサポートしています。その他のパラメータは、構文エラーのみをチェックし、使用されないため、本書では説明を省略します。

● 注意事項

- OpenFrameでは、保存期間が満了する前にデータセットを追加、修正、削除できないようにする機能はサポートしていません。
- 保存期間が満了したデータセットをシステムが自動的に削除する機能のみ提供されます。

● 例

以下は、新しいデータセットのMAX.PRESERVEを磁気テープ・ボリュームに作成し、永久保存することを指定する例です。

```
//DD2 DD DSN=TMAX.PRESERVE,UNIT=848X-1,
//      DISP=(,KEEP),VOL=SER=111111,
//      LABEL=EXPDT=99365
```

2.3.33. LGSTREAM

OpenFrameでは構文解析のみサポートします。

● 構文

```
LGSTREAM = 値
```

項目	説明
データセット名	ピリオド(.)を除く、26文字以内の記号名を指定します。

2.3.34. LIKE

既存のデータセットの属性を新しく作成したデータセットの属性にコピーします。既存のデータセットはカタロギングされている必要があります。LIKEパラメータによってコピーされる属性は、

DSORG、RECFM、KEYLEN、LRECL、BLKSIZE、そしてSPACEパラメータに関連した属性値です。

- 構文

```
LIKE = データセット名
```

項目	説明
データセット名	1～8文字の記号名で指定するか、「記号名[.記号名]」形式で最大44桁まで指定します。

- 注意事項

指定したデータセットがカタロギングされていない場合、エラーを発生します。

- 例

以下は、新規データセットの属性をLIKE.DATASETの属性から取得する例です。

```
//DD1 DD DSN=NEW.DATASET,DISP=(NEW,CATLG),LIKE=LIKE.DATASET
```

2.3.35. LRECL

新規作成されるデータセットのレコード長を指定します。

レコード形式が固定長ブロック・レコードあるいはスパン(Span)レコードの場合に必ず指定します。スパン・レコードでない場合、LRECLオペランドで指定したレコード長は、BLKSIZEオペランドで指定したブロック長を超えることができません。各レコード形式で指定できる最大値は、固定長または長さが未指定のレコードの場合、32760バイトです。

以下は、各レコード形式のレコード長を指定する方法です。

– レコード長を指定します。

```
RECFM={F|FB}
```

– レコードの最大長を指定します。4バイトRDWを含む長さです。

```
RECFM={V|VB|VS|VBS}
```

– レコードの最大長を指定します。

```
RECFM=U
```

以下は、LRECLオペランドの構文です。

- 構文

```
LRECL = 値
```

項目	説明
値	データセットのレコード長であり、0~32760の符号なし整数を指定します。

- 注意事項

- DCBオペランドのパラメータにLRECLが指定された場合、このオペランドはオーバーライドされて無視されます。
- 非VSAMデータセットの最大値は32760です。

- 例

以下は、レコード長を70に指定する例です。

```
//DD1 DD DSN=NEW.DATASET,DISP=(NEW,CATLG),LRECL=70,DCB=(RECFM=FB)
```

2.3.36. MGMTCLAS

SMSマネージメント・クラスを指定します。マネージメント・クラスを指定すると、OpenFrame環境設定の**sms**サブジェクトに指定されているマネージメント・クラスから該当するクラスを検索し、記述されているデータセットのマネージメント関連の設定を使用します。

参考

smsサブジェクトの詳細については、『OpenFrame 環境設定ガイド』を参照してください。

- 構文

```
MGMTCLAS = クラス名
```

項目	説明
クラス名	マネージメント・クラス名を1~8文字の記号名で指定します。指定しない場合は、デフォルト値が指定されます。

- 注意事項

OpenFrame環境設定の**sms**サブジェクトにマネージメント・クラスが設定されていない場合は、エラーが発生します。

- 例

以下は、マネージメント・クラスをMGMT01と指定する例です。

```
//DD1 DD DSN=NEW.DATASET,DISP=(NEW,CATLG),MGMTCLAS=MGMT01
```

2.3.37. MODIFY

SYSOUTデータセットのコピー修飾モジュール名とCHARSオペランドで指定するテーブルのシーケンス番号を指定します。

- 構文

```
MODIFY = (モジュール名[, テーブル番号])
```

項目	説明
モジュール名	コピー修飾モジュール名を1~4文字の(ピリオドを除く)記号名で指定します。
テーブル番号	CHARSオペランドで指定されたテーブルのシーケンス番号(0~3)を指定します。

- 注意事項

- SYSOUTデータセットの出力でなければ、MODIFYオペランドは無視されます。
- OpenFrameでは、印刷出力のためのMODIFY情報を外部プリンター・モジュールに渡すために、ジョブの実行後も保存します。この情報を使用するかどうかは、外部プリンター・モジュールの必要によって異なります。

- 例

以下は、外部プリンター・モジュールにCHARS=(TBJ0,TBJ1)、MODIFY=(TLE2,2)の情報を渡す例です。

```
//OUT1 DD SYSOUT=A,CHARS=(TBJ0,TBJ1),  
//      MODIFY=(TLE2,2)
```

2.3.38. OUTLIM

SYSOUTデータセットに出力できるレコードの最大数を指定します。JCLにOUTLIMがない場合は、OpenFrame環境設定のtjclrunサブジェクトのDDセクションのOUTLIMキーのVALUE項目が適用されます。

参考

tjclrunサブジェクトの詳細については、『OpenFrame 環境設定ガイド』を参照してください。

- 構文

```
OUTLIM = 値
```


項目	説明
値	0~16777215の符号なし整数を指定します。

- 注意事項

OUTLIMを0に設定すると、値に制限がないことを意味します。

2.3.39. OUTPUT

リファレンスするためのOUTPUT文を指定します。OUTPUTオペランドを使用してOUTPUT文をリファレンスすると、該当OUTPUT文で指定した設定で出力処理をします。また、OUTPUT文を複数リファレンスすると、1つのSYSOUTデータセットをリファレンスした各OUTPUT文の設定に従って出力処理をします。

- 構文

```
OUTPUT = (リファレンス[,リファレンス]...)
```

項目	説明
リファレンス	リファレンスは、以下のように3つの形式で指定できます。 – *.名称 : リファレンスするOUTPUT文がJOB文と最初のSTEP文の間にある場合、または同じステップ内にある場合、該当OUTPUT文の名称を指定します。 – *.ステップ名.名称 : リファレンスするOUTPUT文が他のステップ内にある場合、ステップ名と一緒に該当OUTPUT文の名称を指定します。 – *.ステップ名.プロシージャステップ名.名称 : リファレンスするOUTPUT文がステップ内のプロシージャステップ内にある場合、ステップ名と一緒に該当OUTPUT文の名称を指定します。

- 注意事項

リファレンスするOUTPUT文が存在しない場合、エラーが発生します。

- 例

以下は、OUTPUT文をリファレンスする例です。

```
//JOB1      JOB      CLASS=A,MSGCLASS=A
//OUT1      OUTPUT   COPIES=5,OUTDISP=WRITE
//STEP1     EXEC     PGM=TMAXTEST
//OUT2      OUTPUT   COPIES=3,OUTDISP=HOLD
//OUTDD1    DD       SYSOUT=*,OUTPUT=( *.OUT1, *.OUT2)
//SYSOUT    DD       SYSOUT=*
//STEP2     EXEC     PGM=TMAXTEST
//OUTDD2    DD       SYSOUT=*,OUTPUT=*.STEP1.OUT2
```

```
//SYSOUT DD SYSOUT=*  
//
```

上記例の場合、OUTDD1 SYSOUTデータセットはOUT1とOUT2をリファレンスし、OUT1の設定で1つ、OUT2の設定で1つを出力処理をします。OUTDD2 SYSOUTデータセットはSTEP1のOUT2をリファレンスし、OUT2の設定で出力処理をします。結果的に上のJCLは、OUTDD1で2つ、OUTDD2で1つ、各ステップのSYSOUTで1つずつ、5つの出力処理をします。

2.3.40. PATH

UNIXファイルのパスを指定します。このパラメータは、TSOコマンドOCOPY文を使用する場合にのみ有効です。

- 構文

```
PATH = 'file-path'
```

2.3.41. PATHDISP

PATHパラメータで指定したUNIXファイルの状態および後処理の設定を行います。このパラメータは、TSOコマンドOCOPY文を使用する場合にのみ有効です。

- 構文

```
PATHDISP = ([KEEP][,KEEP] )  
            ([DELETE][,DELETE])
```

– パラメータ1

ジョブ・ステップが正常終了した場合（ABENDでない場合）のファイルの後処理方法を指定します。何も指定しない場合、デフォルト値は両方ともKEEPになります。

項目	説明
DELETE	ジョブ・ステップが終了した後、ファイルを削除します。
KEEP	ジョブ・ステップが終了した後、ファイルを保存します。

– パラメータ2

ジョブ・ステップが異常終了（ABEND）した場合のファイルの処理方法を指定します。

項目	説明
DELETE	ジョブ・ステップが終了した後、ファイルを削除します。
KEEP	ジョブ・ステップが終了した後、ファイルを保存します。

2.3.42. PATHMODE

PATHパラメータで指定したUNIXファイルを作成する際に、ユーザーごとにファイルへのアクセス・モードを指定します。ファイルの作成は、「PATHOPTS=OCREAT」の設定によって実行されます。このパラメータは、TSOコマンドOCOPY文を使用する場合にのみ有効です。

● 構文

```
PATHMODE = {file-access-attribute} |  
            {(file-access-attribute[,file-access-attribute]...)}
```

項目	説明
file-access-attribute	クラスのタイプごとに以下のように指定することができます。 – 所有者クラス • SIRUSR : ファイル所有者にファイルの読み取り権限を与えます。 • SIWUSR : ファイル所有者にファイルの書き込み権限を与えます。 • SIXUSR : ファイルがディレクトリである場合、ファイル所有者にファイルの検索権限を与えます。ディレクトリでない場合は、ファイル所有者にファイルの実行権限を与えます。 • SIRWXU : ファイルがディレクトリである場合、ファイル所有者にファイルの読み取り/書き込み権限と検索権限を与えます。ディレクトリでない場合は、ファイル所有者にファイルの読み取り/書き込み権限と実行権限を与えます。 – グループ・クラス • SIRGRP : ファイル・グループ・クラスのユーザーにファイルの読み取り権限を与えます。 • SIWGRP : ファイル・グループ・クラスのユーザーにファイルの書き込み権限を与えます。 • SIXGRP : ファイルがディレクトリである場合、ファイル・グループ・クラスのユーザーにファイルの検索権限を与えます。ディレクトリでない場合は、ファイル・グループ・クラスのユーザーにファイルの実行権限を与えます。 • SIRWXG : ファイルがディレクトリである場合、ファイル・グループ・クラスのユーザーにファイルの読み取り/書き込み権限と検索権限を与えます。ディレクトリでない場合は、ファイル・グループ・クラスのユーザーにファイルの読み取り/書き込み権限と実行権限を与えます。 – その他のクラス • SIROTH : その他のクラスのユーザーにファイルの読み取り権限を与えます。

項目	説明
	• SIWOTH: その他のクラスのユーザーにファイルの書き込み権限を与えます。 • SIXOTH: ファイルがディレクトリである場合、その他のクラスのユーザーにファイルの検索権限を与えます。ディレクトリでない場合は、その他のクラスのユーザーにファイルの実行権限を与えます。 • SIRWXO: ファイルがディレクトリである場合、その他のクラスのユーザーにファイルの読み取り/書き込み権限と検索権限を与えます。ディレクトリでない場合は、その他のクラスのユーザーにファイルの読み取り/書き込み権限と実行権限を与えます。

2.3.43. PATHOPTS

PATHパラメータで指定したUNIXファイルのアクセス・オプションと状態オプションを指定します。このパラメータは、TSOコマンドOCOPY文を使用する場合にのみ有効です。

- 構文

```
PATHOPTS = {file-option} | {(file-option[,file-option]...)}
```

項目	説明
file-option	アクセス・オプションと状態オプションを以下のように指定することができます。 – アクセス・オプション • ORDONLY: ファイルを読み取り専用で開きます。 • OWRONLY: ファイルを書き込み専用で開きます。 • ORDWR: ファイルを読み取りと書き込みの両方のモードで開きます。 – 状態オプション • OAPPEND: PATHパラメータで指定したファイルに書き込む際に、既存のファイルの末尾に続けて書き込むように指定します。 • OCREAT: ファイルが存在しない場合は新規作成し、すでに存在する場合はそのファイルを開きます。パスに指定したディレクトリが存在しない場合は、新しいファイルを作成しません。 • OEXCL: ファイルが存在しない場合は新規作成し、すでに存在する場合はエラーを返し、そのジョブ・ステップの割り当てに失敗します。OCREATと一緒に指定していない場合、この設定は無視されます。

項目	説明
	• ONOCTTY: PATHパラメータが端末デバイスを指定している場合、その端末デバイスがプロセスの制御端末にならないように指定します。 • ONONBLOCK: 読み取りの際に、読み取る内容がない場合、待たずに直ちに結果値を返すように指定します。 • OSYNC: 書き込みの際に、実際に書き込みが完了した後に結果値を返すように指定します。 • OTRUNC: PATHパラメータで指定したファイルがすでに存在する場合、ファイルを上書きするように指定します。

2.3.44. PROTECT

OpenFrameでは構文解析のみサポートします。

- 構文

```
PROTECT = {YES | Y}
```

2.3.45. RECFM

データセットのレコード形式と特性を指定します。OpenFrameでは、以下のレコード形式のみサポートします。

- 構文

```
RECFM={F[B][A|M]}
       {V[B|S|BS][A|M]}
       {L[A|M]}
       {U[A|M]}
```

項目	説明
A	ASAコードを含むレコード形式です。
B	ブロック・レコード形式です。
F	固定長レコード形式です。
L	ライン単位レコード形式です。 Lレコード形式は、オープン環境でテキスト・ファイルを保存する一般的なレコード形式です。OpenFrameに移行した後、データ連携を円滑に行うために新しく追加したレコード形式です。
M	機械制御文字を含むレコード形式です。

項目	説明
S	スパン・レコード形式です。
V	可変長レコード形式です。
U	長さが未指定のレコード形式です。

- 注意事項

DCBオペランドのパラメータにRECFMが指定された場合、このオペランドはオーバーライドされて無視されます。

- 例

以下は、RECFMをFBに指定する例です。

```
//DD1 DD DSN=NEW.DATASET,DISP=(NEW,CATLG),RECFM=FB,LRECL=80
```

2.3.46. REORG

VSAMデータセットを新しく作成する場合に指定する必要があるパラメータとして、生成されるVSAMデータセットのタイプを指定します。

- 構文

```
REORG = {KS | ES | RR | LS}
```

項目	説明
KS	キー順データセットです。
ES	入力順データセットです。
RR	固定長相対レコード・データセットです。
LS	線形データセットです。OpenFrameではサポートされていません。

- 例

以下は、REORGをKSに指定してVSAM KSDSを作成する例です。

```
//DD1 DD DSN=NEW.DATASET,DISP=(NEW,KEEP,DELETE),REORG=KS,LRECL=80
```

2.3.47. REFDD

以前に設定したDDの属性をリファレンスします。REFDDパラメータによってコピーされる属性は、DSORG、RECFM、KEYLEN、LRECL、BLKSIZEパラメータ関連の属性値です。

- 構文

```
REFDD = {*.DD名}
        {*.ステップ名.DD名}
        {*.ステップ名.プロセスステップ名.DD名}
```

項目	説明
.DD名	「[.ステップ名[.プロセスステップ名].DD名」を逆方向参照といいます。
*.ステップ名.DD名	
*.ステップ名.プロセスステップ名.DD名	
	同一ジョブの先行DD文で指定されたデータセット名を使用することを意味します。
	先行DD文の位置が先行ジョブステップなのか、先行プロセスステップなのかによって、各ステップ名またはステップ名とプロセスステップ名を指定します。

- 注意事項

逆方向参照が指定するDDが存在しない場合、ジョブはフラッシュ処理されます。

- 例

以下は、REFDDを使用してDD1のRECFMとLRECLの属性を取得する例です。

```
//DD1 DD DSN=NEW.DATASET,DISP=(NEW,CATLG),RECFM=FB,LRECL=80
//DD2 DD DSN=NEW2.DATASET,DISP=(NEW,CATLG),REFDD=*.DD1
```

2.3.48. RETPD

データセットの保存期間を日数で指定します。データセットを最初に作成する際のみ指定できます。

- 構文

```
RETPD = nnnn
```

項目	説明
nnnn	データセットの保存期間を0~9999の符号なし整数を指定します。

- 注意事項

- LABELオペランドでRETPDを指定した場合、このオペランドはオーバーライドされて無視されます。
- OpenFrameでは、保存期間が満了する前にデータセットを追加、修正、削除できないようにする機能はサポートしていません。保存期間が満了したデータセットをシステムが自動的に削除する機能のみ提供されます。

- 例

以下は、データセットの満了日を、現在の日付から30日間を指定する例です。

```
//DD1 DD DSN=NEW.DATASET,DISP=(NEW,CATLG),RETPD=30
```

2.3.49. RLS

OpenFrameでは構文解析のみサポートします。

- 構文

```
RLS = {NRI | CR}
```

2.3.50. SECMODEL

OpenFrameでは構文解析のみサポートします。

- 構文

```
SECMODEL = 値
```

2.3.51. SEGMENT

OpenFrameでは構文解析のみサポートします。

- 構文

```
SEGMENT = 値
```

2.3.52. SPACE

直接アクセス・ボリュームに新規データセットのスペース割り振り量を指定します。

- 構文

```
SPACE = ( {TRK,} (初期値1[,追加値1][,ディレクトリ1])[,RLSE] [,CONTIG][,ROUND])  
          {CYL,}      [,]                [,]      [,MXIG]  
          {ブロック長,}                                [,ALX]  
          {レコード長,}                                [,]
```

以下は、パラメータについての説明です。

パラメータ	説明
TRK	スペース割り振り量の初期値および追加値の単位がトラックであることを示します。

パラメータ	説明
CYL	スペース割り振り量の初期値および追加値の単位がシリンダーであることを示します。
ブロック長	0~65535の符号なし整数を指定します。 スペース割り振り量の初期値および追加値の単位がブロックであることを示します。AVGRECオペランドを指定していない場合に、ブロック長として処理されます。
レコード長	0~65535の符号なし整数を指定します。 スペース割り振り量の初期値および追加値の単位がレコードであることを示します。AVGRECオペランドを指定している場合に、レコード長として処理されます。
初期値	データセットの最初の割り振りサイズを指定します。 0~16777215の符号なし整数を指定します。
追加値	データセットの作成中にスペースの不足により追加が必要な場合、一度に追加する値を指定します。スペースの追加割り振りが許可される最大回数は、基本的に15回までです。OpenFrameでは、非VSAMデータセットの場合、OpenFrame環境設定のdsサブジェクト、 DATASET_DEFAULT セクションの NVSM_EXTENT_LIMIT キーのVALUE項目に指定できます。dsサブジェクトの詳細については、『OpenFrame 環境設定ガイド』を参照してください。 0~16777215の符号なし整数を指定します。
ディレクトリ	区分データセット(PDS)を作成する際、ディレクトリ領域のスペース割り振り量を指定します。0~16777215の符号なし整数を指定します。 このパラメータが指定された場合、区分データセットとしてのみ見なされます。OpenFrameでは構文チェックのみ行い、実際に指定された値によるスペース割り振りには使用しません。

参考

RLSE、CONTIG、MXIG、ALX、ROUNDパラメータはサポートしていません。

● 注意事項

- SPACEオペランドとDDNAME、AMP、DLM、アスタリスク(*)またはDATAオペランドは互いに排他的であるため、同時に指定してはなりません。
- VSAMデータセットを作成する場合、SPACEオペランドを指定してはなりません。

● 例

以下は、JOB1のDD 文で新しく作成するデータセット・データに20トラックを割り振って、データの記録中にスペースが不足したら、そのたびに5トラックずつ追加のスペースを割り振る例です。RLSEパラメータは、作成を終了して閉じるときに、未使用スペースを解除しますが、OpenFrameでは無視されます。

```
//JOB1 JOB
//      EXEC PGM=PROG1
//DS1 DD DSN=DATA,UNIT=8598,
//      VOL=SER=A11111,
//      SPACE=(TRK,(20,5),RLSE)
//
```

以下は、JOB2のDD文で新しく作成する区分データセットに連続した（CONTIGの機能）10シリンダーを割り振り、スペースの先頭のディレクトリ部にディレクトリ・ブロックを20個割り当てる例です。

OpenFrameでは、20個のディレクトリ・ブロック数とCONTIGパラメータの指定は無視されます。ディレクトリ・ブロック数が指定されているということから、区分データセットを新しく作成します。

```
//JOB2 JOB
//      EXEC PGM=PROG2
//DS2 DD DSN=DATA,UNIT=8598,
//      VOL=SER=B11111,
//      SPACE=(CYL,(10,,20),,CONTIG)
//
```

2.3.53. SPIN

OpenFrameでは構文解析のみサポートします。

- 構文

```
SPIN = {UNALLOC | NO}
```

2.3.54. STORCLAS

SMSストレージ・クラスを指定します。ストレージ・クラスを指定すると、OpenFrame環境設定の **sms** サブジェクトに指定されているストレージ・クラスに記述されているデータセットのストレージ関連設定を使用します。

参考

smsサブジェクトの詳細については、『OpenFrame 環境設定ガイド』を参照してください。

- 構文

STORCLAS = クラス名

項目	説明
クラス名	ストレージ・クラス名は1～8文字の記号名で指定します。

- 注意事項

OpenFrame環境設定の**sms**サブジェクトにストレージ・クラスが設定されていない場合は、エラーが発生します。

- 例

以下は、ストレージ・クラスをSTOR01と指定する例です。

```
//DD1 DD DSN=NEW.DATASET,DISP=(NEW,CATLG),STORCLAS=STOR01
```

2.3.55. SUBSYS

OpenFrameでは構文解析のみサポートします。

- 構文

SUBSYS = 値

2.3.56. SYMBOLS

インストリーム・データで記号パラメータを使用します。

- 構文

SYMBOLS = { JCLONLY | EXEC SYS | CNVTSYS }

項目	説明
JCLONLY	DD文の前にEXPORT文で指定した記号パラメータをインストリーム・データで使用します。使用される記号パラメータは、SET文で指定された値で置き換えられます。
EXEC SYS	OpenFrameでは使用されません。JCLONLYを指定した場合と同様に動作します。
CNVTSYS	OpenFrameでは使用されません。JCLONLYを指定した場合と同様に動作します。

- 例

以下は、A1をTMAXに置き換える例です。B1もSET文で指定していますが、EXPORT文でA1記号パラメータのみエクスポートしているので、インストリーム・データではA1記号パラメータのみ置き換えられます。

```
//EX1      EXPORT SYMLIST=A1
//SET1     SET   A1=TMAX, B1=SOFT
//SORTIN   DD    *,SYMBOLS=JCLONLY
1000&A1
2000&B1
/*
```

2.3.57. SYSOUT

SYSOUTデータセットの出力プロパティを指定します。

- 構文

```
SYSOUT = ([クラス][,プログラム名][,書式番号])
          [*]
```

項目	説明
クラス	SYSOUTデータセットの出力クラスを指定します。出力クラスの属性に応じて、SYSOUTデータセットの出力および出力保留などを指定します。 SYSOUT出力クラスの設定は、OpenFrame環境設定の tjes サブジェクトの OUTCLASS セクションのクラス名をキーとして使用します。詳細については、『OpenFrame 環境設定ガイド』を参照してください。
アスタリスク(*)	SYSOUTデータセットの出力クラスをシステム・メッセージ・クラスと同様に設定することを意味します。 システム・メッセージ・クラスは、JOB文でMSGCLASSオペランドを使用して指定します。
プログラム名	内部読み取りまたは外部書き出しを使用してSYSOUTデータセットを出力する場合、そのプログラム名を指定します。 内部読み取りを使用して出力する場合はINTRDRを指定します。
書式番号	SYSOUTデータセットを出力する用紙の書式番号を指定します。1~4文字のピリオド(.)を含まない記号名で指定します。

- 例

以下は、出力クラスを8、外部書き出しプログラム名をUSERWTR、出力先をGROUPAに指定する例です。この情報を外部プリンター・モジュールに渡します。

```
//OUT DD SYSOUT=(8,USERWTR),DEST=GROUPA
```

2.3.58. TERM

OpenFrameでは構文解析のみサポートします。

- 構文

```
TERM = 値
```

2.3.59. UCS

OpenFrameでは構文解析のみサポートします。

- 構文

```
UCS = 値
```

2.3.60. UNIT

データセットを割り当てるI/Oデバイスを指定します。

- 構文

```
UNIT = ([ddd]          [, 個数][, DEFER])
        [/ddd]         [, P]
        [/dddd]        [, ]
        [デバイス・タイプ]
        [デバイス・グループ]
UNIT = AFF = DD名
```

項目	説明
/dddd	デバイス番号を指定します。16進数で0000からFFFFまで指定可能です。
デバイス・タイプ	デバイス・タイプを指定します。システムは、指定されたタイプのデバイスから1つを選択します。デバイス・タイプは英数字およびハイフン(-)で構成された1~8の文字列で指定します。
デバイス・グループ	デバイス・グループを指定します。デバイス・グループは英数字およびハイフン(-)で構成された1~8の文字列で指定します。

参考

OpenFrameでは、/dddd、デバイス・タイプ、デバイス・グループのみサポートします。

- 注意事項

以下の場合、デバイス情報をシステムが判断できるため、UNITオペランドは省略できます。

- データセットがカタログされている場合

- データセットが先行ステップからパスされた場合
- VOLUME=REFの指定によって、先行DD文またはカタログされているデータセットを参照した場合

OpenFrameでは、メインフレームのデバイスを使用するのではなく、UNIX OSのファイル・システム上にデータセットを実装しているため、デバイスへの割り当ては行いません。ただし、このオペランドによって指定されるデバイス情報を記録しておき、以下のような用途で使

- デバイスのタイプごとに異なる処理が必要な場合に活用します。直接アクセス・デバイスなのか、あるいはテープ・デバイスなのかを区別するために使用します。
- デバイスが決まり、ボリュームが確定された場合、決定されたデバイスからボリュームのシリアル番号を確認するときに使用します。

● 例

以下は、EXDD1 DD文でボリュームのシリアル番号がVOL001のボリュームに一時データセットを新規作成する例です。3390というデバイスから、1台のデバイスを要求しています。

```
//EXDD1 DD UNIT=3390, SPACE=(CYL,(1,1,1)),
//          VOL=SER=VOL001
```

以下は、EXDD DD文でVOL001ボリュームに一時データセットを新規作成することをシステムに要求する例です。UNITオペランドではSYSDAというデバイス・グループのうち、1台のデバイスを割り当てるように要求しています。

```
//EXDD DD UNIT=SYSDA, SPACE=(CYL,(1,1)), VOL=VOL001
```

2.3.61. VOLUME/VOL

データセットのボリュームを指定します。

● 構文

```
{VOLUME | VOL}
=( [PRIVATE] [, RETAIN] [, ボリュームの順序番号] [, 個数] [, ] [SER=ボリュームのシリアル番号] )
    [, ]      [, ]      [SER=(ボリュームのシリアル番号[, ボリュームのシリアル番号]...)]
    [REF=[データセット名]]
    [REF=* .DD名]
    [REF=* .ステップ名 .DD名]
    [REF=* .ステップ名 .プロシージャステップ名 .DD名]
```

以下は、パラメータについての説明です。

項目	説明
SER=ボリュームのシリアル番号、	データセットを作成するボリューム、またはデータセットが存在するボリュームのシリアル番号を指定します。ボリュームのシリアル番号は1~6文字の引用符付き文字列で指定します。

項目	説明
SER=(ボリュームのシリアル番号 [,ボリュームのシリアル番号]...)	VOLUMEオペランドでSERパラメータのみ指定した場合は、 VOLUME=SER=(ボリュームのシリアル番号,...)と指定できます。 ボリュームのシリアル番号を1つのみ指定する場合、括弧は省略できます。指定したボリュームのシリアル番号は、 OpenFrameのvolmgrツールによって事前に指定されている必要があります。
REF=データセット名	指定したデータセットが使用するボリュームと同じボリュームを使用するように指定します。REFパラメータで指定するデータセット名は、世代別データ・グループ名(GDG)と異なる必要があります。また、GDG相対番号メンバー名は使用できません。指定されたデータセットはカタログされている必要があります。 OpenFrameでは、カタログされずにパスされたDD文のデータセット名で参照することはサポートしていません。
REF=*.DD名 REF=*.ステップ名.DD名 REF=*.ステップ名.プロシージャ ステップ名.DD名	「*[.ステップ名[.プロシージャステップ名].DD名」を逆方向参照といいます。同じジョブの先行DD文で指定されたボリュームを使用することを意味します。先行DD文の位置が先行ジョブ・ステップなのか先行プロシージャ・ステップなのかに応じて、それぞれステップ名またはステップ名とプロシージャ・ステップ名を指定します。 VOLUMEオペランドでREFパラメータのみ指定した場合、 VOLUME=REF=(逆方向参照,...)の形式で指定できます。 逆方向参照が指定するDDが存在しない場合、ジョブはフラッシュとして処理されます。

参考

PRIVATE、RETAIN、ボリュームの順序番号、個数のパラメータはサポートしていません。

● 例

以下は、EXDD01 DD文でボリュームのシリアル番号がVOL001のボリュームに存在する既存のデータセットの使用を要求する例です。

```
//EXDD01 DD UNIT=848X,VOL=SER=VOL001,
//          DISP=(OLD,KEEP),DSN=TAPEDS1
```

以下は、カタログされているデータセットFILE.ORGが存在する直接アクセス・ボリュームにデータセットFILE.AUTHを作成する例です。

```
//DD1 DD DSN=FILE.AUTH,DISP=(,KEEP),VOL=REF=FILE.ORG,  
//      SPACE=(CYL,(5,1))
```

以下は、先行ジョブ・ステップのDD文DD1に定義されたデータセット・ファイルを拡張する例です。OpenFrameでは、PRIVATEパラメータとボリューム・カウント2の指定は無視されます。

```
//STEP1 EXEC PGM=BUILD  
//DD1 DD DSN=FILE,DISP=(OLD,KEEP),VOL=SER=111111,  
//      UNIT=8598  
//STEP2 EXEC PGM=EXPAND  
//DD2 DD DSN=FILE,DISP=(MOD,KEEP),  
//      VOL=(PRIVATE,,,2,REF=*.STEP1.DD1)
```

2.4. スペシャルDD文

特別な名前のDD名を使用するDD文について説明します。OpenFrameでは、以下のDD以外はスペシャルDDとして扱いません。

2.4.1. SYSIN DD

ステップのプログラムで使用する標準入力(stdin)データが存在するデータセットを指定します。EXEC文の後から次のEXEC文の間、またはジョブの終わりの間に記述します。

以下は、TMAXSOFTプログラムが標準入力ですysin dataというデータを取得する例です。

```
//STEP1 EXEC PGM=TMAXSOFT  
//SYSIN DD *  
sysin data  
/*
```

以下は、TMAXSOFTプログラムが標準入力ですTMAXSOFT.DATAというデータセットの内容を取得する例です。

```
//STEP1 EXEC PGM=TMAXSOFT  
//SYSIN DD DSN=TMAXSOFT.DATA,DISP=SHR
```

2.4.2. SYSOUT DDとSYSPRINT DD

SYSOUT DDとSYSPRINT DDは、該当するステップのプログラムで出力する標準出力(stdout)と標準出力エラー(stderr)を保存するデータセットを指定します。

SYSOUT DDとSYSPRINT DDが両方とも指定されている場合はSYSOUT DDが使用され、SYSPRINTはいかなる内容も記述されません。EZTPA00のように、他の製品との連携のために使用されるユーティリティでは、SYSPRINT DDが別の用途で使われることもあります。EXEC文の後から次のEXEC文の間、またはジョブの終わりの間に記述します。

参考

EZTPA00でのSYSPRINT DDの使用については、『OpenFrame ユーティリティリファレンスガイド』を参照してください。

以下は、TMAXSOFTプログラムから出力された標準出力(stdout)と標準出力エラー(stderr)を、SYSOUT DDと指定した一時ファイルに記録する例です。

```
//STEP1 EXEC PGM=TMAXSOFT
//SYSOUT DD DSN=SYSOUT, DISP=(NEW,PASS), VOLUME=100000
```

以下は、SYSOUT DDとSYSPRINT DDと一緒に使用すると、SYSOUT DDにTMAXSOFTプログラムから出力された標準出力(stdout)と標準出力エラー(stderr)が記録される例です。

```
//STEP1 EXEC PGM=TMAXSOFT
//SYSOUT DD *
//SYSPRINT DD *
```

2.4.3. JOBLIB DDとSTEPLIB DD

JOBLIB DDとSTEPLIB DDは、両方ともステップで実行するプログラムを検索ために使用されます。

- JOBLIB DD

JOBLIB DDで指定したものは、ジョブ内のすべてのステップで実行するプログラムを検索する際に参照します。JOBLIB DDは、JOB文の後から、最初のEXEC文が現れる前に記述します。

- STEPLIB DD

STEPLIB DDで指定したものは、ステップで実行するプログラムを検索する際に参照します。STEPLIB DDは、EXEC文の後から次のEXEC文の間、またはジョブの終わりの間に記述します。

STEPLIB DDとJOBLIB DDが両方とも記述されると（STEPLIB DD/JOBLIB DD/tjclrun.confファイルの[SYSLIB]設定）順序どおり使用されます。STEPLIB DDとJOBLIB DDが両方とも存在しない場合は、tjclrun.conf [SYSLIB]設定を使用します。

参考

プログラムの検索順の詳細については、[「2.6.8. PGM」](#)を参照してください。

以下は、TMAXSOFTプログラムをJOBLIB DDで宣言したTMAX.JOBLIBデータセットで優先的に検索する例です。

```
//JOB1 JOB
//JOBLIB DD DSN=TMAX.JOBLIB, DISP=SHR
//STEP1 EXEC PGM=TMAXSOFT
//SYSOUT DD *
```

以下は、STEP1のTMAXSOFTプログラムをSTEPLIB DDで宣言したTMAX.STEPLIBデータセットで優先的に検索し、見つからなかった場合でもJOBLIB DDで宣言したTMAX.JOBLIBデータセットでは検索しない例です。

STEP2のTMAXSOFTプログラムをJOBLIB DDで宣言したTMAX.JOBLIBデータセットで検索します。

```
//JOB1      JOB
//JOBLIB    DD DSN=TMAX.JOBLIB,DISP=SHR
//STEP1     EXEC PGM=TMAXSOFT
//STEPLIB   DD DSN=TMAX.STEPLIB,DISP=SHR
//SYSOUT    DD *
//STEP2     PGM=TMAXSOFT
//SYSOUT    DD *
```

2.4.4. JOBCAT DDとSTEP CAT DD

JOBCAT DDとSTEP CAT DDは、両方ともステップで使用するデータセットが登録されているカタログを指定します。

- JOBCAT DD

JOBCAT DDで指定したものは、ジョブ内のすべてのステップで使用するデータセットが登録されているカタログです。JOBCAT DDは、JOB文の後から、最初のEXEC文が現れる前に記述します。

- STEP CAT DD

STEP CAT DDで指定したものは、ステップで使用するデータセットが登録されているカタログです。

STEP CAT DDは、EXEC文の後から、次のEXEC文の間、あるいはジョブの終わりの間に記述します。

STEP CAT DDとJOBCAT DDが両方とも記述されると（STEP CAT DD/JOBCAT DD）順序どおり使用されます。

JOBCAT DDまたはSTEP CAT DDで指定したカタログから該当するデータセットが見つからなかった場合は、マスター・カタログでデータセットを検索します。

以下は、TMAX.SYSINデータセットをJOBCAT DDで宣言したTMAX.JOBCATカタログで優先して検索する例です。

```
//JOB1      JOB
//JOBCAT    DD DSN=TMAX.JOBCAT,DISP=SHR
//STEP1     EXEC PGM=TMAXSOFT
//SYSIN     DD DSN=TMAX.SYSIN,DISP=SHR
//SYSOUT    DD *
```

以下は、STEP1のTMAX.SYSINデータセットをSTEP1CAT DDで宣言したTMAX.STEP1CATカタログで優先的に検索し、見つからなかった場合でもJOB1CAT DDで宣言したTMAX.JOB1CATカタログでは検索しない例です。STEP2のTMAX.SYSINデータセットをJOB2CAT DDで宣言したTMAX.JOB2CATカタログで検索します。

```
//JOB1      JOB
//JOB1CAT   DD DSN=TMAX.JOB1CAT,DISP=SHR
//STEP1     EXEC PGM=TMAXSOFT
//STEP1CAT  DD DSN=TMAX.STEP1CAT,DISP=SHR
//SYSIN     DD DSN=TMAX.SYSIN,DISP=SHR
//SYSOUT    DD *
//STEP2     PGM=TMAXSOFT
//SYSIN     DD DSN=TMAX.SYSIN,DISP=SHR
//SYSOUT    DD *
```

2.5. ENDCNTL文

ENDCNTL文は、CNTL文の後に出力されるプログラム制御文の最後を示します。OpenFrameでは、CNTL文とプログラム制御文、そしてENDCNTL文に対して構文エラーのみをチェックし、使用されません。

- 構文

```
[ラベル]△1ENDCNTL△[コメント]
```

- 例

以下は、CNTL文とENDCNTL文を使用する例です。

```
//STEP1     EXEC      PGM=PRINT
//ALPHA     CNTL      * PROGRAM CONTROL STATEMENT FOLLOWS
//PRGCNTL   PRINTDEV  BUFNO=20,PIMSG=YES,DATAACK=BLOCK
//OMEGA     ENDCNTL
//AGAR      DD        UNIT=AFP1
```

2.6. EXEC文

ステップの開始を示し、そのステップの属性を記述します。

- 構文

```
//ステップ名△1EXEC△1位置オペランド[,キーワードオペランド]...△1[コメント]
```

項目	説明
ステップ名	ステップ名を指定します。「//」に続いて3桁目から始める必要があります。

項目	説明
	ステップ名を省略すると、システムは任意のステップ名を設定します。任意のステップ名の作成は、\$S00000\$から1ずつ増やしていきます。 ステップ名はジョブ内で一意の名前にする必要があります。一意でない場合、意図していないステップを参照するか、参照できないことがあります。 ステップ名は8文字以内の英数字で、最初の文字は必ず英文字にします。
EXEC	オペレーションを記述する位置であり、ステップ名の後に1つ以上の空白を入れて「EXEC」と記述します。
位置オペランド[,キーワードオペランド]	「EXEC」の後に1つ以上の空白を入れて、オペランドを記述します。先に位置オペランドを記述し、その後にキーワード・オペランドを記述します。オペランドはコンマ(,)で区切ります。 詳細については、オペランドと各オペランド節の説明を参照してください。
[コメント]	オペランドの後に1つ以上の空白を入れて記述します。コメントは71桁目まで記述できます。すべてのオペランドを省略した場合、コメントを記述できません。

オペランド

以下は、オペランドについての説明です。各オペランドの詳細については、各節を参照してください。

● 位置オペランド

STEP文の位置オペランドは、PGM、PROC、プロシージャ名のいずれかを記述します。PGMとPROCオペランドはキーワード・オペランドの形式ですが、位置オペランドのようにすべてのキーワード・オペランドの前に記述する必要があります。

項目	説明
PGM	ステップで実行するプログラム名を指定します。
PROC	ステップで実行するプロシージャ名を指定します。

● キーワード・オペランド

項目	説明
ACCT	OpenFrameでは構文解析のみサポートします。
ADDRSPC	OpenFrameでは構文解析のみサポートします。

項目	説明
CCSID	OpenFrameでは構文解析のみサポートします。
COND	ステップの実行有無を決める条件を指定します。
DYNAMNBR	OpenFrameでは構文解析のみサポートします。
MEMLIMIT	OpenFrameでは構文解析のみサポートします。
PARM	ステップで実行するプログラムのパラメータを指定します。
PERFORM	ステップが実行中に属するパフォーマンス・グループを指定します。
RD	OpenFrameでは構文解析のみサポートします。
REGION	OpenFrameでは構文解析のみサポートします。
RLSTMOUT	OpenFrameでは構文解析のみサポートします。
SPARM	システム・パラメータの値を指定します。
TIME	ステップで利用できるCPU時間の最大値を指定します。
記号パラメータ	ステップで実行するプロシージャの記号パラメータの値を指定します。

2.6.1. ACCT

OpenFrameでは構文解析のみサポートします。

- 構文

```
ACCT[.プロシージャステップ名] = 値
```

2.6.2. ADDRSPC

OpenFrameでは構文解析のみサポートします。

- 構文

```
ADDRSPC[.プロシージャステップ名] = {VIRT}
                                   {REAL}
```

2.6.3. CCSID

構文エラーのみをチェックし、使用されません。

- 構文

```
CCSID = 値
```

項目	説明
値	1～65535の符号なし整数を指定します。

2.6.4. COND

ステップを実行するかどうかを決める条件を指定します。CONDオペランドが指定されていると、ステップを開始する前に、前のステップのリターンコードと比較して、そのステップを実行するかどうかを決めます。

CONDオペランドに複数の条件が指定されている場合は、指定された条件のいずれかを満たしたら、COND条件が満たされたと判断します。ステップ名やプロシージャ・ステップ名を指定するときは、以前のステップのいずれかを記述する必要があります。それ以外のステップを記述すると、条件を満たさないと判断します。

● 構文

```
COND[.プロシージャステップ名]= ((コード, 演算符号[, ステップ名[.プロシージャステップ名]]) )
                                ([, (コード, 演算符号[, ステップ名[.プロシージャステップ名]])]...)

                                ([, EVEN] )
                                ([, ONLY] )

COND=EVEN
COND=ONLY
```

項目	説明
コード	ジョブ・ステップのリターンコードと比較する条件コードを符号なし整数で指定します。
演算符号	以下の演算符号から適切な値を指定します。 – EQ: コードとリターンコードが等しい – NE: コードとリターンコードが等しくない – GT: コードがリターンコードより大きい – GE: コードがリターンコードより大きいか等しい – LT: コードがリターンコードより小さい – LE: コードがリターンコードより小さいか等しい
ステップ名	ステップ名を記述すると、条件は記述されたステップのリターンコードとのみ比較します。記号名で指定します。
プロシージャステップ名	ステップ名と一緒に記述し、リターンコードを比較するステップがプロシージャ内のステップの場合に指定します。記号名で指定します。

項目	説明
EVEN、ONLY	条件の最後に記述し、異常終了しても該当のステップを実行するかどうかを決めます。異常終了を発生させるリターンコードは、OpenFrame 環境設定のrcサブジェクトのPGM_NAMEセクションの各プログラムごとに指定できます。詳細については、『OpenFrame 環境設定ガイド』を参照してください。 一般的に前のステップが異常終了すると、以降のステップをスキップしますが、EVENまたはONLYがある場合は実行することもできます。 – EVEN: 前のステップの正常または異常終了と関係なく、条件を満たす場合はスキップし、満たさない場合は現在のステップが実行されます。EVENのみ指定した場合は、常に条件を満たさないと判断します。 – ONLY: 前のステップが正常終了した場合はスキップし、異常終了した場合は条件を比較してスキップするか、実行するかを決めます。ONLYのみ指定した場合は、常に条件を満たさないと判断します。 EVENとONLYの指定については以下で説明します。

以下は、EVENとONLYの指定と関連するステップについての説明です。

CONDオペランド	以前のステップの状態	現在のステップの実行可否
未指定	すべて正常終了	実行
未指定	1つでも異常終了	スキップ
条件のみ指定	すべて正常終了	条件を1つでも満たす場合はスキップ、そうでなければ実行
条件のみ指定	1つでも異常終了	スキップ
EVENのみ指定	すべて正常終了	実行
EVENのみ指定	1つでも異常終了	実行
EVENと条件を指定	すべて正常終了	条件を1つでも満たす場合はスキップ、そうでなければ実行
EVENと条件を指定	1つでも異常終了	条件を1つでも満たす場合はスキップ、そうでなければ実行
ONLYのみ指定	すべて正常終了	スキップ
ONLYのみ指定	1つでも異常終了	実行
ONLYと条件を指定	すべて正常終了	スキップ
ONLYと条件を指定	1つでも異常終了	条件を1つでも満たす場合はスキップ、そうでなければ実行

- 注意事項

- 条件パラメータは最大8つまで有効であり、9番目以上の条件パラメータは無視されます。条件パラメータの数にEVENとONLYは含まれません。
- 条件パラメータを1つのみ指定した場合、一番外側にある括弧を省略できます。
- JOB文とEXEC文にCONDが指定されている場合、JOB文のCONDが満たされるとEXEC文のCONDは考慮されません。
- CONDオペランドの値は構文チェックの際にはチェックしません。

- 例

以下は、STEP1のリターンコードが4であり、STEP2での12が4より大きいため条件を満たしており、STEP2がスキップされる例です。

```
//JOB1      JOB   CLASS=A,MSGCLASS=A
//STEP1     EXEC  RETURN,PARM=4
//STEP2     EXEC  TMAXSOFT,COND=(12,GT)
```

以下は、STEP1のリターンコードが4であり、STEP2での2が4より小さく、12が4より大きいため条件を1つも満たさず、STEP2が実行される例です。

```
//JOB1      JOB   CLASS=A,MSGCLASS=A
//STEP1     EXEC  RETURN,PARM=4
//STEP2     EXEC  TMAXSOFT,COND=((2,GT),(12,LT))
```

以下は、STEP1のリターンコードが4で、STEP2でONLYが指定されており、STEP1が正常終了したため条件を満たしていないが、ONLYパラメータによりSTEP2がスキップされる例です。

```
//JOB1      JOB   CLASS=A,MSGCLASS=A
//STEP1     EXEC  RETURN,PARM=4
//STEP2     EXEC  TMAXSOFT,COND=((2,GT),(12,LT),ONLY)
```

以下は、STEP1のリターンコードが4で、STEP1が異常終了したが、STEP2でONLYが指定されており、かつ条件を満たさないため、STEP2が実行される例です。

```
//JOB1      JOB   CLASS=A,MSGCLASS=A
//STEP1     EXEC  ABEND,PARM=4
//STEP2     EXEC  TMAXSOFT,COND=((2,GT),(12,LT),ONLY)
```

以下は、STEP1のリターンコードが4で、STEP2でEVENが指定されているため、無条件でSTEP2が実行される例です。

```
//JOB1      JOB   CLASS=A,MSGCLASS=A
//STEP1     EXEC  ABEND,PARM=4
//STEP2     EXEC  TMAXSOFT,COND=EVEN
```


以下は、STEP1のリターンコードが4で、STEP2はリターンコードが8だが、STEP3でSTEP1とSTEP2のリターンコードを比較したらSTEP1のリターンコードが4で条件を満たすため、STEP3がスキップされる例です。

```
//JOB1      JOB   CLASS=A,MSGCLASS=A
//STEP1     EXEC  RETURN,PARM=4
//STEP2     EXEC  RETURN,PARM=8
//STEP3     EXEC  TMAXSOFT,COND=(4,EQ)
```

以下は、STEP1のリターンコードが4で、STEP2はリターンコードが8だが、STEP3でSTEP2のリターンコードが4でないため条件を満たされず、STEP3が実行される例です。

```
//JOB1      JOB   CLASS=A,MSGCLASS=A
//STEP1     EXEC  RETURN,PARM=4
//STEP2     EXEC  RETURN,PARM=8
//STEP3     EXEC  TMAXSOFT,COND=(4,EQ,STEP2)
```

以下は、STEP1のプロシージャ・ステップのPSTEP1のリターンコードは4だが、STEP2で比較したら条件を満たすため、STEP2がスキップされる例です。

```
//JOB1      JOB   CLASS=A,MSGCLASS=A
//TMAXPROC  PROC
//PSTEP1    EXEC  PGM=RETURN,PARM=4
//PSTEP2    EXEC  PGM=RETURN,PARM=8
//          PEND
//STEP1     EXEC  TMAXPROC
//STEP2     EXEC  TMAXSOFT,COND=(4,EQ,STEP1.PSTEP1)
```

以下は、STEP1のリターンコードが4で、STEP2のプロシージャ・ステップでPSTEP1は条件を満たしているためスキップ、PSTEP2はCONDオペランドを適用しないため実行される例です。

```
//JOB1      JOB   CLASS=A,MSGCLASS=A
//TMAXPROC  PROC
//PSTEP1    EXEC  PGM=TMAXSOFT
//PSTEP2    EXEC  PGM=TMAXSOFT
//          PEND
//STEP1     EXEC  RETURN,PARM=4
//STEP2     EXEC  TMAXPROC,COND.PSTEP1=(4,EQ)
```

2.6.5. DYNAMNBR

OpenFrameでは構文解析のみサポートします。

- 構文

```
DYNAMNBR[.プロシージャステップ名] = 値
```

項目	説明
値	1～3273の符号なし整数を指定します。

2.6.6. MEMLIMIT

OpenFrameでは構文解析のみサポートします。

- 構文

```
MEMLIMIT = {値M|値G|値T|値P}  
           {NOLIMIT}
```

項目	説明
値M	0～99999の符号なし整数を指定します。
値G	0～99999の符号なし整数を指定します。
値T	0～99999の符号なし整数を指定します。
値P	0～16384の符号なし整数を指定します。

2.6.7. PARM

ステップで実行するプログラムに渡すパラメータ値を指定します。

- 構文

```
PARM[.プロシージャステップ名] = パラメータ値  
PARM[.プロシージャステップ名] = (パラメータ値,パラメータ値)  
PARM[.プロシージャステップ名] = ('パラメータ値',パラメータ値)  
PARM[.プロシージャステップ名] = 'パラメータ値,パラメータ値'
```

項目	説明
パラメータ値	ステップで実行するプログラムに渡すパラメータ値を指定します。

- 例

以下は、STEP1で実行するPRINTプログラムのパラメータとしてargv1を渡す例です。

```
//JOB1      JOB  CLASS=A,MSGCLASS=A  
//STEP1     EXEC PRINT,PARM='argv1'
```

以下は、STEP1のプロシージャ・ステップのPSTEP1にはパラメータとしてargv1を、PSTEP2にはargv2を渡す例です。

```
//JOB1      JOB   CLASS=A,MSGCLASS=A
//TMAXPROC  PROC
//PSTEP1    EXEC  PGM=PRINT,PARM='argv2'
//PSTEP2    EXEC  PGM=PRINT,PARM='argv2'
//          PEND
//STEP1     EXEC  TMAXPROC,PARM.PSTEP1='argv1'
```

2.6.8. PGM

ステップで実行するプログラム名を指定します。このオペランドは、ステップで実行するプログラムを指定します。指定できるプログラムは、UNIXで実行可能なすべての実行バイナリです。

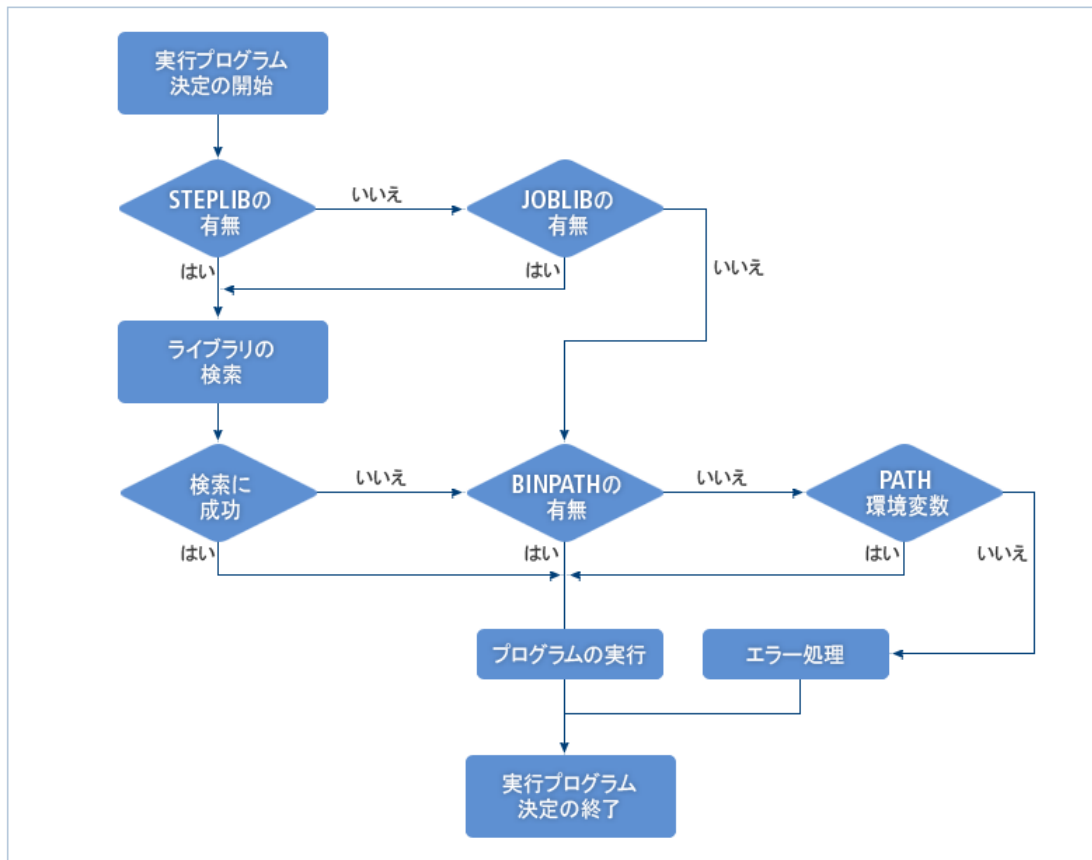
プログラムが共有オブジェクトとしてコンパイルされた場合は直接実行されないため、PGMRTS00ユーティリティを使用して実行できます。PGMRTS00ユーティリティを使用するかどうかは、OpenFrame環境設定の**tjclrun**サブジェクトの**PGM**セクションの**USE_PGMRTS00**キーの**VALUE**項目の値に指定できます。

参考

1. PGMRTS00の詳細については、『OpenFrame ユーティリティリファレンスガイド』を参照してください。
 2. tjclrunサブジェクトの詳細については、『OpenFrame 環境設定ガイド』を参照してください。
-

プログラム名を記述すると、該当するプログラムを以下のプロセスで検索します。

[図 2.1] 実行プログラムの指定



1. STEPLIB DDがステップに指定されている場合、指定されたデータセットのメンバーの中から検索します。
2. STEPLIB DDがステップに指定されておらず、JOB LIB DDがジョブに指定されている場合、指定されたデータセットのメンバーの中から検索します。
3. OpenFrame環境設定の**tjclrun**サブジェクトの**SYSLIB**セクションの**BIN_PATH**キーのVALUEが設定されている場合は、BIN_PATHに指定されているディレクトリから検索します。
4. OpenFrame環境設定の**tjclrun**サブジェクトの**SYSLIB**セクションの**BIN_PATH**キーのVALUEが設定されていない場合は、obmjinitサーバーが起動されるときに環境変数のPATHに指定されているディレクトリから検索します。

以下は、PGMオペランドの説明です。

- 構文

PGM = {プログラム名}

項目	説明
プログラム名	ステップで実行するプログラム名を特殊文字列で指定します。

- 注意事項

EXEC文のオペランドの中で一番最初に記述します。

- 例

以下は、実行プログラムの名前を指定する例です。

```
//JOB1      JOB   CLASS=A,MSGCLASS=A
//STEP1     EXEC  PGM=TMAXSOFT
```

以下は、PROD.BATCHLIBに存在するTMAXSOFTを実行プログラムの名前として指定する例です。

```
//JOB1      JOB   CLASS=A,MSGCLASS=A
//STEP1     EXEC  PGM=TMAXSOFT
//STEPLIB   DD    DSN=PROD.BATCHLIB,DISP=SHR
```

2.6.9. PROC

ステップで実行するプロシージャ名を指定します。このオペランドは、該当するステップで実行するプロシージャを指定します。プロシージャには、カタログ・プロシージャと入カストリーム・プロシージャがあります。

- カatalog・プロシージャは、データセットのメンバーとして指定されているプロシージャを意味し、どのジョブでも実行できます。
- 入カストリーム・プロシージャは、入カストリームの中で指定されているプロシージャであり、入カストリーム内のジョブでのみ一時的に実行できます。

プロシージャの検索順は以下のとおりです。

1. 入カストリーム・プロシージャから検索します。
2. JCLLIB文が記述されている場合は、JCLLIB文に記述されているメンバーから検索します。
3. JES2 JCL文のJOBPARM文にPROCLIB={ddname}オペランドが指定されている場合は、OpenFrame 環境設定のtjesサブジェクトのPROCLIBセクションの{ddname}キーのVALUE項目に指定されているデータセットのメンバーから検索します。
4. OpenFrame環境設定のtjesサブジェクトのPROCLIBセクションのPROC00キーのVALUE項目に指定されているデータセットのメンバーから検索します。
5. SYS1.PROCLIBのメンバーから検索します。

参考

tjesサブジェクトの詳細については、『OpenFrame 環境設定ガイド』を参照してください。

以下は、PROCオペランドの構文です。

- 構文

```
{PROC = プロシージャ名}  
{プロシージャ名}
```

項目	説明
プロシージャ名	ステップで実行するプロシージャ名を記号名で指定します。

- 注意事項

EXEC文のオペランドの中で一番最初に記述します。

- 例

以下は、TMAXPROCプロシージャを実行する例です。

```
//JOB1      JOB   CLASS=A,MSGCLASS=A  
//STEP1     EXEC  TMAXPROC
```

以下は、入力ストリームのTMAXPROCプロシージャを実行する例です。

```
//JOB1      JOB   CLASS=A,MSGCLASS=A  
//TMAXPROC  PROC  
//PSTEP1    EXEC  PGM=TMAXSOFT  
//          PEND  
//STEP1     EXEC  TMAXPROC
```

プロシージャ・ステップ名

オペランドの後に記述できるプロシージャ・ステップ名について説明します。プロシージャを実行するステップで指定したキーワード・オペランドは、プロシージャ内のすべてのステップで指定したキーワード・オペランドをオーバーライドします。プロシージャ内の特定ステップのオペランドのみをオーバーライドしたい場合は、オペランドにプロシージャ・ステップ名を一緒に記述し、当該ステップでのみオペランドをオーバーライドすることができます。

- 構文

```
オペランド[.プロシージャステップ名] = 値
```

項目	説明
プロシージャステップ名	オペランドが適用されるプロシージャ・ステップを記号名で指定します。

- 注意事項

プロシージャ・ステップ名は、プロシージャを実行するステップでのみ有効です。プログラムを実行するステップの場合、プロシージャ・ステップ名は無視されます。

- 例

以下は、プロシージャ内のすべてのステップのCOND値を(8,EQ)にオーバーライドする例です。

```
//JOB1      JOB  CLASS=A,MSGCLASS=A
//TMAXPROC  PROC
//PSTEP1    EXEC  PGM=TMAXSOFT,COND=(0,EQ)
//PSTEP2    EXEC  PGM=TMAXSOFT,COND=(4,EQ)
//          PEND
//STEP1     EXEC  TMAXPROC,COND=(8,EQ)
```

以下は、プロシージャ・ステップのPSTEP1のCOND値を(0,NE)にオーバーライドする例です。

```
//JOB1      JOB  CLASS=A,MSGCLASS=A
//TMAXPROC  PROC
//PSTEP1    EXEC  PGM=TMAXSOFT,COND=(0,EQ)
//PSTEP2    EXEC  PGM=TMAXSOFT,COND=(4,EQ)
//          PEND
//STEP1     EXEC  TMAXPROC,COND.PSTEP1=(0,NE)
```

以下は、プロシージャ・ステップのPSTEP1のCOND値を(0,NE)に、PSTEP2のCOND値を(4,NE)にオーバーライドする例です。

```
//JOB1      JOB  CLASS=A,MSGCLASS=A
//TMAXPROC  PROC
//PSTEP1    EXEC  PGM=TMAXSOFT,COND=(0,EQ)
//PSTEP2    EXEC  PGM=TMAXSOFT,COND=(4,EQ)
//          PEND
//STEP1     EXEC  TMAXPROC,COND.PSTEP1=(0,NE),COND.PSTEP2=(4,NE)
```

2.6.10. PERFORM

ステップが実行中に属するパフォーマンス・グループを指定します。OpenFrameでは、環境設定に登録されたパフォーマンス・グループ番号に応じたNICE値に変更します。

OpenFrameでは、パフォーマンス・グループ番号をUNIXのNICE値に置き換えて使用します。オペランドが指定されると、ランナーはパフォーマンス・グループ番号をNICE値に置き換え、ステップ・プログラムのNICE値を変更します。

STEP文にPERFORMが指定されている場合、JOB文に指定されたPERFORMは無視されます。

● 構文

```
PERFORM = パフォーマンスグループ番号
```

項目	説明
パフォーマンスグループ番号	1~999の符号なし整数でパフォーマンス・グループ番号を指定します。

- 注意事項

- PERFORMオペランドをNICEの値に置き換えて使用する場合は、OpenFrame環境設定の**tjclrun**サブジェクトの**PERFORM**セクションの**USE_PERFORM**キーの**VALUE**項目を**YES**に指定する必要があります。詳細については、『OpenFrame 環境設定ガイド』を参照してください。
- NICEの値を変更するには、スーパーユーザー権限が必要となるため、ランナーに**setuid**を設定する必要があります。setuidの設定については、『OpenFrame TJESガイド』の「第3章 ジョブの実行」の「setuid root tjclrun」を参照してください。

- 例

以下は、STEP1でPERFORMを10に指定する例です。

```
//JOB1    JOB   PERFORM=100
//STEP1   EXEC  PGM=TMAXSOFT,PERFORM=10
```

2.6.11. RD

OpenFrameでは構文解析のみサポートします。

- 構文

```
RD[.プロセスジャステップ名] = {R | RNC | NR | NC}
```

2.6.12. REGION

OpenFrameでは構文解析のみサポートします。

- 構文

```
REGION[.プロセスジャステップ名] = {値K | 値M}
```

項目	説明
値K	0~2096128の符号なし整数を指定します。
値M	0~2047の符号なし整数を指定します。

2.6.13. RLSTMOUT

OpenFrameでは構文解析のみサポートします。

- 構文

```
RLSTMOUT[.プロセスジャステップ名] = 値
```


項目	説明
1440	値に制限がないことを示します。CPU時間に制限がありません。
NOLIMIT	1440と同様に値に制限がないことを示します。CPU時間に制限がありません。
0	1440と同様に値に制限がないことを示します。CPU時間に制限がありません。
MAXIMUM	357912分を示します。

- 注意事項

- 分のみを指定する場合は括弧を省略できます。
- 指定していない場合、デフォルト値は0です。
- tjclrunでCPU時間のチェックを5秒ごとに行うため、最大5秒の誤差が生じる可能性があります。

- 例

以下は、STEP1のTIMEを1分10秒に指定する例です。

```
//JOB1    JOB
//STEP1   EXEC PGM=TMAXSOFT,TIME=(1,10)
```

以下は、STEP1のTIMEを無制限に指定する例です。

```
//JOB1    JOB
//STEP1   EXEC PGM=TMAXSOFT,TIME=NOLIMIT
```

2.6.16. 記号パラメータ

ステップで実行するプロシージャの記号パラメータの値を指定します。

記号パラメータは、ステップで実行するプロシージャの記号パラメータとして使用されます。記号パラメータは、プロシージャのPROC文で指定した同じ記号パラメータの値をオーバーライドします。プロシージャ内で記号パラメータは「&記号パラメータ」形式で使用し、プロシージャの実行時に指定された値に置き換えられます。

- 構文

```
記号パラメータ = [値]
```

項目	説明
値	引用符付き文字列で記号パラメータの値を指定します。

- 注意事項

「&記号パラメータ」形式で使用しますが、当該記号パラメータが定義されていない場合は、「&記号パラメータ」は置き換えられず、そのまま残ります。

- 例

以下は、記号パラメータをA=step1と指定し、PSTEP1のPARMの値はstep1、PSTEP2の値はdefaultとなる例です。

```
//JOB1      JOB  CLASS=A,MSGCLASS=A
//TMAXPROC  PROC  A=default,B=default
//PSTEP1    EXEC  PGM=TMAXSOFT,PARM=&A
//PSTEP2    EXEC  PGM=TMAXSOFT,PARM=&B
// PEND
//STEP1     EXEC  TMAXPROC,A=step1
```

以下は、記号パラメータをA=、B=step1と指定し、PSTEP1のPARMの値がNULL、PSTEP2の値はstep1となる例です。

```
//JOB1      JOB  CLASS=A,MSGCLASS=A
//TMAXPROC  PROC  A=default,B=default
//PSTEP1    EXEC  PGM=TMAXSOFT,PARM=&A
//PSTEP2    EXEC  PGM=TMAXSOFT,PARM=&B
// PEND
//STEP1     EXEC  TMAXPROC,A=,B=step1
```

2.7. IF/THEN/ELSE/ENDIF文

分岐してJCL文を実行できるように条件を指定し、条件の結果に従って実行する構文を記述します。

- 構文

```
//[名前]△1IF△1条件△1THEN△1[コメント]
.
. 条件が満たされる場合、実行
.
//[名前]△1ELSE△1[コメント]
.
. 条件が満たされない場合、実行
.
//[名前]△1ENDIF△1[コメント]
```

項目	説明
名前	名前を指定します。「//」に続いて3桁目から始める必要があります。名前は省略できます。
IF	オペレーションを記述する位置であり、名前の後ろに1つ以上の空白を入れて「IF」と記述します。

項目	説明
条件	IFの後ろに1つ以上の空白を入れて、条件を記述します。条件についての詳細は 条件 の説明を参照してください。
THEN	条件の後ろに1つ以上の空白を入れて「THEN」と記述します。
[コメント]	THENの後ろに1つ以上の空白を入れて記述します。コメントは71桁目から記述できます。
ELSE文	条件が満たされた場合に実行する構文をすべて記述してから、条件が満たされなかった場合に実行する構文を記述します。 ELSE文は省略できます。省略した場合、条件が満たされなかった場合に実行する構文はなくなります。
ENDIF文	IF文の終了を示します。SET文とINCLUDE文は、JCLの構文分析中に実行されるので、IF文とENDIF文の間に記述しても、IF文と関係なく実行されます。

条件

以下は、前述の条件についての説明です。

条件は基本的に以下のように構成されます。以下例において、RCはキーワード、「<」は比較演算子です。

```
// IF RC < 4 THEN
```

上記の場合、以前のステップの終了コードが4より小さければ条件が満たされるものと処理し、4以上であれば条件が満たされないものと処理します。

条件を括弧で囲んでも構いません。

```
// IF (RC < 4) THEN
```

論理演算子を使用して条件をより詳しく記述できます。以下例において「|」と「OR」は論理演算子です。

```
// IF (RC = 4 | RC EQ 8 OR RC = 12) THEN
```

● 演算子

演算子には比較演算子と論理演算子があります。以下の表において、AND、OR、NOTが論理演算子で、それ以外は比較演算子です。

演算子	機能
GT or >	～より大きい
LT or <	～より小さい

演算子	機能
NG or ^> or ~>	～より大きくない
NL or ^< or ~<	～より小さくない
EQ or =	～と等しい
NE or ^= or ~=	～と等しくない
GE or >=	～より大きい か等しい
LE or <=	～より小さい か等しい
AND or &	そして
OR or	または
NOT or ^ or ~	～ではない

以下は、演算子を使用した例です。

```
// IF RC = 4 THEN
// IF RC EQ 4 THEN
```

上の2つの構文は同じ意味です。解析すると以下のとおりです。

```
"RCは4と等しい"
```

```
// IF (RC >= 4 & RC < 8) THEN
// IF RC GE 4 AND RC LT 8 THEN
```

上の2つの構文は同じ意味です。解析すると以下のとおりです。

```
"RCは4より大きい  
か等しい。そしてRCは8より小さい"
```

```
// IF ~(RC > 4 & RC < 8) THEN
// IF (RC <= 4 | RC >= 8) THEN
```

上の2つの構文は、解析は異なりますが同じ結果を示します。

```
RCは4より大きくなく、かつ8より小さくない"
"RCは4より小さいか等しい。またはRCは8より大きい  
か等しい"
```

● キーワード

キーワードには以下の3つがあります。

– RC

ステップの終了コードと比較します。

たとえば、(RC = 8)の場合、ステップの終了コードが8であれば条件が満たされるものと処理します。前に実行されたステップのすべての終了コードと比較して、1つでも条件を満たす場合は条件が満たされたものと処理します。ステップが1つも実行されていない場合、条件は満たされないものと処理されます。

キーワード	説明
ステップ名.RC	指定したステップの終了コードと比較します。指定したステップが実行されていない場合、条件は満たされないものと処理されます。
ステップ名.プロシージャ ステップ名.RC	指定したステップのプロシージャ・ステップの終了コードと比較します。指定したステップのプロシージャ・ステップが実行されていない場合、条件は満たされないものと処理されます。

– ABEND

ステップの異常終了の有無を比較します。

ABEND自体が条件であるため、演算子は別途不要です。

```
// IF ABEND THEN
```

前に実行されたすべてのステップで異常終了の有無を比較し、1つでも異常終了した場合は条件を満たすものと処理します。

キーワード	説明
ステップ名.ABEND	指定したステップが異常終了した場合は条件を満たすものと処理します。指定したステップが実行されていない場合、条件は満たされないものと処理されます。
ステップ名.プロシージャ ステップ名.ABEND	指定したステップのプロシージャ・ステップが異常終了した場合は条件を満たすものと処理します。指定したステップのプロシージャ・ステップが実行されていない場合、条件は満たされないものと処理されます。

NOTの記号「~」あるいは「^」を前につけると、すべての条件が反対に処理されます。

– RUN

ステップの実行有無を比較します。RUNの場合、すべてのステップを比較することはできず、前にあるステップ名やステップ名.プロシージャ・ステップ名を必ずつけます。

キーワード	説明
ステップ名.RUN	指定したステップが実行された場合は条件を満たすものと処理します。
ステップ名.プロシージャ ステップ名.RUN	指定したステップのプロシージャ・ステップが実行された場合は条件を満たすものと処理します。

RUN自体が条件であるため、演算子は別途不要です。

```
// IF STEP1.RUN THEN
```

NOTの記号「~」あるいは「^」を前につけると、すべての条件が反対に処理されます。

2.8. INCLUDE文

指定したメンバーの内容をJCL文に含めることを意味します。

- 構文

```
//[名前]△1INCLUDE△1MEMBER=名前△[コメント]
```

項目	説明
名前	名前を指定します。「//」に続いて3桁目から始める必要があります。名前は省略できます。
INCLUDE	オペレーションを記述する位置であり、名前の後ろに1つ以上の空白を入れて「INCLUDE」と記述します。名前を省略した場合、「//」の後ろに1つ以上の空白を入れて記述します。
MEMBER=名前	INCLUDEの後ろに1つ以上の空白を入れて記述します。JCL文に含めるメンバーを記述します。 メンバーを検索する基準は以下の順序に従います。 – INCLUDE文の前にJCLLIB文が記述されている場合、JCLLIB文で記述されているPDSの順番通りにメンバーを検索します。 – INCLUDE文の前に*PROCLIB文(JES2文)が記述されている場合、*PROCLIB文のPROCLIBパラメータに記述したDD名を使用してメンバーを記述します。 *PROCLIB文のPROCLIB DD名は、OpenFrame環境設定のtjesサブジェクトのPROCLIBセクションでキーワードとして使用され、これを通じて確認されたPDSの順番通りにメンバーを検索します。 – PDSのSYS1.PROCLIBでメンバーを検索します。
[コメント]	MEMBER=名前の後ろに1つ以上の空白を入れて記述します。 コメントは71桁目まで記述できます。

- 例

以下は、INCLUDE文を使用する例で、入力JCLとメンバーAAAの内容です。

```
//TESTJOB    JOB      CLASS=A
//              INCLUDE MEMBER=AAA
//STEP1      EXEC     PGM=TESTPGM
```

```
//SYSOUT      DD      SYSOUT=*
//
```

```
//JOB LIB      DD      DSN=PROD.PROCLIB1,DISP=SHR
//            DD      DSN=PROD.PROCLIB2,DISP=SHR
```

上記の場合、入力JCLは以下のように処理されます。

```
//TESTJOB      JOB      CLASS=A
//JOB LIB      DD      DSN=PROD.PROCLIB1,DISP=SHR
//            DD      DSN=PROD.PROCLIB2,DISP=SHR
//STEP1        EXEC     PGM=TESTPGM
//SYSOUT        DD      SYSOUT=*
//
```

以下は、INCLUDE文でメンバーを検索する例です。

```
//            JCLLIB  ORDER=(INCLUDE.MEMBER1,INCLUDE.MEMBER2)
//            INCLUDE MEMBER=AAA
```

上記のように記述されている場合、メンバーAAAは、INCLUDE.MEMBER1、INCLUDE.MEMBER2、SYS1.PROCLIBの順番で検索されます。

```
//*JOBPARM     PROCLIB=PROC00
//            INCLUDE MEMBER=AAA
```

```
$ ofconfig list -n NODE1 -s tjcs -sec PROCLIB -k PROC00
```

```
=====
SUBJECT  | SECTION      | KEY          | VALUE
=====
tjes     | PROCLIB      | PROC00      | INCLUDE.MEMBER3:INCLUDE.MEMBER4
=====
```

上記のように記述されている場合、メンバーAAAは、INCLUDE.MEMBER3、INCLUDE.MEMBER4、SYS1.PROCLIBの順番で検索されます。

2.9. JCLコマンド文

JCLコマンドを記述します。JCLコマンド文は、JOB文と最初のEXEC文の間に記述します。それ以外の位置に記述されたJCLコマンド文は使用されません。

JCLコマンド文によって実行されたコマンドの成功可否は、ジョブのサブミットまたは実行には影響を与えません。

- 構文


```
// コマンド△1オペランド
```

項目	説明
コマンド	オペレーションを指定します。「//」の後ろに1つ以上の空白を入れてコマンドを記述します。サポートされるコマンドについては、 コマンド を参照してください。
オペランド	オペレーションの後ろに1つ以上の空白を入れてオペランドを記述します。オペランドはコマンドによって異なります。

コマンド

以下は、前述したコマンド項目についての説明です。

コマンド	説明
S	入力したプロシージャをサブミットします。STARTと記述してもかまいません。

参考

上記で説明されていないコマンドはサポートされません。

2.9.1. S

入力したプロシージャをサブミットします。

- 構文

```
// S(START) プロシージャ名, 記号パラメータ=値[, 記号パラメータ=値]...
```

項目	説明
プロシージャ名	サブミットするプロシージャ名を指定します。
記号パラメータ	PROC文の記号パラメータ を参照してください。

2.10. JCLLIB文

JCLで使用するプロシージャや、INCLUDE文で指定したJCL文を取得するライブラリを指定します。

- 構文

```
//[名前]△1JCLLIB△1ORDER=(ライブラリ[,ライブラリ]...)△[コメント]
```

項目	説明
名前	名前を指定します。「//」に続いて3桁目から始める必要があります。名前は省略できます。
JCLLIB	オペレーションを記述する位置であり、名前の後ろに1つ以上の空白を入れて「JCLLIB」と記述します。名前を省略した場合、「//」の後ろに1つ以上の空白を入れて記述します。
ORDER=(ライブラリ, ライブラリ,...)	JCLLIBの後ろに1つ以上の空白を入れて記述します。プロシージャかINCLUDE文で指定したJCL文を取得するためのライブラリを指定します。ライブラリはPDSで記述します。 以下は、基準となる検索順です。 – JCLLIB文で記述されたライブラリ順通りにメンバーを検索します。 – *PROCLIB文(JES2文)が記述されている場合、*PROCLIB文のPROCLIBパラメータに記述したDD名を使用してメンバーを記述します。*PROCLIB文のPROCLIBパラメータに記述したDD名は、OpenFrame環境設定の tjes サブジェクト、 PROCLIB セクションのキーとして使用され、これを通じて確認されたPDSデータセットの順序通りにメンバーを検索します。 – PDSのSYS1.PROCLIBでメンバーを検索します。
[コメント]	オペランドの後ろに1つ以上の空白を入れて記述します。コメントは71桁目まで記述できます。

- 例

以下は、JCLLIB文を使用する例です。

```
//TESTJOB    JOB      CLASS=A
//LIBRARY    JCLLIB   ORDER=PROD.PROCLIB1
//STEP1      EXEC     PROC01
//SYSOUT     DD       SYSOUT=*
```

上記の場合、PROC01というプロシージャはPROC.PROCLIB1で検索します。

2.11. JOB文

ジョブの開始を示し、そのジョブの属性を記述します。

- 構文

```
//JOB名△1 JOB△1位置オペランド[, キーワードオペランド]...△1[コメント]
//JOB名△1 JOB
```

項目	説明
JOB名	ジョブ名を指定します。「//」に続いて3桁目から記号名で記述します。ジョブ名は省略できません。
JOB	オペレーションを指定します。JOB名の後ろに1つ以上の空白を入れて「JOB」と指定します。
位置オペランド[,キーワードオペランド]	JOBの後ろに1つ以上の空白を入れてオペランドを指定します。位置オペランドを指定してから、キーワード・オペランドを指定します。詳細については、 オペランド と各オペランド節を参照してください。
[コメント]	オペランドの後ろに1つ以上の空白を入れて指定します。コメントは71桁目まで指定できます。オペランドを省略すると、コメントを指定できません。

オペランド

以下は、オペランドについての説明です。各オペランドの詳細については、各節を参照してください。

• 位置オペランド

OpenFrameでは、TIMECONTROLLERを除く他の構文は、構文エラーのみをチェックし、使用されません。ただし、2番目のパラメータ(プログラム名)を指定した場合は、環境変数 (PROGRAMMERNAME)に含めてアセンブラーなどのプログラムに渡します。

項目	説明
TIMECONTROLLER	タイム・コントローラーを使用するための値を指定します。

• キーワード・オペランド

項目	説明
ADDRSPC	OpenFrameでは構文解析のみサポートします。
BYTES	OpenFrameでは構文解析のみサポートします。
CARDS	OpenFrameでは構文解析のみサポートします。
CCSID	OpenFrameでは構文解析のみサポートします。
CLASS	ジョブ・クラスを指定します。
COND	ジョブ・ステップを終了した後リターンコードと比較して、以降のステップの実行可否を決定する条件を指定します。
GROUP	OpenFrameでは構文解析のみサポートします。
JESLOG	OpenFrameでは構文解析のみサポートします。

項目	説明
LINES	OpenFrameでは構文解析のみサポートします。
MEMLIMIT	OpenFrameでは構文解析のみサポートします。
MSGCLASS	システム・メッセージが出力されるデータセットの出力クラスを指定します。OpenFrameでは、DD文の「SYSOUT=*」を指定する場合に適用される用途としてのみ使用されます。
MSGLEVEL	ジョブのシステム・メッセージの出力レベルを指定します。
NOTIFY	OpenFrameでは構文解析のみサポートします。
PAGES	OpenFrameでは構文解析のみサポートします。
PASSWORD	パスワードを指定します。
PERFORM	ジョブが実行中に属するパフォーマンス・グループを指定します。
PRTY	ジョブの優先順位を指定します。
RD	OpenFrameでは構文解析のみサポートします。
REGION	OpenFrameでは構文解析のみサポートします。
RESTART	ジョブの再開ステップを指定します。
SECLABEL	OpenFrameでは構文解析のみサポートします。
SCHENV	OpenFrameでは構文解析のみサポートします。
SPARM	システム・パラメータの値を指定します。
TIME	ジョブで利用できるCPU時間の最大値を指定します。
TYPRUN	ジョブの実行タイプを指定します。
USER	ユーザを指定します。

2.11.1. ADDRSPC

OpenFrameでは構文解析のみサポートします。

- 構文

```
ADDRSPC = {VIRT}
          {REAL}
```

2.11.2. BYTES

OpenFrameでは構文解析のみサポートします。

- 構文

```
BYTES = {値          }  
        {( 値, CANCEL) }  
        {( 値, DUMP)   }  
        {( 値, WARNING)}
```

項目	説明
値	0～999999の符号なし整数です。

2.11.3. CARDS

OpenFrameでは構文解析のみサポートします。

- 構文

```
CARDS = {値          }  
        {( 値, CANCEL) }  
        {( 値, DUMP)   }  
        {( 値, WARNING)}
```

項目	説明
値	0～99999999の符号なし整数です。

2.11.4. CCSID

OpenFrameでは構文解析のみサポートします。

- 構文

```
CCSID = 値
```

項目	説明
値	1～65535の符号なし整数です。

2.11.5. CLASS

ジョブ・クラスを指定します。JOB文に指定されたクラスは、該当するジョブのスケジューリングに関与します。TJESでは、各ノードにランナー・スロット(Runner Slot)を設定します。また、ランナー・スロットにはそれぞれ自身が実行できるクラスが指定されています。TJESスケジューラは、サブミットされたジョブのクラスに合うランナー・スロットが空いていると、該当するジョブを実

行するようにします。TJESでは、クラス別に設定して、サブミットされたジョブをSTARTやHOLD状態に設定することができます。

たとえば、AクラスをSTART、BクラスをHOLDに設定すると、Aクラスに指定したジョブはサブミット後START状態になり、Bクラスに指定したジョブはサブミット後HOLD状態になります。未設定のクラスでサブミットしたジョブはSTART状態になります。

JOB文のTYPRUNオペランドを使用してJCLHOLDまたはHOLDを指定した場合、このクラス別の設定とは関係なくHOLD状態になります。

参考

1. スケジューリングの詳細については、『OpenFrame TJESガイド』を参照してください。
 2. ランナー・スロットは、OpenFrame環境設定の**tjes**サブジェクト、**INITDEF**セクションのキーに設定します。詳細については、『OpenFrame 環境設定ガイド』を参照してください。
 3. 各クラスのデフォルト属性は、OpenFrame環境設定の**tjes**サブジェクト、**JOBCLASS**セクションに設定します。詳細については、『OpenFrame 環境設定ガイド』を参照してください。
-

以下は、CLASSオペランドについての説明です。

● 構文

```
CLASS = job class
```

項目	説明
job class	ジョブのスケジューリングに使用されるクラスを、A~Z、0~9の間の1文字で指定します。

● 注意事項

JOB文でこのオペランドを省略した場合、ジョブのクラスは、OpenFrame環境設定の**tjclrun**サブジェクト、**JOB**セクションの**CLASS**キーのVALUE項目に指定された値を使用します。tjesサブジェクトの詳細については、『OpenFrame 環境設定ガイド』を参照してください。

● 例

以下は、ジョブ・クラスをAに指定する例です。

```
//JOB1 JOB CLASS=A
```

2.11.6. COND

ジョブ・ステップを終了した後以降のステップの実行可否を決定するために、リターンコードと比較する条件を指定します。

CONDオペランドが指定されると、各ステップが終了するたびに、ステップのリターンコードを指定のコードと比較します。リターンコードがJOB文のCOND条件を満たす場合は、EXEC文のCONDに基づく各ステップの実行可否を考慮せずに、ジョブを終了します。CONDオペランドに指定された条件が複数ある場合は、指定された条件を1つでも満たせば、COND条件は満たされたものと判断します。

OpenFrameでは、OpenFrame環境設定のrcサブジェクト、PGM_NAME、PGM_TYPEセクションを介してステップのリターンコードをチェックし、エラーを処理するCONDオペランドと同様な機能があります。OpenFrame環境設定のrcサブジェクトは、UNIXへのリホスト時に実行されるプログラムのリターンコードがメインフレームとは異なる場合があるため、ステップのエラーを柔軟に判断するために使用されます。

OpenFrame環境設定のrcサブジェクトの設定がCONDオペランドの設定よりも優先されるため、OpenFrame環境設定のrcサブジェクトの設定によってステップがエラー処理された場合は、COND条件によってステップが実行される必要がある場合にも、そのステップを実行せずに終了します。

参考

rcサブジェクトの詳細については、『OpenFrame 環境設定ガイド』を参照してください。

以下は、CONDオペランドについての説明です。

● 構文

```
COND = ((コード, 演算符号), ...)
```

項目	説明
コード	ジョブ・ステップのリターンコードと比較する条件コードを符号なし整数で指定します。
演算符号	以下の演算符号から適切な値を指定します。 – EQ: コードとリターンコードが等しい – NE: コードとリターンコードが等しくない – GT: コードがリターンコードより大きい – GE: コードがリターンコードより大きいか等しい – LT: コードがリターンコードより小さい – LE: コードがリターンコードより小さいか等しい

● 注意事項

- 条件パラメータは最大8つまで有効であり、9番目以上の条件パラメータは無視されます。条件パラメータを1つのみ指定した場合、最も外側にある括弧を省略することができます。

- JOB文とEXEC文にCONDが指定されている場合、JOB文のCONDが満たされるとEXEC文のCONDは考慮されません。
- CONDオペランドの値は、構文チェックの際にチェックしません。
- 例

以下は、リターンコードが0の場合のみジョブが実行されるように指定する例です。

```
//JOB1 JOB COND=(0,NE)
```

以下は、リターンコードが4、8、12の場合のみジョブを終了するように指定する例です。

```
//JOB1 JOB COND=((4,EQ),(8,EQ),(12,EQ))
```

以下は、リターンコードが0~12の場合のみジョブが実行されるように指定する例です。

```
//JOB1 JOB COND=(12,LT)
```

以下は、リターンコードが4~12の場合のみジョブが実行されるように指定する例です。

```
//JOB1 JOB COND=((4,GT),(12,LT))
```

2.11.7. GROUP

OpenFrameでは構文解析のみサポートします。

- 構文

```
GROUP = 値
```

項目	説明
値	8文字以内の記号名で指定します。

2.11.8. JESLOG

OpenFrameでは構文解析のみサポートします。

- 構文

```
JESLOG = {SPIN | NOSPIN | SUPPRESS}
```

2.11.9. LINES

OpenFrameでは構文解析のみサポートします。

- 構文

```
LINES = {値          }  
        {(値,CANCEL) }  
        {(値,DUMP)   }  
        {(値,WARNING)}
```

項目	説明
値	0～999999の符号なし整数です。

2.11.10. MEMLIMIT

OpenFrameでは構文解析のみサポートします。

- 構文

```
MEMLIMIT = {値M|値G|値T|値P}  
           {NOLIMIT}
```

項目	説明
値M	0～99999の符号なし整数です。
値G	0～99999の符号なし整数です。
値T	0～99999の符号なし整数です。
値P	0～16384の符号なし整数です。

2.11.11. MSGCLASS

ジョブ内のDD文でSYSOUT=*を指定するとき、「*」の値を指定します。

- 構文

```
MSGCLASS = class
```

項目	説明
class	出力クラスをA～Z、0～9の1文字で指定します。

- 注意事項

JOB文でこのオペランドを省略した場合、MSGCLASSはOpenFrame環境設定の**tjclrun**サブジェクト、**JOB**セクションの**MSGCLASS**キーのVALUE項目に指定された値が使用されます。tjclrunサブジェクトの詳細については、『OpenFrame 環境設定ガイド』を参照してください。

- 例

以下は、MSGCLASSをZに指定する例です。

```
//JOB1 JOB 8493,KIM,MSGCLASS=Z
```

2.11.12. MSGLEVEL

ジョブのシステム・メッセージの出力レベルを指定します。

- 構文

```
MSGLEVEL = ([値][,{0}])  
           {1}
```

項目	説明
{0} {1}	システム・メッセージの出力レベルを指定します。 – 0: システム・メッセージのうち、エラーメッセージのみを出力します。 – 1: システム・メッセージのうち、エラーメッセージと一緒にデータセットのLOCK、ALLOCATEなどの情報メッセージを出力します。

- 注意事項

JOB文でこのオペランドを省略した場合、MSGLEVELはOpenFrame環境設定の**tjclrun**サブジェクト、**JOB**セクションの**MSGLEVEL**キーのVALUE項目に指定された値が使用されます。tjclrunサブジェクトの詳細については、『OpenFrame 環境設定ガイド』を参照してください。

- 例

以下は、MSGCLASSをZ、MSGLEVELを1、0に指定する例です。

```
//JOB1 JOB MSGCLASS=Z,MSGLEVEL=(1,0)
```

2.11.13. NOTIFY

OpenFrameでは構文解析のみサポートします。

- 構文

```
NOTIFY = {nodename.userid}  
         {userid      }
```

項目	説明
nodename	8文字以内の記号名で指定します。
userid	8文字以内の記号名で指定します。

2.11.14. PAGES

構文エラーのみをチェックし、使用されません。

- 構文

```
PAGES = {値          }  
        {(値,CANCEL) }  
        {(値,DUMP)  }  
        {(値,WARNING)}
```

項目	説明
値	0～999999999の符号なし整数です。

2.11.15. PASSWORD

パスワードを指定します。JOB文のUSERオペランドと一緒に使用されます。そのため、USERオペランドを指定しないと、このオペランドは無視されます。USERオペランドを指定したものの、このオペランドを指定しない場合、システムはTACFのサロゲート・ユーザー機能を使用するものと判断します。

参考

TACFのサロゲート・ユーザー機能についての詳細は、『OpenFrame TJESガイド』の「3.1.12 セキュリティ」を参照してください。

以下は、PASSWORDオペランドについての説明です。

- 構文

```
PASSWORD = password
```

項目	説明
password	OpenFrameセキュリティ・モジュールであるTACFのユーザー・パスワードを指定します。 パスワードは8文字以内の英数字を指定します。

- 例

以下は、ユーザーをKIMに指定し、パスワードをtmaxsoftに指定する例です。

```
//JOB1 JOB USER=KIM, PASSWORD=tmaxsoft
```

以下は、ユーザーをKIMに指定したもののパスワードは指定しなかったため、TACFのサロゲート・ユーザー機能を使用する例です。

```
//JOB1 JOB USER=KIM
```

2.11.16. PERFORM

ジョブが実行中に属するパフォーマンス・グループを指定します。OpenFrameでは、環境設定に登録されたパフォーマンス・グループ番号に応じたNICE値に変更させます。

OpenFrameでは、パフォーマンス・グループ番号をUNIXのNICE値に置き換えて使用します。オペランドが指定されると、ランナーはパフォーマンス・グループ番号をNICE値に置き換えて、ステップ・プログラムのNICE値を変更します。

JOB文とSTEP文の両方ともPERFORMが指定されている場合は、該当するステップのPERFORM設定に従い、残りのステップではジョブのPERFORM設定に従います。

● 構文

```
PERFORM = パフォーマンスグループ番号
```

項目	説明
パフォーマンスグループ番号	1~999の符号なし整数でパフォーマンス・グループ番号を指定します。

● 注意事項

- PERFORMオペランドをNICE機能に置き換えて使用する場合は、OpenFrame環境設定の**tjclrun**サブジェクト、**PERFORM**セクションの**USE_PERFORM**キーの**VALUE**項目を**YES**に設定する必要があります。tjclrunサブジェクトの詳細については、『OpenFrame 環境設定ガイド』を参照してください。
- NICEの値を変更するには、スーパーユーザーの権限が必要であるため、ランナーに**setuid**を一緒に設定します。詳細については、『OpenFrame TJESガイド』の「第3章 ジョブの実行」の「**setuid root tjclrun**」を参照してください。

● 例

以下は、PERFORMを100に指定する例です。

```
//JOB1 JOB PERFORM=100
```

2.11.17. PRTY

ジョブの優先順位を指定します。ジョブの優先順位は、ジョブ・クラスと一緒にスケジューリングに使用され、数字が高いほど優先順位が高いです。

スケジューラーでは、同じクラスを持つジョブがある場合、このオペランドの値に応じてどのジョブを先に実行するかを決定します。同じ優先順位である場合は、先にサブミットされたジョブが実

行されます。ジョブの優先順位は、エイジング(Aging)ポリシーに従って、時間が経つにつれ高くなります。優先順位については、OpenFrame環境設定の**tjes**サブジェクト、**SCHEDULING**セクションの**PRTYHIGH**、**PRTYLOW**キーのVALUE項目を参照してください。

参考

tjesサブジェクトの詳細については、『OpenFrame 環境設定ガイド』を参照してください。

- 構文

```
PRTY = 優先順位
```

項目	説明
優先順位	0~15の符号なし整数でジョブの優先順位を指定します。

- 注意事項

このオペランドを使用するには、OpenFrame環境設定の**tjes**サブジェクト、**SCHEDULING**セクションの**PRTYJOB**キーのVALUE項目をYESに設定する必要があります。

- 例

以下は、PRTYを5に指定する例です。

```
//JOB1 JOB PRTY=5
```

2.11.18. RD

OpenFrameでは構文解析のみサポートします。

- 構文

```
RD = {R | RNC | NR | NC}
```

2.11.19. REGION

OpenFrameでは構文解析のみサポートします。

- 構文

```
REGION = {値K | 値M}
```

項目	説明
値K	0~2096128の符号なし整数です。
値M	0~2047の符号なし整数です。

2.11.20. RESTART

ジョブの再開ステップを指定します。ジョブが何らかの理由で実行中に終了され、その後このジョブを再実行する際、途中から再開したい場合に指定するオペランドです。このオペランドの値を指定すると、ジョブは指定されたステップから実行されます。

該当するステップ名が複数あるときは、一番最初に出てくるステップから始めます。存在しないステップを適用すると、フラッシュが発生します。

- 構文

```
RESTART = {*}  
          {STEP名}  
          {stepname.procstepname}
```

項目	説明
(*)アスタリスク	再開ステップが最初のステップであることを示します。
STEP名	再開ステップの名前を指定します。名前は8文字以内の記号名で指定します。
stepname.procstepname	再開ステップがプロシージャ内のステップである場合は、プロシージャ・ステップ名と再開するプロシージャ内のステップ名を以下の例のように指定します。

- 例

以下は、RESTARTをアスタリスク(*)に指定する例です。

```
//JOB1 JOB RESTART=*
```

以下は、RESTARTをSTEP3に指定する例です。

```
//JOB1 JOB RESTART=STEP3
```

以下は、プロシージャ内のステップ名を一緒に指定する例です。

```
//JOB1 JOB RESTART=(STEP3.PSTEP4)
```

2.11.21. SECLABEL

OpenFrameでは構文解析のみサポートします。

- 構文

```
SECLABEL = 値
```

項目	説明
値	8文字以内の記号文字です。

2.11.22. SCHENV

OpenFrameでは構文解析のみサポートします。

- 構文

```
SCHENV = 値
```

項目	説明
値	16文字以内の記号文字です。

2.11.23. SPARM

システム・パラメータの値を指定します。

- 構文

```
SPARM = '[システムパラメータ]=[値]'
```

項目	説明
システムパラメータ	システムに通知するパラメータを指定します。現在は、JCLの実行日付を設定するDATEと実行時間を設定するTIMEのみサポートします。
値	DATEパラメータは以下の形式で指定します。 – YYYY.MM.DD : YYYYは1900年代または2000年代の4桁を表現 – YY.MM.DD : YYは1900年代の2桁を表現 TIMEパラメータは「HH:MM:SS」形式で指定します。

- 例

以下は、JOB文のSPARMのDATEパラメータに2010年4月1日を、TIMEパラメータに12時24分30秒を指定する例です。

```
//JOB1 JOB CLASS=A,MSGCLASS=X,SPARM='DATE=2010.04.01,TIME=12:24:30'
```

2.11.24. TIME

ジョブで使用できるCPU時間の最大値を指定します。指定されたCPU時間を超過した場合、該当するジョブはフラッシュで終了されます。

- 構文

```
TIME = {[分],[秒]}
      {1440      }
      {NOLIMIT   }
      {MAXIMUM   }
      {0         }
```

項目	説明
分	0~357912の符号なし整数です。CPU時間の分単位の値です。
秒	0~59の符号なし整数です。CPU時間の秒単位の値です。
1440	値に制限がないことを示します。CPU時間に制限がありません。
NOLIMIT	1440と同様に値に制限がないことを示します。CPU時間に制限がありません。
0	1440と同様に値に制限がないことを示します。CPU時間に制限がありません。
MAXIMUM	357912分を示します。

- 注意事項

- 分のみを指定する場合は括弧を省略できます。
- 指定しない場合は、OpenFrame環境設定の**tjclrun**サブジェクト、**JOB**セクションの**TIME**キーのVALUE項目の値が使用されます。設定された値がない場合は、0です。tjclrunサブジェクトの詳細については、『OpenFrame 環境設定ガイド』を参照してください。
- tjclrunでCPU時間のチェックを5秒ごとに行うため、最大5秒の誤差が生じることがあります。
- JOB文にTIME=1440が指定された場合、EXEC文のすべてのTIME値およびデフォルトのTIME値は無効になります。ジョブ内のすべてのステップはTIME=1440またはTIME=NOLIMITを指定したように、CPU時間に制限がありません。

- 例

以下は、JOB1のTIMEを10秒に指定する例です。

```
//JOB1      JOB      CLASS=A,MSGCLASS=A,TIME=(,10)
//STEP1     EXEC     PGM=IEFBR14
//SYSOUT    DD       SYSOUT=*
//
```

以下は、ジョブ1のTIMEを2分に指定する例です。


```
//JOB1      JOB      CLASS=A,MSGCLASS=A,TIME=2
//STEP1     EXEC     PGM=IEFBR14
//SYSOUT    DD       SYSOUT=*
//
```

以下は、ジョブ1のTIMEをMAXIMUMに指定する例です。

```
//JOB1      JOB      CLASS=A,MSGCLASS=A,TIME=MAXIMUM
//STEP1     EXEC     PGM=IEFBR14
//SYSOUT    DD       SYSOUT=*
//
```

2.11.25. TIMECONTROLLER

ジョブで日付または時間を設定するために値を指定します。

- 構文

```
{xxcyyddd}
  {xxdhmm}
  {xxYDnn}
```

項目	説明
xx	タイム・コントローラーを使用するための2文字の接頭辞です。
c	世紀を示します。0は20世紀、1は21世紀を示します。
yy	2桁の年度を示します。00～99を作成できます。
ddd	ユリウス日を示します。000～366を作成できます。
d	現在の日付を基準にしてオフセットの時間と分を指定します。 – P : 実際のシステム時間にhhmmを足します。 – M : 実際のシステム時間からhhmmを引きます。 – F : hhmmに指定した値をシステム時間として設定した後、ユーザー・アプリケーションが時間を要求すると、本構文で設定したシステム時間から経過した時間を表示します – A : hhmmに指定した値を使用します。 – E : Pと同じ機能です。 – W : Mと同じ機能です。
hh	時間を示します。00～23を作成できます。
mm	分を示します。00～59を作成できます。

項目	説明
Y	年度のオフセットを指定するために必要な文字です。
D	現在の日付を基準にしてオフセットの年度を指定します。 – P : 年度を増加させます。 – M : 年度を減少させます。
nn	年度のオフセット値を指定します。

● 注意事項

- OpenFrame環境設定の**tjclrun**サブジェクト、**OPTION**セクションの**TIME_CONTROLLER**キーのVALUE項目がYESに設定されている必要があります。NOに設定されているか、値が指定されていない場合は使用できません。
- OpenFrame環境設定の**tjclrun**サブジェクト、**OPTION**セクションの**TIME_CONTROLLER_PREFIX**キーのVALUE項目に2文字の接頭辞が設定されている必要があります。

参考

tjclrunサブジェクトの詳細については、『OpenFrame 環境設定ガイド』を参照してください。

● 例

以下は、年度を「2030」に指定する例です。

```
//JOB1      JOB      0000000000, '//TCYP10'
//STEP1     EXEC     PGM=IEFBR14
//SYSOUT    DD       SYSOUT=*
//
```

以下は、日付を「20201231」に指定する例です。

```
//JOB1      JOB      0000000000, '//TC120365'
//STEP1     EXEC     PGM=IEFBR14
//SYSOUT    DD       SYSOUT=*
//
```

以下は、システム時間に1時間を足した時間を指定する例です。

```
//JOB1      JOB      0000000000, '//TCP0100'
//STEP1     EXEC     PGM=IEFBR14
//SYSOUT    DD       SYSOUT=*
//
```

2.11.26. TYPRUN

ジョブの実行タイプを指定します。OpenFrameでは、HOLD、JCLHOLD、JEM、SCANのみサポートします。

JCLがサブミットされる手順は以下のとおりです。

1. JCLが入力されます。
2. 入力されたJCLの基本的な構文をチェックする構文解析を行います。

基本的な構文チェックには、ジョブ制御文の位置、それぞれの制御文に適したキーワード、そのキーワードに入力可能な値、継続行の指定についてのチェックなどが含まれます。

3. 入力されたJCLで使用するプロシージャをスプールにコピーし、そのプロシージャの基本的な構文チェックを行うコンバージョン処理をします。
4. スケジューリング処理をします。

以下は、TYPRUNオペランドについての説明です。

● 構文

```
TYPRUN = {COPY}
         {HOLD}
         {JCLHOLD}
         {JEM}
         {SCAN}
```

項目	説明
COPY	OpenFrameでは構文解析のみサポートします。
HOLD	JCLの構文解析とコンバージョンまで処理し、スケジューリング処理はしていない状態で、HOLD状態になります。 TJESMGRのPSまたはPSJOBコマンドを使用して確認できます。 スケジューリング処理は、TJESMGRのSTARTコマンドを使用して変更できます。
JCLHOLD	JCLの構文解析のみ処理し、JCLHOLD状態になります。 ジョブの[W]HOLD状態であり、TJESMGRのPSとPSJOBコマンドを実行するか、または OpenFrame Manager の[Batch]メニューで確認できます。 コンバージョンとスケジューリングの実行は、TJESMGRのSTARTコマンドを使用して変更できます。
JEM	JCLに記述されたプログラム、データセット、データセットのカatalogが存在するかどうかをチェックし、SYSMSGにそのJCLの論理的なエラーに関するレポートを記録します。エラーがない場合はDONE(M00000)、エラーがある場合はDONE(M00001)で終了されます。

項目	説明
	実際にジョブを実行せずに、エラーが発生し得る要素を事前に把握できる機能です。このオペランドを指定されたジョブはスケジューリングされ、ランナーで処理されます。
SCAN	JCLの構文分析とコンバージョンまで処理され、スケジューリングは実行されていない状態です。結果レポートをSYSMSGに記録した後、DONE(D00000)で終了されます。 このオペランドを指定すると、ジョブのサブミットのみチェックされ、実際のスケジューリングは実行されません。

- 注意事項

TYPRUNオペランドが指定されていない場合、JCLはすべてのサブミット・プロセスを経てスケジューリング処理されます。

- 例

以下は、TYPRUNをSCANに指定する例です。

```
//JOB1    JOB    CLASS=A,MSGCLASS=A,TYPRUN=SCAN
//STEP1   EXEC   PGM=IEFBR14
//SYSOUT  DD     SYSOUT=*
//
```

以下は、SYSMSGに記録された結果レポートです。

```
-----
** JCL SCAN start **
-----
1 //JOB1    JOB    CLASS=A,MSGCLASS=A,TYPRUN=SCAN
2 //STEP1   EXEC   PGM=IEFBR14
3 //SYSOUT  DD     SYSOUT=*
  //
-----
** JCL SCAN finish **
-----
```

以下は、TYPRUNをSCANに指定し、JCLの構文解析でエラーが発生した例です。

```
//JOB1    JOB    CLASS=A,MSGCLASS=A,TYPRUN=SCAN
//STEP1   EXEC   PGM=IEFBR14, this comma is wrong.
//SYSOUT  DD     SYSOUT=*
//
```

以下は、SYSMSGに記録されたレポートです。

```

1 //JOB1      JOB    CLASS=A,MSGCLASS=A,TYPRUN=SCAN
2 //STEP1     EXEC   PGM=IEFBR14,  this comma is wrong.
3 //SYSOUT    DD     SYSOUT=*
//
Syntax Error [Line:2;Column: ;Keyword: ;Message:Expected continuation not
received]

```

以下は、TYPRUNをJEMに指定し、JCLエラー・チェックによってエラーが検出された例です。STEP01ではIEFBR141というプログラムがなく、STEP02ではSHRに指定したDD02が存在しないため、エラーが発生しています。

```

//JEMTEST    JOB TYPRUN=JEM
//STEP01     EXEC PGM=IEFBR141
//DD01       DD DSN=TMAX.JEM.DATASET,DISP=(MOD,DELETE)
//STEP02     EXEC PGM=IEFBR14
//DD02       DD DSN=TMAX.JEM.DATASET,DISP=SHR
//STEP03     EXEC PGM=IEFBR14
//DD03       DD DSN=TMAX.JEM.DATASET,DISP=MOD

```

以下は、SYSMSGに記録されたレポートです。

```

JOBID=JOB00028   JOBPOS=0   RUNPID=8998
JEMTEST JEM (test)
=====

----- STEP01 EXEC PGM step -----

EXEC PGM=IEFBR141 (JEM mode)
(JRN0143F) no program file(IEFBR141) exist - check JCL JOBLIB, JCL STEPLIB, and
environment PATH
----- STEP02 EXEC PGM step -----

EXEC PGM=IEFBR14 (JEM mode)
(JRN2008E) [OLDD02] Dataset 'TMAX.JEM.DATASET' does not exists.
----- STEP03 EXEC PGM step -----

EXEC PGM=IEFBR14 (JEM mode)
=====

(JRN2010I) JEM RUN skip enqueue for output processing ok
(JRN2017I) JEM RUN total error count : 2
----- JEM PROCESS FINISHED -----

```

2.11.27. USER

ユーザーを指定します。このオペランドは、PASSWORDオペランドのパスワードと一緒に使用し、TACFにユーザー認証およびリソースへの使用権限をチェックする用途で使用します。

JCLをサブミットすると、システムはTACFでユーザーへの認証をチェックします。USERオペランドが指定されている場合、PASSWORDオペランドのパスワードと一緒に認証チェック（実行ユーザー）を行います。USERオペランドが指定されていない場合は、JCLをサブミットしたユーザーでユーザー認証チェック（サブミット・ユーザー）を行います。

このオペランドを指定したものの、PASSWORDオペランドは指定しなかった場合、システムはTACFサロゲート・ユーザー機能を使用して、ユーザー認証チェック（サロゲート・ユーザー）を行います。

ランナーでは、ユーザー認証が行われた実行ユーザー、サブミット・ユーザーまたはサロゲート・ユーザーで使用権限をチェックします。使用権限チェックの対象となるリソースは、ジョブで使用するすべてのデータセットおよびプログラムです。ユーザーが特定のリソースについて権限チェックを行うには、対象リソースをTACFに登録する必要があります。そのリソースがデータセットであればDATASET CLASSに、プログラムであればUTILITY CLASSに登録します。

参考

TACFのサロゲート・ユーザー機能の詳細については、『OpenFrame TJESガイド』の「3.1.12 セキュリティ」を参照してください。

以下は、USERオペランドの説明です。

- 構文

```
USER = user
```

項目	説明
user	OpenFrameセキュリティ・モジュールであるTACFのユーザーを指定します。 ユーザーは8文字以内の記号名で指定します。

- 注意事項

DATASETとUTILITYの使用権限チェックについては、OpenFrame環境設定のtjclrunサブジェクト、TACFセクションのCHECK_DSAUTH、CHECK_UTAUTHキーのVALUE項目から確認します。tjclrunサブジェクトの詳細については、『OpenFrame 環境設定ガイド』を参照してください。

- 例

以下は、ユーザーをKIMに指定し、パスワードをtmaxsoftに指定する例です。

```
//JOB1 JOB USER=KIM,PASSWORD=tmaxsoft
```

以下は、ユーザーをKIMに指定したが、パスワードを指定しなかったため、TACFサロゲート・ユーザー機能を使用する例です。

```
//JOB1 JOB USER=KIM
```

2.12. OUTPUT文

出力処理をするためのSYSOUTデータセットの属性を指定します。

- 構文

```
//名前 OUTPUT△1キーワードオペランド[,キーワードオペランド]...△1[コメント]
```

項目	説明
名前	名前を指定します。「//」に続いて3桁目から始める必要があります。OUTPUT文の名前は省略することができません。DD文でOUTPUTオペランドを使用して該当するOUTPUT文を参照する際に使用されます。
OUTPUT	オペレーションを指定します。名前の後ろに1つ以上の空白を入れて「OUTPUT」と記述します。
キーワードオペランド[, キーワードオペランド]	OUTPUTの後ろに1つ以上の空白を入れて、キーワード・オペランドを記述します。キーワード・オペランドの順序は関係ありません。詳細については、 オペランド と各オペランドの節を参照してください。
[コメント]	オペランドの後ろに1つ以上の空白を入れて記述します。コメントは71桁目まで記述できます。

オペランド

以下は、オペランドについての説明です。各オペランドの詳細については、各節を参照してください。

● キーワード・オペランド

項目	説明
ADDRESS	SYSOUTデータセットが渡されるアドレスを指定します。
AFPSTATS	OpenFrameでは構文解析のみサポートします。
BUILDING	SYSOUTデータセットと関連する建物名を指定します。
BURST	OpenFrameでは構文解析のみサポートします。
CHARS	データセットを出力する際の出力文字とサイズに関する設定テーブルを指定します。
CKPTLINE	OpenFrameでは構文解析のみサポートします。
CKPTPAGE	OpenFrameでは構文解析のみサポートします。
CKPTSEC	OpenFrameでは構文解析のみサポートします。
CLASS	SYSOUTデータセットの出力クラスを指定します。DD文のSYSOUTパラメータにクラスが指定されていない場合にのみ適用されます。
COLORMAP	OpenFrameでは構文解析のみサポートします。
COMPACT	OpenFrameでは構文解析のみサポートします。
COMSETUP	OpenFrameでは構文解析のみサポートします。
CONTROL	OpenFrameでは構文解析のみサポートします。
COPIES	SYSOUTデータセットのコピー回数を指定します。
DATAACK	OpenFrameでは構文解析のみサポートします。
DEFAULT	OpenFrameでは構文解析のみサポートします。
DEPT	出力データセットと関連する部署のIDを指定します。
DEST	SYSOUTデータセットの出力先を指定します。
DPAGELBL	OpenFrameでは構文解析のみサポートします。
DUPLEX	OpenFrameでは構文解析のみサポートします。
FCB	FCB名を指定します。
FLASH	用紙に一定の書式、範囲をあらかじめ印刷する場合、使用するフィルム・オーバーレイの識別名を指定します。
FORMDEF	FORMDEF名を指定します。
FORMLEN	OpenFrameでは構文解析のみサポートします。
FORMS	SYSOUTデータセットの書式番号を指定します。
FSSDATA	OpenFrameでは構文解析のみサポートします。

項目	説明
GROUPID	OpenFrameでは構文解析のみサポートします。
INDEX	OpenFrameでは構文解析のみサポートします。
INTRAY	OpenFrameでは構文解析のみサポートします。
JESDS	OpenFrameでは構文解析のみサポートします。
LINDEX	OpenFrameでは構文解析のみサポートします。
LINECT	OpenFrameでは構文解析のみサポートします。
MAILBCC	Eメールのブラインドカーボンコピー(BCC)宛先の1つ以上のEメール・アドレスを指定します。
MAILCC	Eメールのカーボンコピー(CC)宛先の1つ以上のEメール・アドレスを指定します。
MAILFILE	OpenFrameでは構文解析のみサポートします。
MAILFROM	Eメールの送信側の名前または他のIDを指定します。
MAILTO	Eメールの受信側の1つ以上のEメール・アドレスを指定します。
MODIFY	SYSOUTデータセットのコピー修飾モジュール名と、CHARSオペランドで指定するテーブルのうち、使用するテーブルの順序番号を指定します。
NAME	SYSOUTデータセットと関連する名前を指定します。
NOTIFY	OpenFrameでは構文解析のみサポートします。
OFFSETXB	OpenFrameでは構文解析のみサポートします。
OFFSETXF	OpenFrameでは構文解析のみサポートします。
OFFSETYB	OpenFrameでは構文解析のみサポートします。
OFFSETYF	OpenFrameでは構文解析のみサポートします。
OUTBIN	OpenFrameでは構文解析のみサポートします。
OUTDISP	SYSOUTデータセットで作成されたOUTPUTの処理方法を指定します。
OVERLAYB	印刷するページの背面に置かれるオーバーレイ名を指定します。
OVERLAYF	印刷するページの前面に置かれるオーバーレイ名を指定します。
OVFL	OpenFrameでは構文解析のみサポートします。
PAGEDEF	PAGEDEF名を指定します。
PIMSG	OpenFrameでは構文解析のみサポートします。
PORTNO	OpenFrameでは構文解析のみサポートします。
PRTERORR	OpenFrameでは構文解析のみサポートします。

項目	説明
PRTOPTNS	OpenFrameでは構文解析のみサポートします。
PRTQUEUE	OpenFrameでは構文解析のみサポートします。
PRTY	OpenFrameでは構文解析のみサポートします。
REPLYTO	Eメールの受信側が応答できるEメール・アドレスを指定します。
RESFMT	OpenFrameでは構文解析のみサポートします。
RETAINS	正常に送信されたデータ・セットを保存する時間を 指定します。
RETAINF	OpenFrameでは構文解析のみサポートします。
RETRYL	OpenFrameでは構文解析のみサポートします。
RETRYT	OpenFrameでは構文解析のみサポートします。
ROOM	SYSOUTデータセットの出力の分離ページに部屋IDを指定します。
SYSAREA	OpenFrameでは構文解析のみサポートします。
THRESHLD	OpenFrameでは構文解析のみサポートします。
TITLE	SYSOUTデータセットの出力の分離ページに印刷する説明を指定します。
TRC	SYSOUTデータセット内の各出力行の論理レコードにテーブル参照文字 (TRC)コードが含まれるかどうかを指定します。
UCS	汎用文字セットを指定します。
USERDATA	ユーザー定義データを指定します。
USERLIB	USERLIBデータセット名を指定します。
USERPATH	OpenFrameでは構文解析のみサポートします。
WRITER	外部出力モジュールを指定します。

2.12.1. ADDRESS

SYSOUTデータセットが渡されるアドレスを指定します。

- 構文

```
ADDRESS = (アドレス値[,アドレス値]...)
```

項目	説明
アドレス値	出力先のアドレスを文字列で指定します。アドレス値の最大長は60バイトまでで、10個まで指定できます。

- 注意事項

OpenFrameでは、印刷出力のためのADDRESS情報を外部プリンター・モジュールに渡すために、ジョブ実行の完了後に保存しています。その情報を使用するかどうかは、外部プリンター・モジュールの必要に応じて異なります。

- 例

以下は、SYSOUTデータセットが渡されるアドレスを '1234 Main Street', 'ABCD'に指定する例です。

```
//OUT1 OUTPUT ADDRESS=('1234 Main Street','ABCD')
//OUT DD SYSOUT=A,OUTPUT=*.OUT1
```

2.12.2. AFPSTATS

OpenFrameでは構文解析のみサポートします。

- 構文

```
AFPSTATS = {YES | Y | NO | N}
```

2.12.3. BUILDING

SYSOUTデータセットと関連する建物名を指定します。

- 構文

```
BUILDING = 建物名
```

項目	説明
建物名	出力データセットと関連する建物名を指定します。建物名の値は、最大60バイトまでの文字列です。

- 注意事項

OpenFrameでは、印刷出力のためのBUILDING情報を外部プリンター・モジュールに渡すために、ジョブ実行の完了後に保存しています。その情報を使用するかどうかは、外部プリンター・モジュールの必要に応じて異なります。

- 例

以下は、SYSOUTデータセットと関連する建物名を 'ABC Tower'に指定する例です。

```
//OUT1 OUTPUT BUILDING='ABC Tower'
//OUT DD SYSOUT=G,OUTPUT=*.OUT1
```

2.12.4. BURST

OpenFrameでは構文解析のみサポートします。

- 構文

```
BURST = {YES | Y | NO | N}
```

2.12.5. CHARS

データセットを出力する際の出力文字とサイズについての設定テーブルを指定します。

- 構文

```
CHARS = (テーブル名[,テーブル名]...)
```

項目	説明
テーブル名	データセットを出力する際の出力文字とサイズについての設定テーブルを1～4文字の英数字で指定します。最大4つまで指定できます。指定するテーブルが1つの場合は、括弧を省略できます。

- 注意事項

OpenFrameでは、印刷出力のためのCHARS情報を外部プリンター・モジュールに渡すために、ジョブ実行の完了後に保存しています。その情報を使用するかどうかは、外部プリンター・モジュールの必要に応じて異なります。

- 例

以下は、データ出力文字とサイズ設定テーブルとしてTBL1、TBL2を指定する例です。

```
//OUT1 OUTPUT CHARS=(TBL1,TBL2)
//OUT DD SYSOUT=A, OUTPUT=* .OUT1
```

2.12.6. CKPTLINE

OpenFrameでは構文解析のみサポートします。

- 構文

```
CKPTLINE = 値
```

項目	説明
値	0～32767の符号なし整数を指定します。

2.12.7. CKPTPAGE

OpenFrameでは構文解析のみサポートします。

- 構文

```
CKPTPAGE = 値
```

項目	説明
値	1～32767の符号なし整数を指定します。

2.12.8. CKPTSEC

OpenFrameでは構文解析のみサポートします。

- 構文

```
CKPTSEC = 値
```

項目	説明
値	1～32767の符号なし整数を指定します。

2.12.9. CLASS

SYSOUTデータセットの出力クラスを指定します。DD文のSYSOUTパラメータにクラスが指定されていない場合にのみ適用されます。

- 構文

```
CLASS = output class
```

項目	説明
output class	A～Z、0～9の1文字を指定します。

- 注意事項

DD文のSYSOUTパラメータにクラスが指定されている場合はその設定が優先され、指定されていない場合にのみこの設定が適用されます。

- 例

以下は、SYSOUTデータセットの出力クラスをBに指定する例です。

```
//OUT1 OUTPUT CLASS=B  
//OUT DD SYSOUT=*,OUTPUT=*.OUT1
```

2.12.10. COLORMAP

OpenFrameでは構文解析のみサポートします。

- 構文

```
COLORMAP = 値
```

項目	説明
値	8文字以内の記号名を指定します。

2.12.11. COMPACT

OpenFrameでは構文解析のみサポートします。

- 構文

```
COMPACT = 値
```

項目	説明
値	8文字以内の記号名を指定します。

2.12.12. COMSETUP

OpenFrameでは構文解析のみサポートします。

- 構文

```
COMSETUP = 値
```

項目	説明
値	8文字以内の記号名を指定します。

2.12.13. CONTROL

OpenFrameでは構文解析のみサポートします。

- 構文

```
CONTROL = {PROGRAM | SINGLE | DOUBLE | TRIPLE}
```

2.12.14. COPIES

SYSOUTデータセットのコピー回数を指定します。

- 構文

```
COPIES= ([コピー回数][,グループコピー回数...])
```

項目	説明
コピー回数	SYSOUTデータセットのコピー回数を1～255の符号なし整数で指定します。コピー回数のみ指定する場合は、括弧を省略できます。（デフォルト値: 1）
グループコピー回数	SYSOUTデータセットのページ単位のグループ・コピー回数を1～255の符号なし整数で指定します。最大8つまで指定できます。

- 注意事項

OpenFrameでは、印刷出力のためのCOPIES情報を外部プリンター・モジュールに渡すために、ジョブ実行の完了後に保存しています。その情報を使用するかどうかは、外部プリンター・モジュールの必要に応じて異なります。

- 例

以下は、SYSOUTデータセットのコピー回数を10に指定する例です。

```
//OUT1 OUTPUT COPIES=10  
//OUT DD SYSOUT=J,OUTPUT=*.OUT1
```

2.12.15. DATAK

OpenFrameでは構文解析のみサポートします。

- 構文

```
DATAK = {BLOCK | UNBLOCK | BLKCHAR | BLKPOS}
```

2.12.16. DEFAULT

OpenFrameでは構文解析のみサポートします。

- 構文

```
DEFAULT = {YES | Y | NO | N}
```

2.12.17. DEPT

出力データセットと関連する部署のIDを指定します。

- 構文

```
DEPT = 部署ID
```

項目	説明
部署ID	出力データセットと関連する部署のIDを指定します。最大60バイトの文字列で指定します。

- 注意事項

OpenFrameでは、印刷出力のためのDEPT情報を外部プリンター・モジュールに渡すために、ジョブ実行の完了後に保存しています。その情報を使用するかどうかは、外部プリンター・モジュールの必要に応じて異なります。

- 例

以下は、SYSOUTデータセットと関連する部署のIDを'FINANCE'に指定する例です。

```
//OUT1 OUTPUT DEPT='FINANCE'  
//OUT DD SYSOUT=J, OUTPUT=*.OUT1
```

2.12.18. DEST

SYSOUTデータセットの出力先を指定します。

- 構文

```
EST = デスティネーション
```

項目	説明
出力先	SYSOUTデータセットのデスティネーションを指定します。

- 注意事項

OpenFrameでは、印刷出力のためのDEST情報を外部プリンター・モジュールに渡すために、ジョブ実行の完了後に保存しています。その情報を使用するかどうかは、外部プリンター・モジュールの必要に応じて異なります。

- 例

以下は、SYSOUTデータセットのデスティネーションをBCCOMPに指定する例です。

```
//OUT1 OUTPUT DEST=BCCOMP  
//OUT DD SYSOUT=J, OUTPUT=*.OUT1
```


2.12.19. DPAGELBL

OpenFrameでは構文解析のみサポートします。

- 構文

```
DPAGELBL = {YES | Y | NO | N}
```

2.12.20. DUPLEX

OpenFrameでは構文解析のみサポートします。

- 構文

```
DUPLEX = {NO | N | NORMAL | TUMBLE}
```

2.12.21. FCB

FCB名を指定します。

- 構文

```
FCB = FCB名
```

項目	説明
FCB名	FCB名を1～4文字の記号名で指定します。

- 注意事項

OpenFrameでは、印刷出力のためのFCB情報を外部プリンター・モジュールに渡すために、ジョブ実行の完了後に保存しています。その情報を使用するかどうかは、外部プリンター・モジュールの必要に応じて異なります。

- 例

以下は、FCB名をAA33に指定する例です。

```
//OUT1 OUTPUT FCB=AA33
//OUT DD SYSOUT=J, OUTPUT=*.OUT1
```

2.12.22. FLASH

用紙に一定の書式、範囲をあらかじめ印刷する場合に使用するフィルム・オーバーレイの識別名を指定します。

- 構文

FLASH = フィルム名[, 適用枚数]

項目	説明
フィルム名	用紙に一定の書式、範囲をあらかじめ印刷する場合に使用するフィルム・オーバーレイの識別名を指定します。識別名は1～4文字の記号名で指定します。
適用枚数	用紙に一定の書式、範囲をあらかじめ印刷する場合に使用するフィルム・オーバーレイの適用枚数を1～255の符号なし整数で指定します。（デフォルト値: 0）

- 注意事項

OpenFrameでは、印刷出力のためのFLASH情報を外部プリンター・モジュールに渡すために、ジョブ実行の完了後に保存しています。その情報を使用するかどうかは、外部プリンター・モジュールの必要に応じて異なります。

- 例

以下は、フィルム・オーバーレイの識別名をFLM1に指定する例です。

```
//OUT1 OUTPUT FLASH=FLM1
//OUT DD SYSOUT=A, SYSOUT=* .OUT1
```

2.12.23. FORMDEF

FORMDEF名を指定します。

- 構文

FORMDEF = メンバー

項目	説明
メンバー	FORMDEF名を1～6文字の記号名で指定します。ただし、メンバー名にピリオド(.)は使用できません。

- 注意事項

OpenFrameでは、印刷出力のためのFORMDEF情報を外部プリンター・モジュールに渡すために、ジョブ実行の完了後に保存しています。その情報を使用するかどうかは、外部プリンター・モジュールの必要に応じて異なります。

- 例

以下は、FORMDEFのメンバーをJJPRトに指定する例です。

```
//OUT1 OUTPUT FORMDEF=JJPRト
//OUT DD SYSOUT=A, OUTPUT=* .OUT1
```

2.12.24. FORMLEN

OpenFrameでは構文解析のみサポートします。

- 構文

```
FORMLEN = 値
```

2.12.25. FORMS

SYSOUTデータセットの書式番号を指定します。

- 構文

```
FORMS = 書式番号
```

項目	説明
書式番号	SYSOUTデータセットの書式番号を1～4文字の記号名で指定します。ただし、書式番号にピリオド(.)は使用できません。

- 注意事項

OpenFrameでは、印刷出力のためのFORMS情報を外部プリンター・モジュールに渡すために、ジョブ実行の完了後に保存しています。その情報を使用するかどうかは、外部プリンター・モジュールの必要に応じて異なります。

- 例

以下は、SYSOUTデータセットの書式番号を5に指定する例です。

```
//OUT1 OUTPUT FLASH=FLM1, FORMS=5  
//OUT DD SYSOUT=A, OUTPUT=*.OUT1
```

2.12.26. FSSDATA

OpenFrameでは構文解析のみサポートします。

- 構文

```
FSSDATA = 値
```

2.12.27. GROUPID

OpenFrameでは構文解析のみサポートします。

- 構文

GROUPID = 値

項目	説明
値	8文字以内の記号名を指定します。

2.12.28. INDEX

OpenFrameでは構文解析のみサポートします。

- 構文

INDEX = 値

項目	説明
値	1～31の符号なし整数を指定します。

2.12.29. INTRAY

OpenFrameでは構文解析のみサポートします。

- 構文

INTRAY = 値

項目	説明
値	1～255の符号なし整数を指定します。

2.12.30. JESDS

OpenFrameでは構文解析のみサポートします。

- 構文

JESDS = {ALL | JCL | LOG | MSG}

2.12.31. LINDEX

OpenFrameでは構文解析のみサポートします。

- 構文

LINDEX = 値

項目	説明
値	1～31の符号なし整数を指定します。

2.12.32. LINECT

OpenFrameでは構文解析のみサポートします。

- 構文

LINECT = 値

項目	説明
値	1～255の符号なし整数を指定します。

2.12.33. MAILBCC

Eメールのブラインドカーボンコピー(BCC)宛先の1つ以上のEメール・アドレスを指定します。

- 構文

MAILBCC = (メールアドレス[, メールアドレス]...)

項目	説明
メールアドレス	Eメールのブラインドカーボンコピー(BCC)宛先のメール・アドレスを指定します。最大長は60バイトまでで、10個まで指定できます。

- 注意事項

OpenFrameでは、印刷出力のためのMAILBCC情報を外部プリンター・モジュールに渡すために、ジョブ実行の完了後に保存しています。その情報を使用するかどうかは、外部プリンター・モジュールの必要に応じて異なります。

- 例

以下は、Eメールのブラインドカーボンコピー(BCC)宛先のメール・アドレスを指定する例です。

```
//OUT1 OUTPUT MAILBCC=('michael@companya.com','jane@companyb.net')
//OUT DD SYSOUT=J, OUTPUT=*.OUT1
```

2.12.34. MAILCC

Eメールのカーボンコピー(CC)宛先の1つ以上のEメール・アドレスを指定します。

- 構文

```
MAILCC = (メールアドレス[, メールアドレス]...)
```

項目	説明
メールアドレス	Eメールのカーボンコピー(CC)宛先のメール・アドレスを指定します。最大長は60バイトまでで、10個まで指定できます。

- 注意事項

OpenFrameでは、印刷出力のためのMAILCC情報を外部プリンター・モジュールに渡すために、ジョブ実行の完了後に保存しています。その情報を使用するかどうかは、外部プリンター・モジュールの必要に応じて異なります。

- 例

以下は、Eメールのカーボンコピー(CC)宛先のメール・アドレスを指定する例です。

```
//OUT1 OUTPUT MAILCC='robert@company.com'  
//OUT DD SYSOUT=J, OUTPUT=*.OUT1
```

2.12.35. MAILFILE

OpenFrameでは構文解析のみサポートします。

- 構文

```
MAILFILCC = 値
```

2.12.36. MAILFROM

Eメールの送信側の名前または他のIDを指定します。

- 構文

```
MAILFROM = 値
```

項目	説明
値	Eメールの送信側の名前または他のIDを指定します。最大60バイトまでの文字列で指定します。

- 注意事項

OpenFrameでは、印刷出力のためのMAILFROM情報を外部プリンター・モジュールに渡すために、ジョブ実行の完了後に保存しています。その情報を使用するかどうかは、外部プリンター・モジュールの必要に応じて異なります。

- 例

以下は、Eメールの送信側のユーザーIDを指定する例です。

```
//OUT1 OUTPUT MAILFROM='Young R. Sender '  
//OUT DD SYSOUT=K, OUTPUT=*.OUT1
```

2.12.37. MAILTO

Eメールの受信側の1つ以上のEメール・アドレスを指定します。

- 構文

```
MAILTO = (メールアドレス[,メールアドレス]...)
```

項目	説明
メールアドレス	Eメールの受信側のEメール・アドレスを指定します。最大長は60バイトまでで、10個まで指定できます。

- 注意事項

OpenFrameでは、印刷出力のためのMAILTO情報を外部プリンター・モジュールに渡すために、ジョブ実行の完了後に保存しています。その情報を使用するかどうかは、外部プリンター・モジュールの必要に応じて異なります。

- 例

以下は、SYSOUTデータセットを受信するメール・アドレスを指定する例です。

```
//OUT1 OUTPUT MAILTO='hong@companyd.com '  
//OUT DD SYSOUT=T, OUTPUT=*.OUT1
```

2.12.38. MODIFY

SYSOUTデータセットのコピー修飾モジュール名と、CHARSオペランドで指定するテーブルのうち使用するテーブルの順序番号を指定します。

- 構文

```
MODIFY = モジュール名  
          ([モジュール名][, テーブル番号])
```

項目	説明
モジュール名	SYSOUTデータセットのコピー修飾モジュール名を1～4文字の記号名で指定します。
テーブル番号	SYSOUTデータセットのCHARSオペランドで指定するテーブルのうち使用するテーブルの順序番号(0～3)を指定します。

- 注意事項

OpenFrameでは、印刷出力のためのMODIFY情報を外部プリンター・モジュールに渡すために、ジョブ実行の完了後に保存しています。その情報を使用するかどうかは、外部プリンター・モジュールの必要に応じて異なります。

- 例

以下は、コピー修飾モジュール名にEDITを指定し、CHARSで指定したテーブルのうちTBL2を指定する例です。

```
//OUT1 OUTPUT COPIES=(3,3),CHARS=(TBL1,TBL2),MODIFY=(EDIT,1)
//OUT DD SYSOUT=J,OUTPUT=*.OUT1
```

2.12.39. NAME

SYSOUTデータセットと関連する名前を指定します。

- 構文

```
NAME = 名前
```

項目	説明
名前	SYSOUTデータセットと関連する名前を指定します。最大60バイトまでの文字列で指定します。

- 注意事項

OpenFrameでは、印刷出力のためのNAME情報を外部プリンター・モジュールに渡すために、ジョブ実行の完了後に保存しています。その情報を使用するかどうかは、外部プリンター・モジュールの必要に応じて異なります。

- 例

以下は、SYSOUTデータセットと関連する名前を指定する例です。

```
//OUT1 OUTPUT NAME='K. Smith'
//OUT DD SYSOUT=D,OUTPUT=*.OUT1
```


2.12.40. NOTIFY

OpenFrameでは構文解析のみサポートします。

- 構文

```
NOTIFY = {nodename.userid}
        {userid      }
        {[node1.]userid1,[node2.]userid2,...)}
```

項目	説明
nodename	8文字以内の記号名で指定します。
userid	8文字以内の記号名で指定します。

2.12.41. OFFSETXB

OpenFrameでは構文解析のみサポートします。

- 構文

```
OFFSETXB = 値
```

2.12.42. OFFSETXF

OpenFrameでは構文解析のみサポートします。

- 構文

```
OFFSETXF = 値
```

2.12.43. OFFSETYB

OpenFrameでは構文解析のみサポートします。

- 構文

```
OFFSETYB = 値
```

2.12.44. OFFSETYF

OpenFrameでは構文解析のみサポートします。

- 構文

```
OFFSETYF = 値
```

2.12.45. OUTBIN

OpenFrameでは構文解析のみサポートします。

- 構文

```
OUTBIN = 値
```

項目	説明
値	1～65535の符号なし整数を指定します。

2.12.46. OUTDISP

SYSOUTデータセットで作成されたOUTPUTの処理方法を指定します。

- 構文

```
OUTDISP = ([WRITE][,WRITE])  
           [HOLD] [,HOLD])  
           [KEEP] [,KEEP])  
           [LEAVE][,LEAVE])  
           [PURGE][,PURGE])  
           [,]
```

– パラメータ1

ジョブが正常終了した場合（ABENDではない場合）の、SYSOUTデータセットで作成されたOUTPUTの処理方法を指定します。

項目	説明
WRITE	OUTPUTがスケジューリングされて印刷される必要があるという意味です。プリンター・ソリューションを呼び出した後、OUTPUTが削除されます。
HOLD	ユーザーがデプロイするまで処理を待機します。
KEEP	WRITEと同じです。ただし、出力作業が終了後に削除せず、LEAVE状態で残っています。
LEAVE	HOLDと同じです。ただし、ユーザーがデプロイするとKEEP状態になります。
PURGE	OUTPUTQからパージします。

– パラメータ2

ジョブが異常終了(ABEND)した場合の、SYSOUTデータセットで作成されたOUTPUTの処理方法を指定します。各項目の説明は、パラメータ 2 と同様です。

参考

パラメータ2が省略された場合、パラメータ1の値が使用されます。パラメータ2が指定されたがパラメータ1が省略された場合、パラメータ1の値はWRITEになります。

- 注意事項
 - OUTDISPが省略された場合、OpenFrame環境設定の**tjes**サブジェクト、**OUTCLASS**セクションに各クラスごとに設定されたキーのVALUE項目の値が使用されます。クラスが設定されていない場合、OUTDISPはパーシされます。tjesサブジェクトの詳細については、『OpenFrame 環境設定ガイド』を参照してください。
 - OUTPUTのクラスは、DD文のSYSOUTオペランドを介して指定されます。

- 例
- 以下は、OUTDISPを指定する例です。

```
//OUT1 OUTPUT OUTDISP=(WRITE,PURGE)
//OUT DD SYSOUT=J,OUTPUT=*.OUT1
```

2.12.47. OVERLAYB

印刷する用紙の背面に置かれるオーバーレイ名を指定します。

- 構文

OVERLAYB = 値

項目	説明
値	8文字以内の記号名を指定します。

- 注意事項
- OpenFrameでは、印刷出力のためのOVERLAYB情報を外部プリンター・モジュールに渡すために、ジョブ実行の完了後に保存しています。その情報を使用するかどうかは、外部プリンター・モジュールの必要に応じて異なります。
- 例
- 以下は、オーバーレイ名を 'MYOVLY'に指定する例です。

```
//OUT1 OUTPUT OVERLAYB=MYOVLY
//OUT DD SYSOUT=J,OUTPUT=*.OUT1
```

2.12.48. OVERLAYF

印刷する用紙の前面に置かれるオーバーレイ名を指定します。

- 構文

```
OVERLAYF = 値
```

項目	説明
値	8文字以内の記号名を指定します。

- 注意事項

OpenFrameでは、印刷出力のためのOVERLAYF情報を外部プリンター・モジュールに渡すために、ジョブ実行の完了後に保存しています。その情報を使用するかどうかは、外部プリンター・モジュールの必要に応じて異なります。

- 例

以下は、オーバーレイ名を 'MYOVLY' に指定する例です。

```
//OUT1 OUTPUT OVERLAYF=MYOVLY  
//OUT DD SYSOUT=J, OUTPUT=*.OUT1
```

2.12.49. OVFL

OpenFrameでは構文解析のみサポートします。

- 構文

```
OVFL = {ON | OFF}
```

2.12.50. PAGEDEF

PAGEDEF名を指定します。

- 構文

```
PAGEDEF = メンバー
```

項目	説明
メンバー	PAGEDEF名を1～6文字(ピリオド(.)を除く)の記号名で指定します。

- 注意事項

OpenFrameでは、印刷出力のためのPAGEDEF情報を外部プリンター・モジュールに渡すために、ジョブ実行の完了後に保存しています。その情報を使用するかどうかは、外部プリンター・モジュールの必要に応じて異なります。

- 例

以下は、PAGEDEFのメンバーをSSPGEと指定する例です。

```
//OUT1 OUTPUT PAGEDEF=SSPGE
//OUT DD SYSOUT=A, OUTPUT=* .OUT1
```

2.12.51. PIMSG

OpenFrameでは構文解析のみサポートします。

- 構文

```
PIMSG = {YES | NO}
        {(YES[,msg-count])}
        {(NO[,msg-count]) }
```

項目	説明
msg-count	0～999の符号なし整数を指定します。

2.12.52. PORTNO

OpenFrameでは構文解析のみサポートします。

- 構文

```
PORTNO = 値
```

項目	説明
msg-count	1～65535の符号なし整数を指定します。

2.12.53. PRMODE

OpenFrameでは構文解析のみサポートします。

- 構文

```
PRMODE = 値
```

項目	説明
値	8文字以内(ピリオド(.))を除く)の記号名を指定します。

2.12.54. PRTATTRS

OpenFrameでは構文解析のみサポートします。

- 構文

```
PRTATTRS = 値
```

2.12.55. PRTERORR

OpenFrameでは構文解析のみサポートします。

- 構文

```
PRTERORR = {DEFAULT | QUIT | HOLD}
```

2.12.56. PRTOPTNS

OpenFrameでは構文解析のみサポートします。

- 構文

```
PRTOPTNS = 値
```

項目	説明
値	16文字以内(ピリオド(.))を除く)の記号名を指定します。

2.12.57. PRTQUEUE

OpenFrameでは構文解析のみサポートします。

- 構文

```
PRTQUEUE = 値
```

項目	説明
値	127文字以内の記号名を指定します。

2.12.58. PRTY

OpenFrameでは構文解析のみサポートします。

- 構文

```
PRTY = 値
```

項目	説明
値	3文字の記号名を指定します。

2.12.59. REPLYTO

Eメールの受信側が応答できるEメール・アドレスを指定します。

- 構文

```
REPLYTO = メールアドレス
```

項目	説明
メールアドレス	Eメールの受信側が応答できるEメール・アドレスを指定します。最大60バイトの文字列で指定します。

- 注意事項

OpenFrameでは、印刷出力のためのREPLYTO情報を外部プリンター・モジュールに渡すために、ジョブ実行の完了後に保存しています。その情報を使用するかどうかは、外部プリンター・モジュールの必要に応じて異なります。

- 例

以下は、受信側が応答できるEメール・アドレスを指定する例です。

```
//OUT1 OUTPUT REPLYTO='reply@companye.net'  
//OUT DD SYSOUT=*,OUTPUT=*.OUT1
```

2.12.60. RESFMT

OpenFrameでは構文解析のみサポートします。

- 構文

```
RESFMT = {P240 | P300}
```

2.12.61. RETAINS

正常に送信されたデータ・セットを保存する時間を 指定します。

- 構文

```
RETAINS = {値}
```

項目	説明
値	記号名を指定します。

- 注意事項

OpenFrameでは、印刷出力のためのRETAINS情報を外部プリンター・モジュールに渡すために、ジョブ実行の完了後に保存しています。その情報を使用するかどうかは、外部プリンター・モジュールの必要に応じて異なります。

- 例

以下は、SYSOUTデータセットの保持時間を1時間に指定する例です。

```
//OUT1 OUTPUT RETAINS='0001:00:00'  
//OUT DD SYSOUT=P,OUTPUT=*.OUT1
```

2.12.62. RETAINF

OpenFrameでは構文解析のみサポートします。

- 構文

```
RETAINF = {hhhh:mm:ss | FOREVER}
```

項目	説明
hhhh	0~9999の符号なし整数を指定します。
mm	0~59の符号なし整数を指定します。
ss	0~99の符号なし整数を指定します。

2.12.63. RETRYL

OpenFrameでは構文解析のみサポートします。

- 構文

```
RETRYL = 値
```


項目	説明
値	0～32767の符号なし整数を指定します。

2.12.64. RETRYT

OpenFrameでは構文解析のみサポートします。

- 構文

```
RETRYT = hhhh:mm:ss
```

項目	説明
hhhh	0～9999の符号なし整数を指定します。
mm	0～59の符号なし整数を指定します。
ss	0～99の符号なし整数を指定します。

2.12.65. ROOM

SYSOUTデータセットの出力の分離ページに部屋IDを指定します。

- 構文

```
ROOM = room ID
```

項目	説明
room ID	部署IDを最大60バイトの文字列で指定します。

- 注意事項

OpenFrameでは、印刷出力のためのROOM情報を外部プリンター・モジュールに渡すために、ジョブ実行の完了後に保存しています。その情報を使用するかどうかは、外部プリンター・モジュールの必要に応じて異なります。

- 例

以下は、SYSOUTデータセットの部署IDを 'Main Hall'に指定する例です。

```
//OUT1 OUTPUT ROOM='Main Hall'
//OUT DD SYSOUT=Y,OUTPUT=*.OUT1
```

2.12.66. SYSAREA

OpenFrameでは構文解析のみサポートします。

- 構文

```
SYSAREA = {YES | Y | NO | N}
```

2.12.67. THRESHLD

OpenFrameでは構文解析のみサポートします。

- 構文

```
THRESHLD = 値
```

項目	説明
値	1～999999999の符号なし整数を指定します。

2.12.68. TITLE

SYSOUTデータセットの出力の分離ページに印刷する説明を指定します。

- 構文

```
TITLE = 値
```

項目	説明
値	分離ページに印刷する説明を最大60バイトの文字列で指定します。

- 注意事項

OpenFrameでは、印刷出力のためのTITLE情報を外部プリンター・モジュールに渡すために、ジョブ実行の完了後に保存しています。その情報を使用するかどうかは、外部プリンター・モジュールの必要に応じて異なります。

- 例

以下は、SYSOUTデータセットの出力の分離ページに印刷する値を 'ANNUAL REPORT'に指定する例です。

```
//OUT1 OUTPUT TITLE='ANNUAL REPORT'  
//OUT DD SYSOUT=0, OUTPUT=*.OUT1
```

2.12.69. TRC

SYSOUTデータセット内の各出力行の論理レコードにテーブル参照文字(TRC)コードが含まれるかどうかを指定します。

- 構文

```
TRC = {YES | Y | NO | N}
```

項目	説明
YES(Y) NO(N)	テーブル参照文字コードが含まれるかどうかをYES(Y)またはNO(N)に指定します。

- 注意事項

OpenFrameでは、印刷出力のためのTRC情報を外部プリンター・モジュールに渡すために、ジョブ実行の完了後に保存しています。その情報を使用するかどうかは、外部プリンター・モジュールの必要に応じて異なります。

- 例

以下は、SYSOUTデータセット内の各出力行の論理レコードにテーブル参照文字コードが含まれないことを指定する例です。

```
//OUT1 OUTPUT TRC=NO  
//OUT DD SYSOUT=W, OUTPUT=*.OUT1
```

2.12.70. UCS

汎用文字セット(UCS)を指定します。

- 構文

```
UCS = 値
```

項目	説明
値	4文字以内の記号名を指定します。

- 注意事項

OpenFrameでは、印刷出力のためのUCS情報を外部プリンター・モジュールに渡すために、ジョブ実行の完了後に保存しています。その情報を使用するかどうかは、外部プリンター・モジュールの必要に応じて異なります。

- 例

以下は、汎用文字セットをA11に指定する例です。

```
//OUT1 OUTPUT UCS=A11
//OUT DD SYSOUT=N, OUTPUT=*.OUT1
```

2.12.71. USERDATA

ユーザー定義データを指定します。

- 構文

```
USERDATA = (値[, 値]...)
```

項目	説明
項目	ユーザー定義データを最大60バイトの文字列で指定します。16個まで指定できます。

- 注意事項

OpenFrameでは、印刷出力のためのUSERDATA情報を外部プリンター・モジュールに渡すために、ジョブ実行の完了後に保存しています。その情報を使用するかどうかは、外部プリンター・モジュールの必要に応じて異なります。

- 例

以下は、USERDATAを指定する例です。

```
//OUT1 OUTPUT USERDATA=('Installation data', 'USERKEY1=User', 'ABC')
//OUT DD SYSOUT=V, OUTPUT=*.OUT1
```

2.12.72. USERLIB

USERLIBデータセット名を指定します。

- 構文

```
USERLIB = {データセット名}
          {(データセット名1, データセット名2, ... データセット名8)}
```

項目	説明
データセット名	USERLIBで使用されるデータセット名を指定します。 データセット名は、最大44文字まで指定できます。データセット名の形式については、『OpenFrame データセットガイド』を参照してください。

- 注意事項

OpenFrameでは、印刷出力のためのUSERLIB情報を外部プリンター・モジュールに渡すために、ジョブ実行の完了後に保存しています。その情報を使用するかどうかは、外部プリンター・モジュールの必要に応じて異なります。

- 例

以下は、TMAX.DATASETに指定する例です。

```
//OUT1 OUTPUT FORMS=5, FORMDEF=JJPR1, USERLIB=TMAX.DATASET
//OUT DD SYSOUT=A, OUTPUT=*.OUT1
```

2.12.73. USERPATH

OpenFrameでは構文解析のみサポートします。

- 構文

```
USERPATH = 値
```

2.12.74. WRITER

外部プリンター・モジュールを指定します。

- 構文

```
WRITER = 名前
```

項目	説明
名前	外部プリンター・モジュールを1～8文字の記号名で指定します。

- 注意事項

OpenFrameでは、印刷出力のためのWRITER情報を外部プリンター・モジュールに渡すために、ジョブ実行の完了後に保存しています。その情報を使用するかどうかは、外部プリンター・モジュールの必要に応じて異なります。

- 例

以下は、WRITERをMYPGMに指定する例です。

```
//OUT1 OUTPUT WRITER=MYPGM
//OUT DD SYSOUT=A, OUTPUT=*.OUT1
```

2.13. PEND文

プロシージャの終わりを示します。入カストリーム・プロシージャでは最後に記述する必要がありますが、カタログ・プロシージャでは最後に記述しなくてもかまいません。

- 構文

```
//[PEND名]△1PEND△[コメント]
```

項目	説明
[PEND名]	名前を指定します。「//」に続いて3桁目から始める必要があります。名前は省略できます。
PEND	オペレーションを記述する位置であり、名前の後ろに1つ以上の空白を入れて「PEND」と記述します。名前を省略した場合は、「//」の後ろに空白なしで指定できます。
[コメント]	PENDの後ろに1つ以上の空白を入れて記述します。

2.14. PROC文

プロシージャの始まりを意味します。プロシージャには以下の2タイプがあります。

区分	説明
入カストリーム・プロシージャ	入力されたジョブのストリームに指定されているため、該当するジョブでのみ使用できる一時的なプロシージャです。 PROC文は、JOB文の後から次のジョブが出てくるまで、どの位置にでも記述できます。入カストリーム・プロシージャの場合、PEND文を使用してプロシージャの終わりを必ず記述します
カタログ・プロシージャ	カタログに登録されており、どのジョブでも呼び出して使用できるプロシージャです。

- 構文

```
//[PROC名]△1PROC△1[記号パラメータ=[値]]...△1[コメント]
```

項目	説明
PROC名	PROC名を指定します。「//」に続いて3桁目から始める必要があります。 PROC名は、入カストリーム・プロシージャでは必ず記述する必要があり、英数字で指定します。カタログ・プロシージャでは記述しなくてもかまいません。カタログ・プロシージャをEXEC文で呼び出す際に使用されるプロシージャ名は、カタログ・プロシージャが登録されているメンバー名です。
PROC	オペレーションを指定します。PROC名の後ろに1つ以上の空白を入れて「PROC」と記述します。PROC名を省略した場合は、「//」の後ろに1つ以上の空白を入れて記述します。

項目	説明
[記号パラメータ=[値]]	PROCの後ろに1つ以上の空白を入れて記号パラメータを記述します。記号パラメータはEXEC文のオペランドと同じものは使用できません。記号パラメータの詳細については、 記号パラメータ を参照してください。
[コメント]	記号パラメータの後ろに1つ以上の空白を入れて指定します。記号パラメータを1つも指定しなかった場合、コメントは記述できません。

記号パラメータ

プロシージャで使用する記号パラメータの値を1～8の記号名で指定します。記号パラメータのみ定義し、値を指定していない場合、システムはNULL値が指定されたと判断します。プロシージャを呼び出すEXEC文で記号パラメータが指定されている場合は、プロシージャで同じ記号パラメータはオーバーライドされ、異なる記号パラメータは追加されます。

記号パラメータは、プロシージャ内の制御文で以下のように使用されます。

- 記号パラメータの前にアンパサンド(&)を付けて使用します。

システムは、「&記号パラメータ」形式のものを、対応する記号パラメータの値に置き換えます。

- 「&記号パラメータ」で記号パラメータの終了は以下のように指定できます。

- 空白、ピリオド(.)、コンマ(,)、括弧、「\n」、単一引用符(')がある場合、その前までを記号パラメータとして認識します。

以下の例では、「&ABC」の次にコンマ(,)があるため、「ABC」を記号パラメータとして認識します。

```
//STEP1 EXEC PGM=&ABC, PARM=8
```

- ピリオド(.)を使って記号パラメータの終了を示す場合、このピリオドまでが記号パラメータの値に置き換えられます。

以下の例では、「&ABC」の次にピリオドがあるため、「&ABC.」がTMAXに置き換えられ、PGMの値はTMAXSOFTとなります。

```
//PROC PROC ABC=TMAX
//STEP1 EXEC PGM=&ABC.SOFT, PARM=8
```

- 置き換えできる記号パラメータが複数ある場合、最も長い名前の記号パラメータが置き換えられます。

以下の例では、「&ABC」と「&ABCD」が両方とも記号パラメータですが、長さがより長い「ABCD」を置き換え、PGMの値は「tmaxEFG」となります。

```
//PROC PROC ABC=TMAX,ABCD=tmax
//STEP1 EXEC PGM=&ABCDEFG,PARM=8
```

記号パラメータは、以下のような場合は置き換えられません。

- 「&&記号パラメータ」のようにアンパサンド(&)が2つある場合は置き換えられません。「&&&記号パラメータ」の場合は、「&&置き換え値」に置き換えます。
- 置き換える記号パラメータが定義されていない場合は置き換えません。

2.15. SET文

JCLで使用する記号パラメータの値を指定します。

```
//[名前]△1SET△1記号パラメータ=値[, 記号パラメータ=値]...△1[コメント]
```

項目	説明
名前	名前を指定します。「//」に続いて3桁目から始める必要があります。名前は省略できます。
SET	オペレーションを指定します。名前の後ろに1つ以上の空白を入れて「SET」と記述します。名前を省略する場合、「//」の後ろに1つ以上の空白を入れて記述します。
記号パラメータ=値[, 記号パラメータ=値]...	SETの後ろに1つ以上の空白を入れて、記号パラメータを記述します。 記号パラメータの詳細については、 記号パラメータ を参照してください。
[コメント]	記号パラメータの後ろに1つ以上の空白を入れて指定します。

記号パラメータ

JCLの後に呼び出されるプロシージャで使用する記号パラメータの値(ピリオド(.)を除く記号名)を指定します。記号パラメータのみ定義し、値を指定していない場合、システムはNULL値が指定されたものと判断します。

SET文で指定された記号パラメータは、その後のEXEC文や呼び出すプロシージャのPROC文で同じ記号パラメータを指定すると、該当するプロシージャではEXEC文やPROC文で指定した値が使用されます。

記号パラメータはJCL文で以下のように使用されます。

- 記号パラメータの前にアンパサンド(&)をつけて使用します。
システムは、「&記号パラメータ」形式になっているものを該当する記号パラメータの値に置換します。

- 「&記号パラメータ」で記号パラメータの終了は、以下のように指定できます。

- 空白、ピリオド(.)、コンマ(,), 括弧、[「\n」、単一引用符(')がある場合、その前までを記号パラメータと認識します。

以下の例では、「&ABC」の次にコンマ(,)があるため、「ABC」を記号パラメータと認識します。

```
//STEP1 EXEC PGM=&ABC,PARM=8
```

- ピリオド(.)で記号パラメータの終了を区別する場合、このピリオドまでが記号パラメータの値に置換されます。

以下の例では、「&ABC」の次にピリオドがあるため、「&ABC.」がTMAXに置換され、PGMの値はTMAXSOFTになります。

```
//          SET  ABC=TMAX
//STEP1 EXEC PGM=&ABC.SOFT,PARM=8
```

- 置換できる記号パラメータが複数ある場合、最も長い名前の記号パラメータに置換します。

以下の例では、「&ABC」と「&ABCD」が両方とも記号パラメータですが、より長い「ABCD」を置換し、PGMの値は「tmaxEFG」となります。

```
//          SET  ABC=TMAX,ABCD=tmax
//STEP1 EXEC PGM=&ABCDEFG,PARM=8
```

- SET文とEXEC文（またはPROC文）で記号パラメータを使用する場合、以下のように処理されます。

- SET文で指定したパラメータは、プロシージャ内ではEXEC文やPROC文で同じパラメータを指定していない場合に使用されます。同じパラメータが指定されている場合、プロシージャ内ではEXEC文やPROC文で指定されたパラメータの値が使用されます。

- JCLでは、SET文で指定したパラメータはSET文で再指定するまでは継続して使用されます。

- プロシージャ内でSET文で指定したパラメータは、該当プロシージャ内でのみ使用されます。

以下の例では、「&ABC」の値がどのように置換されるかを示しています。

```
JCL
//          SET  ABC=1
//STEP1 EXEC PROCTEST,ABC=2      -- In PROCTEST PSTEP1 step, PARM parameter's
value is 2
//STEP2 EXEC PGM=TEST,PARM=&ABC  -- PARM parameter's value is 1
//STEP3 EXEC PROCTEST           -- In PROCTEST PSTEP1 step, PARM parameter's
value is 1
```

```
PROCTEST
//PROCTEST PROC
//PSTEP1   EXEC PGM=TEST,PARM=&ABC
```

記号パラメータは、以下のような場合、置換されません。

- 「&&記号パラメータ」形式でアンパサンド(&)2個が続いている場合、置換されません。「&&&記号パラメータ」形式の場合、「&&置換値」に置換されます。
- 置換しようとする記号パラメータが定義されていない場合、置換されません。

2.16. XMIT文

他のノードにデータを送信する構文です。OpenFrameでは、構文エラーを避けるために、XMIT文の後に出力されるインストリーム・データまで処理します。

- 構文

```
//[名前]△1XMIT オペランド[,オペランド...]△[* コメント]
```

項目	説明
オペランド	DEST DLM SUBCHARSのいずれかを指定します。

- 例

以下は、XMIT文を使用する例です。

```
//JOB1   JOB
//X1      XMIT DEST=LOCAL
      .
      .
      (records to be transmitted)
      .
/*
```

2.17. 空文

ジョブの最後を示します。空文の後から次のJOB文が現れるまでのデータは無視します。

- 構文

```
//
```

2.18. 段落見出し

インストリーム・データの最後の行のすぐ後に位置し、インストリーム・データの最後を示します。

インストリーム・データの始まりを示すDD文（「DD *」または「DD DATA」）が存在しない状態でインストリーム・データ（「//」または「/*」で始まらない構文）が現れた場合、システムは以下の構文を自動的に生成し、SYSIN DDのインストリーム・データに含まれるようにします。

```
//SYSIN DD *
```

- 構文

```
/*[コメント]
```

項目	説明
[コメント]	コメントは「/*」に続けて3桁目から指定します。 コメントを3桁目に記述すると、「/*PRIORITY」のようにJES2文として認識する可能性があるため、3桁目には空白を入れて記述することをお勧めします。

2.19. コメント文

コメントを記述します。コメント文はどこにでも記述できます。ただし、インストリーム・データの中に記述すると、その位置でインストリーム・データが終了されたと判断するため、インストリーム・データの間には記述しないでください。また、その以降のインストリーム・データは新しいSYSIN DDとして登録されます。

OpenFrameでは、「##」で始まる構文もコメントとして処理します。ただし、インストリーム・データで「##」で始まるものはデータとして処理されます。

- 構文

```
/*[コメント]
```

項目	説明
[コメント]	コメントは「/*」に続いて4桁目から記述します。コメントを複数行に亘って記述する場合は、常に先頭の3桁は「/*」で始まる必要があります。

第3章 JES2 JCL文

本章では、JES2 JCL文と各オペランドについて説明します。

3.1. 概要

以下は、JES2 JCL制御文の一覧です。

JES2 JCL文	説明
JES2コマンド文	JES2コマンドを記述します。
JOBPARM文	ジョブの制御情報を記述します。
MESSAGE文	OpenFrameでは構文解析のみサポートします。
NETACCT文	OpenFrameでは構文解析のみサポートします。
NOTIFY文	OpenFrameでは構文解析のみサポートします。
OUTPUT文	OpenFrameでは構文解析のみサポートします。
PRIORITY文	ジョブの優先順位を指定します。
ROUTE文	OpenFrameでは構文解析のみサポートします。
SETUP文	OpenFrameでは構文解析のみサポートします。
SIGNOFF文	OpenFrameでは構文解析のみサポートします。
SIGNON文	OpenFrameでは構文解析のみサポートします。
XEQ文	OpenFrameでは構文解析のみサポートします。
XMIT文	OpenFrameでは構文解析のみサポートします。

3.2. JES2コマンド文

JES2コマンドを記述します。JES2コマンド文は、最初のJOB文が現れる前に記述します。それ以外の位置に記述されたJES2コマンド文は使用されません。

JES2コマンド文を使用して実行されたコマンドの成功可否は、JES2コマンド文の後に記述されたジョブのサブミットあるいは実行に対して影響を与えません。

JOB文なしでJES2コマンド文を記述することもできます。JES2コマンドのみ存在するJCLを実行する場合、OpenFrame Batchでは一時的にJOBNAME(TJESSUBM)を割り当ててJES2コマンドを実行し、ジョブはDONEで終了します。

JES2コマンドの実行結果は、OpenFrame環境設定の**ofsys**サブジェクト、**DIRECTORY**セクションの**LOG_DIR**キーの**VALUE**項目に指定されたディレクトリのサブディレクトリjob内に「submit_YYYYMMSS.log」形式で記録されます。

参考

1. 実行結果のみsubmig.logに記録されるため、JES2コマンドの実行による追加メッセージは、各サブシステムのログを確認する必要があります。JCL構文エラーが発生した場合は、obmjmsvrサーバー・ログとtjesmgrログに出力されます。
 2. ofsysサブジェクトの詳細については、『OpenFrame 環境設定ガイド』を参照してください。
-

以下は、JES2コマンド文についての説明です。

● 構文

```
/*\コマンド△°オペランド
```

項目	説明
コマンド	オペレーションを記述します。「/*」または「/*\$」に続けて4桁目にコマンドを記述します。サポートされるコマンドについては、 コマンド を参照してください。
オペランド	オペレーションの後ろに1つ以上の空白またはコンマ(,)を入れて記述します。オペランドはコマンドによって異なります。

コマンド

以下は、前述したコマンド項目についての説明です。

● VS

入力したOSコマンドを実行します。

```
/*\VS[ ], 'OS command'
```

以下は、OSコマンドについての説明です。

コマンド	説明
S	入力したプロシージャをサブミットします。
/DBR, /STA(/START)	OSIシステムを通じて該当するコマンドを実行します。

参考

上述したコマンド以外は、OpenFrameではサポートしていません。

3.2.1. S

入力したプロシージャをサブミットします。

- 構文

```
s プロシージャ名, 記号 パラメータ=値[, 記号 パラメータ=値]...
```

項目	説明
プロシージャ名	サブミットするプロシージャ名を指定します。
記号パラメータ	PROC文の記号パラメータ を参照してください。

3.2.2. /DBR, /STA(/START)

OSIシステムを通じて該当するコマンドを実行します。

- 構文

```
/DBR オペランド  
/STA(/START) オペランド
```

項目	説明
オペランド	コマンドで使用するオペランドを指定します

参考

各コマンドの詳細については、『OpenFrame OSIコマンドリファレンスガイド』を参照してください。

3.3. JOBPARM文

ジョブの制御情報を記述します。JOBPARM文はJOB文の後に記述します。オペランドが重複する場合、最初に記述された値が使用されます。

- 構文

```
/*JOBPARM△1キーワードオペランド[,キーワードオペランド]...△1[コメント]
```

項目	説明
JOBPARM	オペレーションを記述します。「/*」に続けて3桁目に「JOBPARM」と記述します。
キーワードオペランド [,キーワードオペランド]	オペレーションの後ろに1つ以上の空白を入れてキーワード・オペランドを記述します。オペランドの順序は関係ありません。詳細については、 オペランド と各オペランド節を参照してください。
[コメント]	オペランドの後ろに1つ以上の空白を入れて記述します。コメントは71桁目まで記述できます。

オペランド

以下は、オペランドについての説明です。各オペランドについての詳しい内容は、該当する節を参照してください。

- キーワード・オペランド

項目	説明
BURST/B	OpenFrameでは構文解析のみサポートします。
BYTES/M	OpenFrameでは構文解析のみサポートします。
CARDS/C	OpenFrameでは構文解析のみサポートします。
COPIES/N	OpenFrameでは構文解析のみサポートします。
FORMS/F	OpenFrameでは構文解析のみサポートします。
LINECT/K	OpenFrameでは構文解析のみサポートします。
LINES/L	最大の出力行数を指定します。
PAGES/G	OpenFrameでは構文解析のみサポートします。
PROCLIB/P	ステップで実行するプロシージャを検索するために使われます。
RESTART/E	OpenFrameでは構文解析のみサポートします。
ROOM/R	OpenFrameでは構文解析のみサポートします。
SYSAFF/S	OpenFrameでは構文解析のみサポートします。
TIME/T	JOBの実行時間が指定された値(分)を経過すると、コンソール画面に警告メッセージを表示します。

3.3.1. BURST/B

OpenFrameでは構文解析のみサポートします。

- 構文

```
BURST | B = 値
```

3.3.2. BYTES/M

OpenFrameでは構文解析のみサポートします。

- 構文

```
BYTES | M = 値
```

3.3.3. CARDS/C

OpenFrameでは構文解析のみサポートします。

- 構文

```
CARDS | C = 値
```

3.3.4. COPIES/N

OpenFrameでは構文解析のみサポートします。

- 構文

```
COPIES | N = 値
```

3.3.5. FORMS/F

OpenFrameでは構文解析のみサポートします。

- 構文

```
FORMS | F = 値
```

3.3.6. LINECT/K

OpenFrameでは構文解析のみサポートします。

- 構文

```
LINECT | K= 値
```

3.3.7. LINES/L

最大の出力行数を指定します。

- 構文

```
LINES | L = 値
```

3.3.8. PAGES/G

OpenFrameでは構文解析のみサポートします。

- 構文

```
PAGES | G = 値
```

3.3.9. PROCLIB/P

ステップで実行するプロシージャを検索するために使用されます。PROCLIBが指定されると、システムはPROCLIBを使用してステップで実行するプロシージャを検索します。

OpenFrame環境設定の**tjes**サブジェクト、**PROCLIB**セクションの**{ddname}**キーのVALUE項目に指定されたカタログ・プロシージャ・ライブラリのDD名でデータセット・リストを取得します。取得したデータセット・リストのメンバーからプロシージャを優先的に検索します。

ステップで実行するプロシージャの検索順は以下のとおりです。5項まで検索しても対象が見つからない場合はフラッシュが発生します。

1. 入カストリーム・プロシージャの中から検索します。
2. JCLLIB文が記述されていると、JCLLIB文に記述されたメンバーを利用して検索します。
3. JES2 JCL文のJOBPARM文にPROCLIB={ddname}オペランドが指定されている場合は、OpenFrame環境設定の**tjes**サブジェクト、**PROCLIB**セクションの**{ddname}**キーのVALUE項目に指定されているデータセットのメンバーから検索します。
4. OpenFrame環境設定の**tjes**サブジェクト、**PROCLIB**セクションの**PROC00**キーのVALUE項目に指定されているデータセットのメンバーから検索します。
5. SYS1.PROCLIBのメンバーから検索します。

参考

tjesサブジェクトの詳細については、『OpenFrame 環境設定ガイド』を参照してください。

以下は、PROCLIBオペランドについての説明です。

- 構文

PROCLIB | P = {ddname}

項目	説明
{ddname}	カタログ・プロシージャを検索するときに参照するライブラリ・リストが設定されたDD名を指定します。

- 注意事項

OpenFrame環境設定の**tjes**サブジェクト、**PROCLIB**セクションの**{ddname}**キーのVALUE項目にカタログ・プロシージャ・ライブラリのDDがない場合、そのオペランドは使用されません。

- 例

以下は、JOBPARM文でPROCLIBを使用する例です。PROCLIBをPROC00に指定したため、STEP1のPROC01は、OpenFrame環境設定の**tjes**サブジェクト、**PROCLIB**セクションの**PROC00**キーのVALUE項目に指定されているデータセットのメンバーから検索します。

```
//JOB1      JOB
/*JOBPARM  PROCLIB=PROC00
//STEP1    EXEC  PROC01
//SYSOUT   DD    SYSOUT=*
```

3.3.10. RESTART/E

OpenFrameでは構文解析のみサポートします。

- 構文

RESTART | E = 値

3.3.11. ROOM/R

OpenFrameでは構文解析のみサポートします。

- 構文

ROOM | R = 値

3.3.12. SYSAFF/S

OpenFrameでは構文解析のみサポートします。

- 構文

SYSAFF | S = 値

3.3.13. TIME/T

JOBの実行時間が指定された値(分)を経過すると、コンソール画面に警告メッセージを表示します。

- 構文

```
TIME | T = {minutes}
```

項目	説明
{minutes}	コンソール画面に警告メッセージを表示するJOBの経過時間を分単位で指定します。

- 例

指定された時間が経過して表示されたコンソール・メッセージの例です。

```
IEFUTIL01 ** JOBNAME ** JOB EXECUTION TIME EXCEEDED
```

3.4. MESSAGE文

OpenFrameでは構文解析のみサポートします。

- 構文

```
/*MESSAGE△1オペランド
```

3.5. NETACCT文

OpenFrameでは構文解析のみサポートします。

- 構文

```
/*NETACCT△1オペランド
```

3.6. NOTIFY文

ジョブ実行の完了後に通知されたメッセージを、NOTIFYによって指定されたユーザーがtjesmgrでコマンドを実行してメッセージを確認できます。

- 構文

```
/*NOTIFY△1{userid}
```

項目	説明
NOTIFY	オペレーションを記述します。「/*」に続けて3桁目に「NOTIFY」と指定します。

項目	説明
userid	8文字以内の記号名を指定します。

- 注意事項

JOB文とJES JCL文の両方にNOTIFYが指定されている場合は、JOB文のNOTIFYパラメータの値は無視され、JES JCL文のNOTIFYの値が使用されます。

- 例

以下は、USER1ユーザーにメッセージを通知する例です。

```
/*NOTIFY USER1
```

3.7. OUTPUT文

OpenFrameでは構文解析のみサポートします。

- 構文

```
/*OUTPUT△1オペランド
```

3.8. PRIORITY文

ジョブの優先順位を指定します。PRIORITY文はJOB文の前に記述します。

- 構文

```
/*PRIORITY△1優先順位
```

項目	説明
PRIORITY	オペレーションを記述する位置であり、「/*」に続く3桁目に「PRIORITY」と記述します。
優先順位	オペレーションの後ろに1つ以上の空白を入れて1~15の数字を指定します。 優先順位の詳細については、 優先順位 を参照してください。

優先順位

ジョブの優先順位はジョブ・クラスと一緒にスケジューリングに使用され、数字が高いほど優先順位が高くなります。スケジューラでは、同じクラスを有するジョブがある場合、このオペランドの値に応じてどのジョブを先に実行するかを決定します。

同じ優先順位の場合は、先にサブミットされたジョブが実行されます。ジョブの優先順位はエージング・ポリシーに従って時間が経つにつれ高くなります。優先順位は、**tjes**サブジェクト、**SCHEDULING**セクションの **PRTYHIGH**、**PRTYLOW**キーに設定します。

参考

tjesサブジェクトの詳細については、『OpenFrame 環境設定ガイド』を参照してください。

● 構文

優先順位

項目	説明
優先順位	ジョブのスケジューリングに関連する優先順位を1~15の数字で指定します。

● 注意事項

- オペランドを使用するには、OpenFrame環境設定の**tjes**サブジェクト、**SCHEDULING**セクションの**PRTYJECL**キーのVALUE項目をYESに設定する必要があります。
- PRIORITY文に優先順位を指定し、またJOB文にもPRTYオペランドを使用して優先順位を指定した場合は、PRIORITY文の優先順位が適用されます。

● 例

以下は、優先順位を10に指定した例です。

```
/*PRIORITY 10
//JOB1 JOB CLASS=A
```

以下は、JOB文とPRIORITY文に同時に優先順位を指定した場合、PRIORITY文の優先順位に従う例です。以下の例では、ジョブの優先順位が5に指定されます。

```
/*PRIORITY 5
//JOB1 JOB CLASS=A, PRTY=10
```

3.9. ROUTE文

SYSOUTデータセットの出力先を指定するか、ジョブが実行されるネットワーク・ノードを識別します。現在OpenFrameでは、PRINT文のみサポートしています。

● 構文

```
/*ROUTE△1PRINT△1出力先
```

項目	説明
ROUTE	「/*」に続いて3桁目に「ROUTE」と指定します。
PRINT	SYSOUTデータセットの出力先を指定します。PRINTオペランドの後ろに1つ以上の空白を入力した後指定します。 SYSOUT DD文やOUTPUT文でDESTの指定がない場合にのみ適用されます。これらの指定がある場合、適用の優先順位は以下のようになります。 1. SYSOUT DD文 2. OUTPUT文 3. ROUTE文

3.10. SETUP文

OpenFrameでは構文解析のみサポートします。

- 構文

```
/*SETUP△1オペランド
```

3.11. SIGNOFF文

OpenFrameでは構文解析のみサポートします。

- 構文

```
/*SIGNOFF
```

3.12. SIGNON文

OpenFrameでは構文解析のみサポートします。

- 構文

```
/*SIGNON△1オペランド
```

3.13. XEQ文

OpenFrameでは構文解析のみサポートします。

- 構文

```
/*XEQ△1オペランド
```

3.14. XMIT文

他のノードにデータを送信する構文です。OpenFrameでは構文解析のみサポートします。

構文エラーを避けるために、XMIT文の後に出力されるインストリーム・データまで処理します。

- 構文

```
/*XMIT オペランド△1[DLM=値]
```

項目	説明
オペランド	14文字以内の記号名を指定します。

- 例

以下は、XMIT文を使用する例です。

```
//JOB1 JOB
/*XMIT TMAX DLM=AA
.
.
(records to be transmitted)
.
AA
```


第4章 動的変数制御文

本章では、動的変数制御文と各オペランドについて説明します。

4.1. 概要

OpenFrameでは、2種類の動的変数制御文をサポートします。2種類の制御文と一緒に使用することはできません。

以下は、動的変数制御文の一覧です。

動的変数制御文	説明
%%制御文	JCLで「%%」で始まる構文は、%%動的変数制御文を示します。
OPC制御文	JCLのコメント文の後ろに「%OPC」で始まる構文は、OPC動的変数制御文を示します。

4.2. %%制御文

%%制御文は、JCLで「%%」で始まるすべての変数を示します。JCL構文とは関係なく、インストリーム・データを含むどの位置にも記述できます。変数の前に「%%」が記述されている構文を、OpenFrameでは%%制御文として認識します。

%%制御文が含まれたJCLを実行するには、**textrun**ツールを使用してジョブをサブミットする必要があります。textrunを実行する場合は、**textrun**サブジェクトの**AUTOEDIT**セクションの**USE**キーの値をYESに設定する必要があります。

参考

1. textrunツールの詳細については、『OpenFrame ツールリファレンスガイド』を参照してください。
 2. textrunサブジェクトの詳細については、『OpenFrame 環境設定ガイド』を参照してください。
-

以下は、%%制御文についての説明です。

- 使用方法

```
%%変数名
```

項目	説明
変数名	特定の文字列の前に「%%」が記述されている場合、その構文は%%制御文として扱われます。 変数名は、単純な変数、またはOpenFrameでサポートするオペレーションになります。オペレーションおよびシステム変数の種類は下で説明します。

オペレーション

以下は、変数名に記述できるオペレーションの一覧です。各オペレーションの詳細については、各節で説明します。

オペレーション	説明
SET	変数に値を指定します。
GLOBAL	特定のパスのファイルに記述された変数リストを呼び出します。
IF/ELSE/ENDIF	条件を指定し、条件の結果に従って実行する構文を記述します。

参考

OpenFrameでは、上記のオペレーション以外はサポートされません。

システム変数

以下は、%%制御文で定義しているシステム変数の一覧です。OpenFrameでサポートしている%%制御文のシステム変数は以下のとおりです。

システム変数	説明
TIME	システムの現在の時間を hhmmss形式で出力します。
DAY	システムの現在の日付の日(day)を dd形式で出力します。
ODAY	入力された日付の日(day)を dd形式で出力します。
RDAY	システムの現在の日付の日(day)を dd形式で出力します。
RWDAY	システムの現在の曜日を1桁の数字で出力します。 – 1：日曜日 – 2：月曜日 – 3：火曜日 – 4：水曜日

システム変数	説明
	– 5 : 木曜日 – 6 : 金曜日 – 0 : 土曜日
OJULDAY	入力された日付をユリウス日付形式(nnn)で出力します。
DATE	システムの現在の日付を yymmdd形式で出力します。
\$DATE	システムの現在の日付を yyyyymmdd形式で出力します。
ODATE	入力された日付を yymmdd形式で出力します。
\$ODATE	入力された日付を yyyyymmdd形式で出力します。
\$RDATE	入力された日付を yyyyymmdd形式で出力します。
RDATE	入力された日付を yymmdd形式で出力します。
OMONTH	入力された日付の月(month)を mm形式で出力します。
RMONTH	入力された日付の月(month)を mm形式で出力します。
OYEAR	入力された日付の年度(year)を yy形式で出力します。
\$OYEAR	入力された日付の年度(year)を yyyy形式で出力します。
RYEAR	入力された日付の年度(year)を yy形式で出力します。
\$RYEAR	入力された日付の年度(year)を yyyy形式で出力します。
OCENT	入力された日付の年度(year)の最初の2桁を yy形式で出力します。
BLANKn	n個の空白を出力します。
RN	OpenFrameではエラーを防ぐために構文のみサポートしています。

4.2.1. SET

SET文の後に指定された%%変数に値を指定します。

- 使用方法

```
%%SET %%変数名 = 演算式
```

項目	説明
変数名	SET文の後ろに「%%変数名」形式で指定します。値を代入する変数名を記述します。
演算式	変数名に代入する演算式を指定します。

項目	説明
	整数の変数または演算式を記述することができます。変数に接頭辞の「%%」が付いていない場合は、数字または文字列として認識します。 以下は、演算式でのみ使用できるシステム変数です。 – CALCDATE : 日付を計算して6桁(yymmdd)で出力します。 – \$CALCDTE : 日付を計算して8桁(yyyymmdd)で出力します。 – \$WCALC : 日付を計算して8桁(yyyymmdd)で出力します。 – \$JULIAN : グレゴリオ暦の日付(yyyymmdd)をユリウス暦の日付(yyyyddd)に変更します。 – SUBSTR : 指定した文字列で部分文字列を抽出します。 – PLUS : 前の値と後ろの値を足します。 – MINUS : 前の値から後ろの値を引きます。 演算式には、数字・文字形式の変数、およびシステム指定の変数を記述できます。

● 使用例

- 以下は、SET文で変数A、Bに値を指定する例です。A、Bに代入される値はそれぞれ100、300になります。

```
%%SET %%A = 100
%%SET %%B = %%A + 200
```

- 以下は、SET文で変数C、D、Eに値を指定する例です。入力された日付が「20191231」である場合、C、D、Eに代入される値はそれぞれ「20191226」、「191226」、「191229」になります。

```
%%SET %%C = %%CALCDATE %%$ODATE - 5
%%SET %%D = %%$CALCDTE %%$ODATE - 5
%%SET %%E = %%D %%PLUS 3
```

- 以下は、SET文で変数F、Gに値を指定する例です。変数Fには「OPENFRAME」という文字列が保存されます。変数Gには変数Fに保存された文字列の最初のバイトから4桁の値の「OPEN」が保存されます。

```
%%SET %%F = OPENFRAME
%%SET %%G = %%SUBSTR %%F 1 4
```

4.2.2. GLOBAL

GLOBAL文を使用して、変数リストを保存したファイル呼び出して使用することができます。この機能を使用すると、JCLに同じSET文を記述する必要はなくなります。

- 使用方法

```
%%GLOBAL ファイル名
```

項目	説明
ファイル名	変数リストが定義されたファイルの名前をGLOBAL文の後ろに指定します。

4.2.3. IF/ELSE/ENDIF

条件を指定し、条件の結果に従って実行する構文を記述します。ELSE文はIF文が先に記述された場合にのみ使用することができ、IF文はENDIF文で終了する必要があります。IF文を重複して使用することもできます。

- 使用方法

```
%%IF %%変数名 条件文  
    AAA  
%%ELSE  
    BBB  
%%ENDIF
```

項目	説明
条件文	IF文で指定可能な条件は以下のとおりです。 – EQ : 左辺と右辺が等しい場合に真になります。 – NE : 左辺と右辺が等しくない場合に真になります。 – GT : 左辺が右辺より大きい場合に真になります。 – GE : 左辺が右辺以上の場合に真になります。 – LT : 左辺が右辺より小さい場合に真になります。 – LE : 左辺が右辺以下の場合に真になります。

- 使用例

以下は、IF/ELSE/ENDIF文の使用例です。入力日付が「yyyymmdd」である場合、変数Sには最初の2文字のyyが保存されます。yyの値が20より大きい場合は、変数Xの値が1になり、そうでない場合は0になります。

```

%%SET %%T = %%$ODATE
%%SET %%S = %%SUBSTR 1 2
%%IF %%S GT 20
    %%SET %%X = 1
%%ELSE
    %%SET %%X = 0
%%ENDIF

```

4.3. OPC制御文

OPC制御文は、JCLで「`//*%OPC`」で始まり、JCL構文とは関係なく、どの位置にも記述できます。OPC制御文が含まれたJCLをサブミットするには、**textrun**ツールを使用してジョブをサブミットする必要があります。textrunを実行する場合は、OpenFrame環境設定の**textrun**サブジェクトの**TWS**セクションの**USE**キーのValue項目の値をYESに設定する必要があります。

参考

1. textrunツールの詳細については、『OpenFrame ツールリファレンスガイド』を参照してください。
2. textrunサブジェクトの詳細については、『OpenFrame 環境設定ガイド』を参照してください。

以下は、OPC制御文についての説明です。

● 使用方法

```

/*%OPC  △1オペレーション  △1オペランド

```

項目	説明
オペレーション	「 <code>/*%OPC</code> 」の後ろに1つ以上の空白を入れてオペレーションを記述します。サポートされるオペレーションについては、 オペレーション を参照してください。
オペランド	オペレーションの後ろに1つ以上の空白を入れてオペランドを記述します。オペランドはオペレーションによって異なります。

オペレーション

以下は、上記で説明したオペレーションについての説明です。OpenFrameでサポートしているオペレーションは以下のとおりです。

オペレーション	説明
SCAN	SCAN文の後に記述されたオペレーションのみ動作し、SCAN文の前に記述されたオペレーションは動作しません。
SETFORM	日付変数の形式を指定します。
SETVAR	変数の値を指定します。
BEGIN	ドメインの始めを指定します。
END	ドメインの最後を指定します。

参考

OpenFrameでは、上記で説明していないオペレーションはサポートされません。

4.3.1. SCAN

SCAN文の後に記述されたオペレーションのみ動作し、SCAN文の前に記述されたオペレーションは動作しません。JCLのどの位置にも記述できますが、1つのJCLにSCAN文を2回記述すると、エラーが発生します。

- 使用方法

```
//*%0PC SCAN
```

4.3.2. SETFORM

日付変数の形式を指定します。

- 使用方法

```
//*%0PC SETFORM 日付変数=(文字列形式)
```

項目	説明
日付変数	形式を指定する日付変数を指定します。現在OpenFrameでサポートしている日付変数はOCDATEです。
文字列形式	日付変数に適用する形式を指定します。 現在OpenFrameでサポートしているキーワードは以下のとおりです。以下のキーワード以外の文字はすべて文字列として処理されます。 – CC: 日付の年度4桁のうち前の2桁の数字を示します。必ずYYキーワードと組み合わせて使用される必要があります。 – YY: 日付の年度4桁のうち後ろの2桁の数字を示します。 – MM: 日付の月を示します。

項目	説明
	– DDD: ユリウス日付を示します。文字列の中でD文字が連続して3つ以上出現する場合、DDDキーワードが優先して適用され、残りの文字は文字列かDDキーワードとして処理されます。 – DD: 日付の日を示します。

● 使用例

以下は、SETFORM文でOCDATEの形式を指定する例です。入力された日付が「19980201」である場合、OCDATEの結果は「032011998」になります。

```
//*%0PC SETFORM OCDATE=(DDDDCCYY)
```

4.3.3. SETVAR

変数の値を指定します。現在、OpenFrameでサポートしている形式は次の2つです。SETVAR文を使用する際、日付変数と単位が一致しない場合は、エラーが発生します。

● 使用方法

```
//*%0PC SETVAR 変数=(日付変数 {+|-} nnnTT)
//*%0PC SETVAR 変数=('文字列')
```

項目	説明
変数	変数の名前を指定します。変数名は必ず「T」で始まる必要があります。
日付変数	日付変数を指定します。現在OpenFrameでサポートしている日付変数は、OCDATE、OYYY、OYY、OMM、ODDです。
nnn	0から999までの数字を指定します。
TT	単位を指定します。 現在OpenFrameでサポートしている単位は以下のとおりです。 – YR: 年 – MO: 月 – CD: カレンダーの基準日

● 使用例

以下は、SETVAR文で変数を指定する例です。入力されたOCDATE変数の値が「19980507」の場合、TAAの値は「19990507」になります。

```
//*%0PC SETVAR TAA=(OCDATE+1YR)
```


4.3.4. BEGIN

ドメインの始めを指定します。BEGIN文を使用する場合は、END文とペアで使用する必要があります。BEGIN-ENDドメインは他のBEGIN-ENDドメインと重複したり、入り交じって使用されることができません。COMPパラメータに記述された条件式が真の場合はドメインが動作し、偽の場合はドメインが動作しません。

- 使用方法

```
//%OPC BEGIN ACTION={INCLUDE|EXCLUDE|NOSCAN}, COMP((式1)}.{EQ|NE|GE|GT|LE|LT}).(式2))
```

項目	説明
INCLUDE	ドメインをジョブに含めます。
EXCLUDE	ドメインをジョブから除外します。現在はサポートしていません。指定する場合は、INCLUDEを指定した場合と同様に動作します。
NOSCAN	ドメインに含まれた領域が置換されないようにします。現在はサポートしていません。指定する場合は、INCLUDEを指定した場合と同様に動作します。
EQ	EQ比較演算子。左辺と右辺が等しい場合に真になります。
NE	NE比較演算子。左辺と右辺が等しくない場合に真になります。
GE	GE比較演算子。左辺が右辺以上の場合に真になります。
GT	GT比較演算子。左辺が右辺より大きい場合に真になります。
LE	LE比較演算子。左辺が右辺以下の場合に真になります。
LT	LT比較演算子。左辺が右辺より小さい場合に真になります。

- 使用例

以下は、BEGIN文の使用例です。入力されたOCDATEの値が「19980507」の場合に、COMPパラメータの条件式が真になってドメインが動作し、SETVARに指定したTBB変数が設定されます。

```
//%OPC BEGIN ACTION=INCLUDE, COMP(&OCDATE..EQ.(19980507))
//%OPC SETVAR TBB=(OCDATE+1YR)
//%OPC END ACTION=INCLUDE
```

4.3.5. END

ドメインの最後を指定します。

- 使用方法

```
//%OPC END ACTION={INCLUDE|EXCLUDE|NOSCAN}
```

項目	説明
INCLUDE	ドメインをジョブに含めます。
EXCLUDE	ドメインをジョブから除外します。現在はサポートしていません。指定する場合は、INCLUDEを指定した場合と同様に動作します。
NOSCAN	ドメインに含まれた領域が置換されないようにします。現在はサポートしていません。指定する場合は、INCLUDEを指定した場合と同様に動作します。

索引

シンボル

%%制御文オペレーション

GLOBAL, 163

IF/ELSE/ENDIF, 163

SET, 161

オペレーション, 160

%%制御文システム変数

システム変数, 160

C

CNTL文, 14

D

DD文, 14

ACCODE, 20

AMP, 20

AVGREC, 20

BLKSIZE, 21

BLKSZLIM, 22

BURST, 22

CCSID, 22

CHARS, 23

CHKPT, 23

CNTL, 23

COPIES, 24

DATACLAS, 24

DCB, 25

DDNAME, 28

DEST, 29

DISP, 30

DLM, 32

DSID, 33

DSNAME/DSN, 33

DSNTYPE, 37

DUMMY, 38

DYNAM, 38

EXPDT, 38

FCB, 39

FILEDATA, 39

FLASH, 39

FREE, 40

HOLD, 40

KEYLEN, 41

KEYOFF, 41

LABEL, 42

LGSTREAM, 43

LIKE, 43

LRECL, 44

MGMTCLAS, 45

MODIFY, 46

OUTLIM, 46

OUTPUT, 47

PATH, 48

PATHDISP, 48

PATHMODE, 49

PATHOPTS, 50

PROTECT, 51

RECFM, 51

RECORD, 52

REFDD, 52

RETPD, 53

RLS, 54

SECMODEL, 54

SEGMENT, 54

SPACE, 54

SPIN, 56

STORCLAS, 56

SUBSYS, 57

SYMBOLS, 57

SYSOUT, 58

TERM, 59

UCS, 59

UNIT, 59

VOLUME/VOL, 60

アスタリスク(*)とDATA, 19

オペランド, 16

E

ENDCNTL文, 65
EXEC文, 65
 ACCT, 67
 ADDRSPC, 67
 CCSID, 67
 COND, 68
 DYNAMNBR, 71
 MEMLIMIT, 72
 PARM, 72
 PERFORM, 77
 PGM, 73
 PROC, 75
 RD, 78
 REGION, 78
 RLSTMOUT, 78
 SPARM, 79
 TIME, 79
 記号パラメータ, 80
 オペランド, 66

I

IF/THEN/ELSE/ENDIF文, 81
INCLUDE文, 85

J

JCL %%制御文
 %%制御文, 159
JCL OPC制御文
 OPC制御文, 164
JCLLIB文, 87
JCLコマンド文, 86
 S, 87
 コマンド, 87
JCL文
 CNTL文, 14
 DD文, 14
 ENDCNTL文, 65
 EXEC文, 65
 IF/THEN/ELSE/ENDIF文, 81
 INCLUDE文, 85
 JCLLIB文, 87

JCLコマンド文, 86
JES2コマンド文, 144
JOB文, 88
OUTPUT文, 109
PEND文, 139
PROC文, 140
SET文, 142
空文, 144
コメント文, 145
スペシャルDD文, 62
段落見出し, 144
JCL文の記述手順, 10
JES2 JCL文
 JES2コマンド文, 147
 JOBPARM文, 149
 MESSAGE文, 154
 NETACCT文, 154
 NOTIFY文, 154
 OUTPUT文, 155
 PRIORITY文, 155
 ROUTE文, 156
 SETUP文, 157
 SIGNOFF文, 157
 SIGNON文, 157
 XEQ文, 157
 XMIT文, 158
JES2 JCL文の記述手順, 11
JES2コマンド文, 147
 /DBR
 /STA(/START), 149
 S, 149
 コマンド, 148
JOBPARM文, 149
 BURST/B, 150
 BYTES/M, 151
 CARDS/C, 151
 COPIES/N, 151
 FORMS/F, 151
 LINECT/K, 151
 LINES/L, 152
 PAGES/G, 152
 PROCLIB/P, 152
 RESTART/E, 153

ROOM/R, 153
SYSAFF/S, 153
TIME/T, 154
オペランド, 150

JOB文, 88

ADDRSPC, 90
BYTES, 90
CARDS, 91
CCSID, 91
CLASS, 91
COND, 92
GROUP, 94
JESLOG, 94
LINES, 94
MEMLIMIT, 95
MSGCLASS, 95
MSGLEVEL, 96
NOTIFY, 96
PAGES, 97
PASSWORD, 97
PERFORM, 98
PRTY, 98
RD, 99
REGION, 99
RESTART, 100
SCHENV, 101
SECLABEL, 100
SPARM, 101
TIME, 102
TIMECONTROLLER, 103
TYPRUN, 105
USER, 108
オペランド, 89

M

MESSAGE文, 154

N

NETACCT文, 154
NOTIFY文, 154

O

OPC制御文のオペレーション

BEGIN, 167
END, 167
SCAN, 165
SETFORM, 165
SETVAR, 166
オペレーション, 164

OUTPUT文, 109, 155

ADDRESS, 112
AFPSTATS, 113
BUILDING, 113
BURST, 114
CHARS, 114
CKPTLINE, 114
CKPTPAGE, 115
CKPTSEC, 115
CLASS, 115
COLORMAP, 116
COMPACT, 116
COMSETUP, 116
CONTROL, 116
COPIES, 117
DATAACK, 117
DEFAULT, 117
DEPT, 118
DEST, 118
DPAGELBL, 119
DUPLEX, 119
FCB, 119
FLASH, 119
FORMDEF, 120
FORMLEN, 121
FORMS, 121
FSSDATA, 121
GROUPID, 121
INDEX, 122
INTRAY, 122
JESDS, 122
LINDEX, 122
LINECT, 123
MAILBCC, 123

MAILCC, 124
MAILFILE, 124
MAILFROM, 124
MAILTO, 125
MODIFY, 125
NAME, 126
NOTIFY, 127
OFFSETXB, 127
OFFSETXF, 127
OFFSETYB, 127
OFFSETYF, 127
OUTBIN, 128
OUTDISP, 128
OVERLAYB, 129
OVERLAYF, 130
OVFL, 130
PAGEDEF, 130
PIMSG, 131
PORTNO, 131
PRMODE, 131
PRTATTRS, 132
PRTEROR, 132
PRTOPTNS, 132
PRTQUEUE, 132
PRTY, 133
REPLYTO, 133
RESFMT, 133
RETAINF, 134
RETAINS, 134
RETRYL, 134
RETRYT, 135
ROOM, 135
SYSAREA, 136
THRESHLD, 136
TITLE, 136
TRC, 137
UCS, 137
USERDATA, 138
USERLIB, 138
USERPATH, 139
WRITER, 139
オペランド, 109

P

PEND文, 139
PRIORITY文, 155
優先順位, 155
PROC文, 140

R

ROUTE文, 156

S

SETUP文, 157
SET文, 142
SIGNOFF文, 157
SIGNON文, 157

X

XEQ文, 157
XMIT文, 144, 158

か

空文, 144
コメント文, 145

さ

スペシャルDD文, 62
JOB CAT DD, 64
JOB LIB DD, 63
STEP CAT DD, 64
STEP LIB DD, 63
SYS IN DD, 62
SYS OUT DD, 62
SYS PRINT DD, 62

た

段落見出し, 144